

深度学习

张烨菲

2025年11月

第八章 卷积网络

- 8.1 卷积运算

- 8.2 边缘扩充

- 8.3 卷积步长

- 8.4 三维卷积

8.5 卷积层

8.6 简单卷积网络

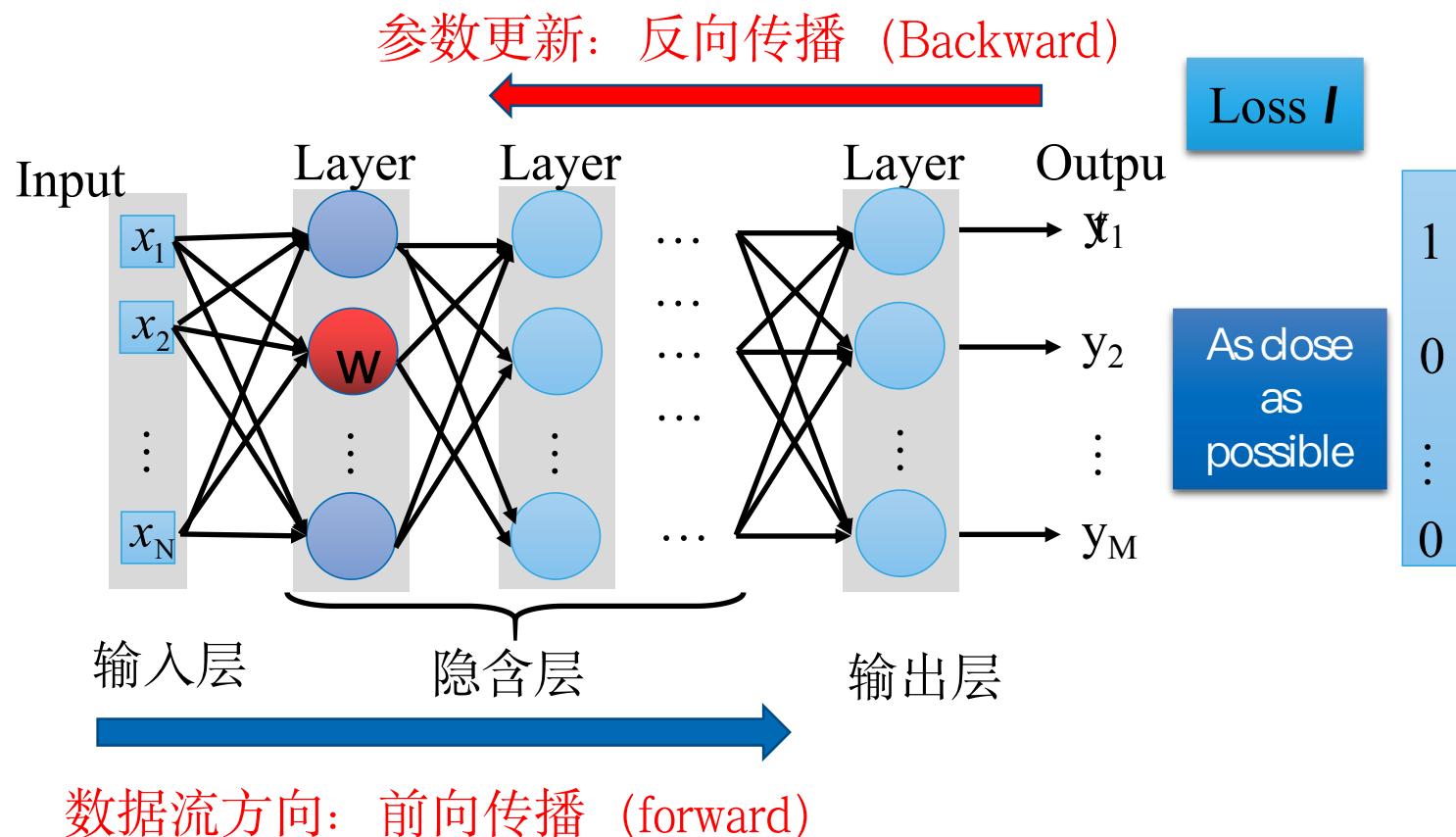
8.7 池化层

8.8 经典卷积神经网络

知识点回顾：深度前馈神经网络

➤ 模型架构设计：

- 隐藏层
 - 代价函数
 - 输出单元
 - 激活函数
 - 数据流方向：前向传播
- 参数更新方向：反向传播



8 卷积网络

卷积网络 (convolutional network) (LeCun, 1989), 也叫做卷积神经网络 (convolutional neural network, CNN), 是一种专门用来处理具有类似网格结构的数据的神经网络。例如时间序列数据 (可以认为是在时间轴上有规律地采样形成的一维网格) 和图像数据 (可以看作是二维的像素网格)。卷积网络在诸多应用领域都表现优异。“卷积神经网络”一词表明该网络使用了卷积 (convolution) 这种数学运算。卷积是一种特殊的线性运算。卷积网络是指那些至少在网络的一层中使用卷积运算来替代一般的矩阵乘法运算的神经网络。

8.1 卷积运算

5

$$y(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m) = x(n) * h(n)$$

计算步骤如下：

(1) 翻褶：先在坐标轴 m 上画出 $x(m)$ 和 $h(m)$ ，将 $h(m)$ 以纵坐标为对称轴折叠成 $h(-m)$ 。

(2) 移位：将 $h(-m)$ 移位 n ，得 $h(n-m)$ 。当 n 为正数时，右移 n ；当 n 为负数时，左移 n 。

(3) 相乘：将 $h(n-m)$ 和 $x(m)$ 的对应序列值相乘。

(4) 相加：把所有的乘积累加起来，即得 $y(n)$ 。

8.1 卷积运算

6

$$y(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m) = x(n) * h(n)$$

例 已知 $x(n)$ 和 $h(n)$ 分别为：

$$x(n) = \begin{cases} 1, & 0 \leq n \leq 4 \\ 0, & \text{其它} \end{cases}$$

和

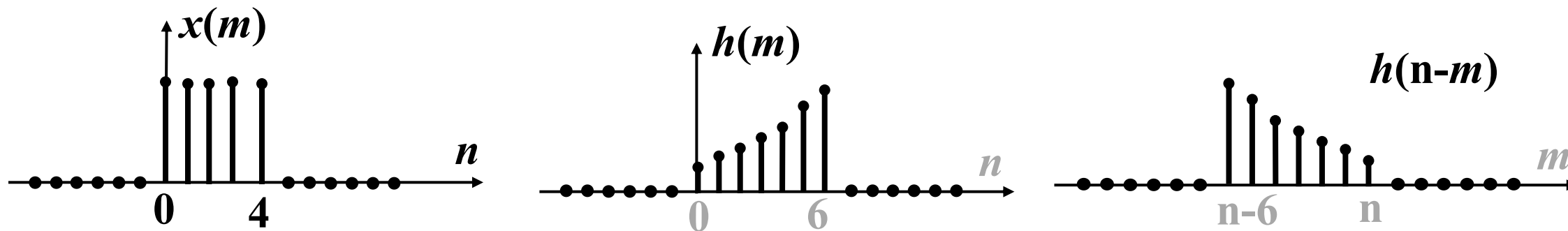
$$h(n) = \begin{cases} a^n, & 0 \leq n \leq 6 \\ 0, & \text{其它} \end{cases}$$

a 为常数，且 $1 < a$ ，试求 $x(n)$ 和 $h(n)$ 的卷积。

8.1 卷积运算

7

解 参看图，分段考虑如下：



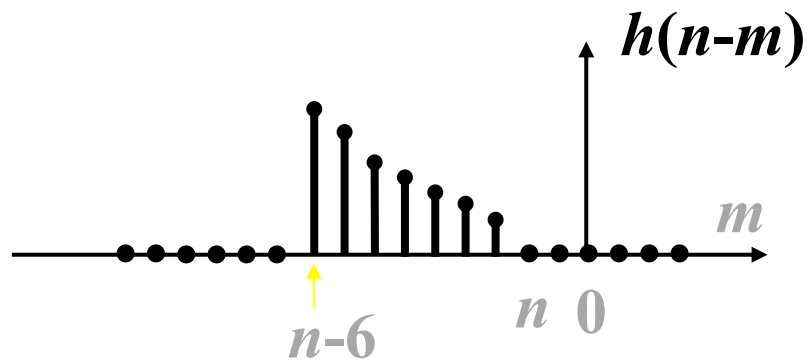
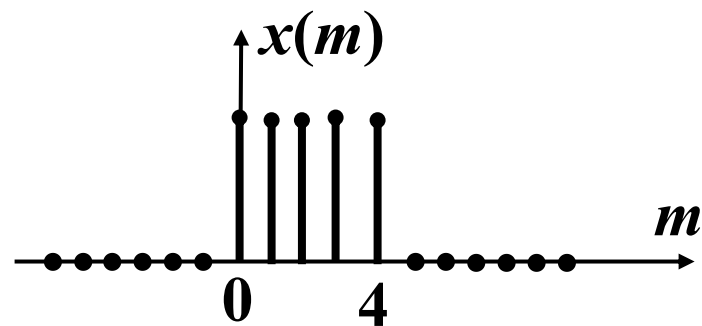
- (1) 对于 $n < 0$;
- (2) 对于 $0 \leq n \leq 4$;
- (3) 对于 $n > 4$, 且 $n-6 \leq 0$, 即 $4 < n \leq 6$;
- (4) 对于 $n > 6$, 且 $n-6 \leq 4$, 即 $6 < n \leq 10$;
- (5) 对于 $(n-6) > 4$, 即 $n > 10$.

8.1 卷积运算

8

(1) $n < 0$

$$y(n) = x(n) * h(n) = 0$$

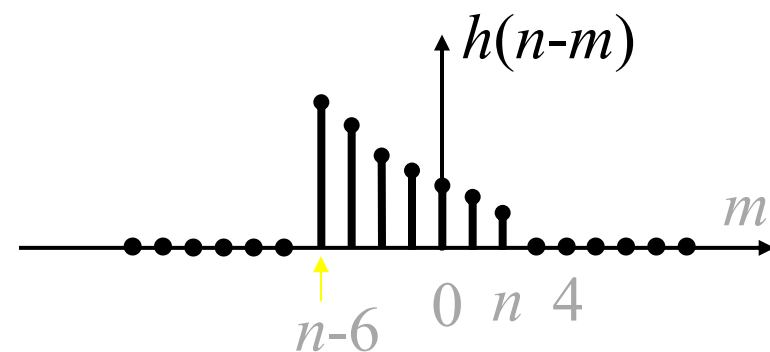
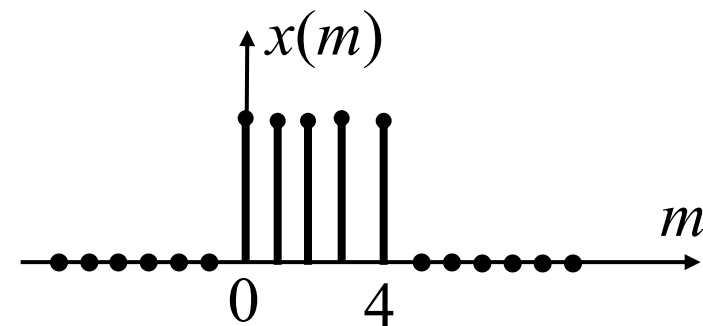


8.1 卷积运算

9

(2) 在 $0 \leq n \leq 4$ 区间上

$$\begin{aligned} y(n) &= \sum_{m=0}^n x(m)h(n-m) = \sum_{m=0}^n 1 \cdot a^{n-m} \\ &= a^n \sum_{m=0}^n a^{-m} = a^n \frac{1 - a^{-(n+1)}}{1 - a^{-1}} = \frac{1 - a^{1+n}}{1 - a} \end{aligned}$$

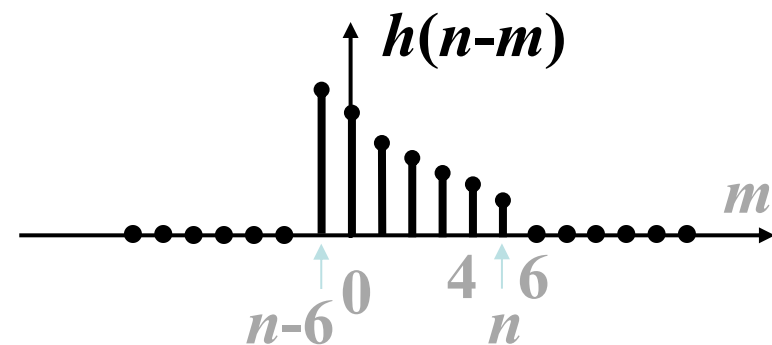
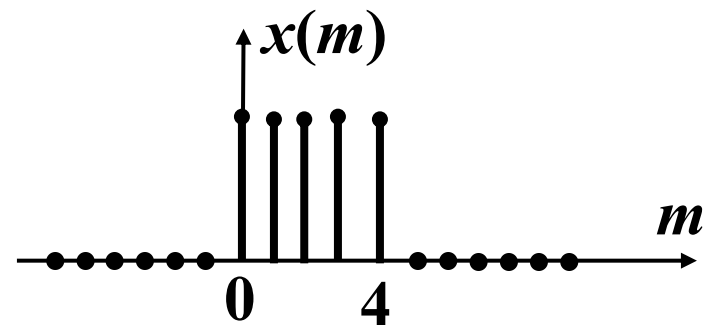


8.1 卷积运算

10

(3) 在 $4 < n \leq 6$ 区间上

$$\begin{aligned} y(n) &= \sum_{m=0}^4 x(m)h(n-m) \\ &= \sum_{m=0}^4 1 \cdot a^{n-m} = a^n \sum_{m=0}^4 a^{-m} \\ &= a^n \frac{1 - a^{-(1+4)}}{1 - a^{-1}} = \frac{a^{n-4} - a^{1+n}}{1 - a} \end{aligned}$$

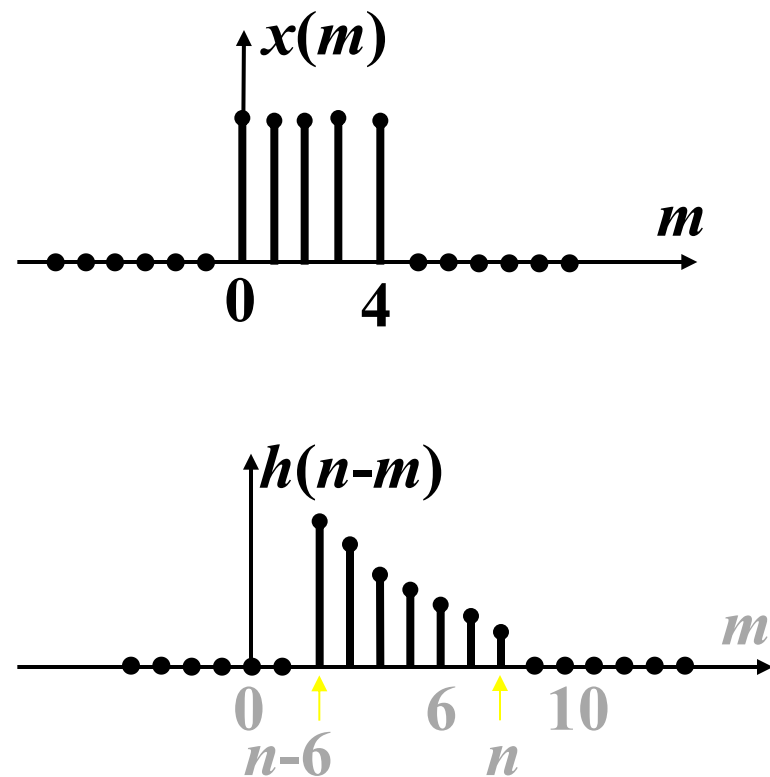


8.1 卷积运算

11

(4) 在 $6 < n \leq 10$ 区间上

$$\begin{aligned}\therefore y(n) &= \sum_{m=n-6}^n x(m)h(n-m) \\ &= \sum_{m=n-6}^n 1 \cdot a^{n-m} = a^n \sum_{m=n-6}^4 a^{-m} \\ &= a^n \frac{a^{-(n-6)} - a^{-(4+1)}}{1 - a^{-1}} = \frac{a^{n-4} - a^7}{1 - a}\end{aligned}$$



8.1 卷积运算

12

综合以上结果， $y(n)$ 可归纳如下：

$$y(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m) = x(n) * h(n)$$

$$x(n) = \begin{cases} 1, & 0 \leq n \leq 4 \\ 0, & \text{其它} \end{cases}$$

$$h(n) = \begin{cases} a^n, & 0 \leq n \leq 6 \\ 0, & \text{其它} \end{cases}$$

$$y(n) = \begin{cases} 0, & n < 0 \\ \frac{1-a^{1+n}}{1-a}, & 0 \leq n \leq 4 \\ \frac{a^{n-4}-a^{1+n}}{1-a}, & 4 < n \leq 6 \\ \frac{a^{n-4}-a^7}{1-a}, & 6 < n \leq 10 \\ 0, & 10 < n \end{cases}$$

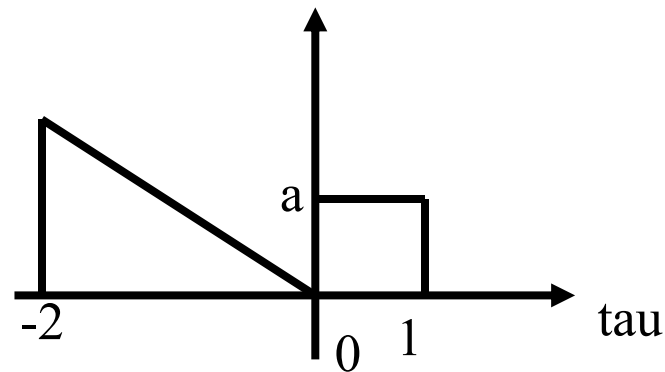
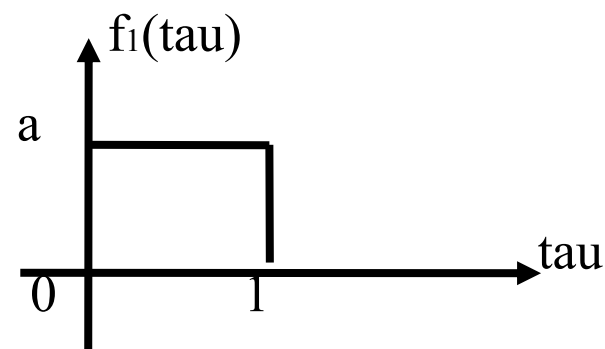
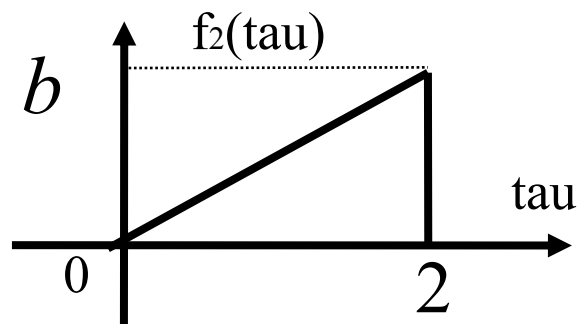
8.1 卷积运算

13

计算 $f_1 * f_2 = \int_{-\infty}^{\infty} f_1(\tau) f_2(t - \tau) d\tau$

$$f_1(t) = a \quad 0 < t < 1$$

$$f_2(t) = \frac{b}{2}t \quad 0 < t < 2$$



8.1 卷积运算

14

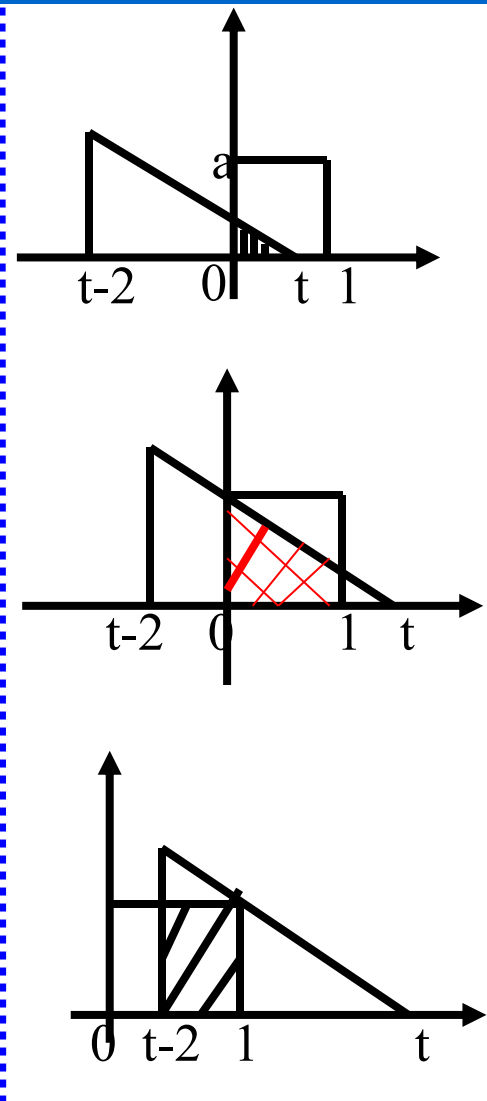
1. $-\infty \leq t \leq 0$

重合面积为零: $f_1(t) * f_2(t) = 0$

2. *if* $0 \leq t \leq 1$

$$f_1 * f_2 = \int_{-\infty}^{\infty} f_1(\tau) f_2(t - \tau) d\tau$$

$$= \int_0^t a \frac{b}{2} (t - \tau) d\tau = -\frac{ab}{4} (t - \tau)^2 \Big|_0^t = \frac{ab}{4} t^2$$

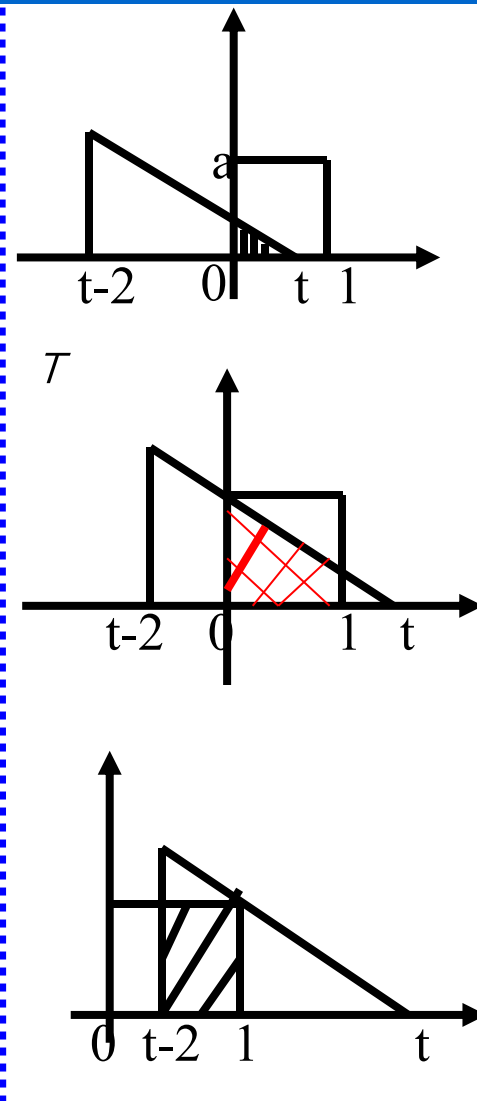


8.1 卷积运算

15

3. *if* $1 \leq t \leq 2$

$$\begin{aligned} f_1 * f_2 &= \int_0^1 a \frac{b}{2} (t - \tau) d\tau = -\frac{ab}{4} (t - \tau)^2 \Big|_0^1 \\ &= \frac{ab}{4} (2t - 1) \end{aligned}$$



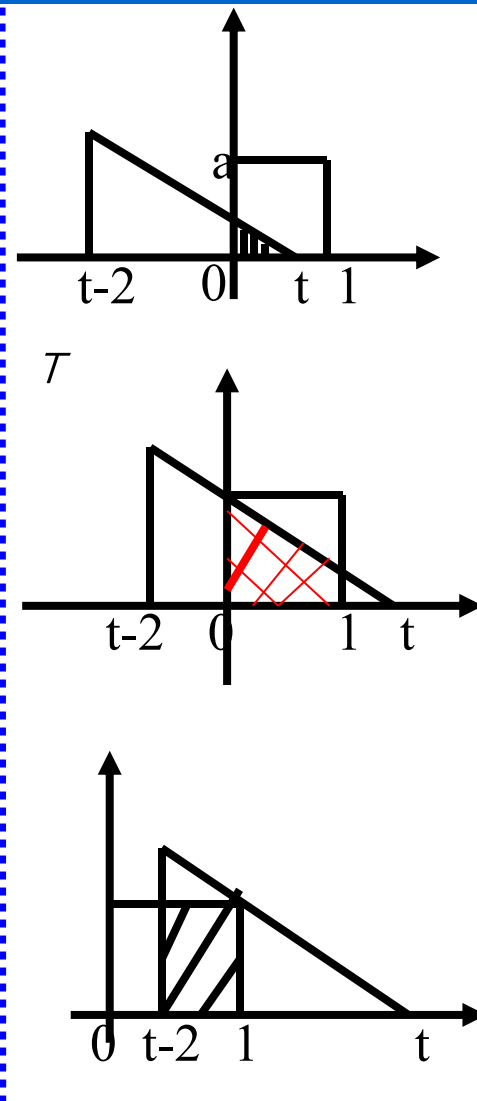
8.1 卷积运算

16

4. *if* $2 \leq t \leq 3$

$$\begin{aligned} f_1 * f_2 &= \int_{t-2}^1 a \frac{b}{2} (t - \tau) d\tau \\ &= -\frac{ab}{4} (t - \tau)^2 \Big|_{t-2}^1 = \frac{ab}{4} (3 + 2t - t^2) \end{aligned}$$

5. *if* $3 \leq t \leq \infty$ $f_1 * f_2 = 0$



彻底理解卷积

59:52

【小动画】彻底理解卷积

【超形象】卷的由来，小元

12.0万 2020-03-31

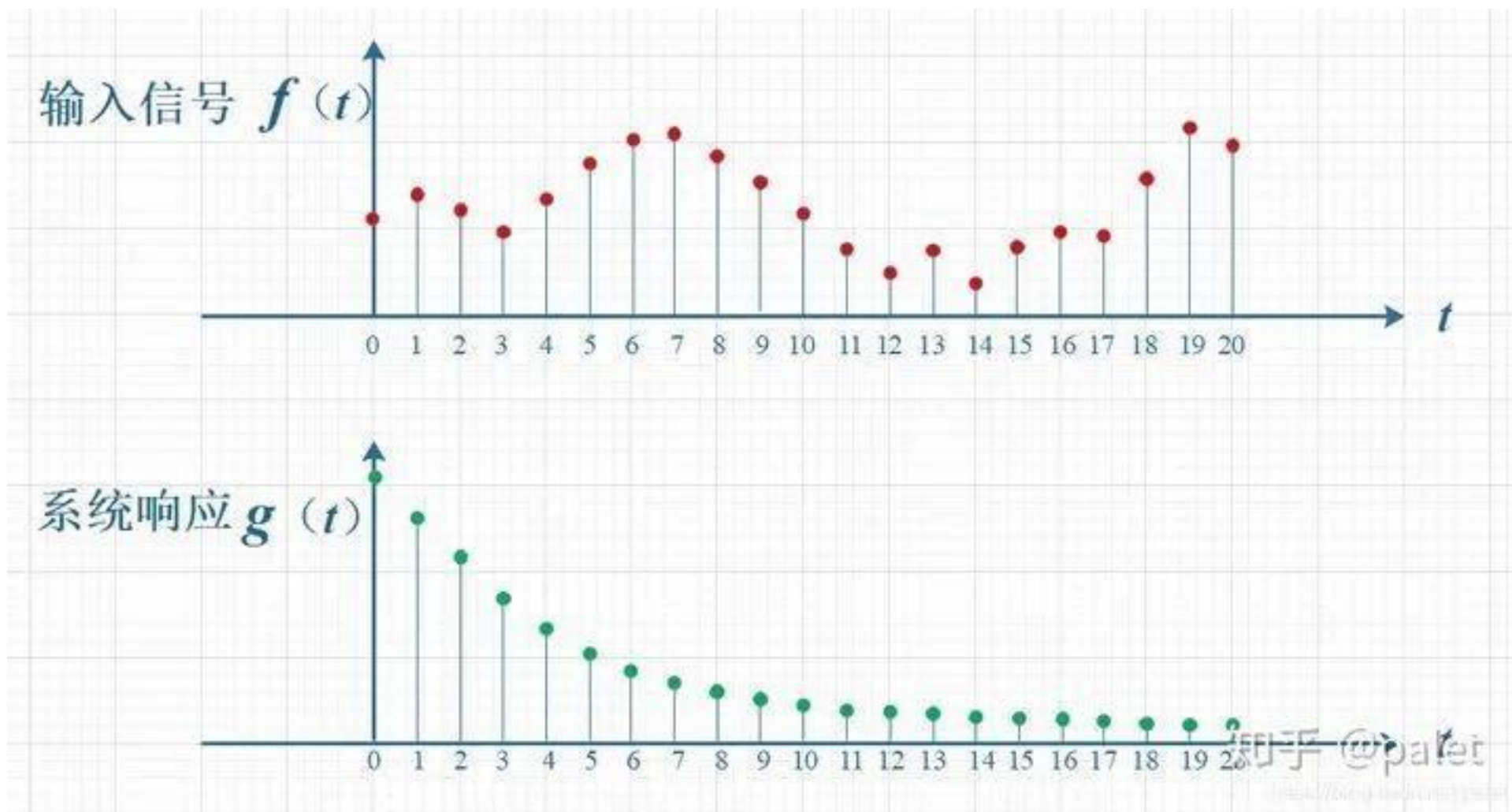
小元老师高数线代概率

我依然能搞明白卷积是什么

科学声音

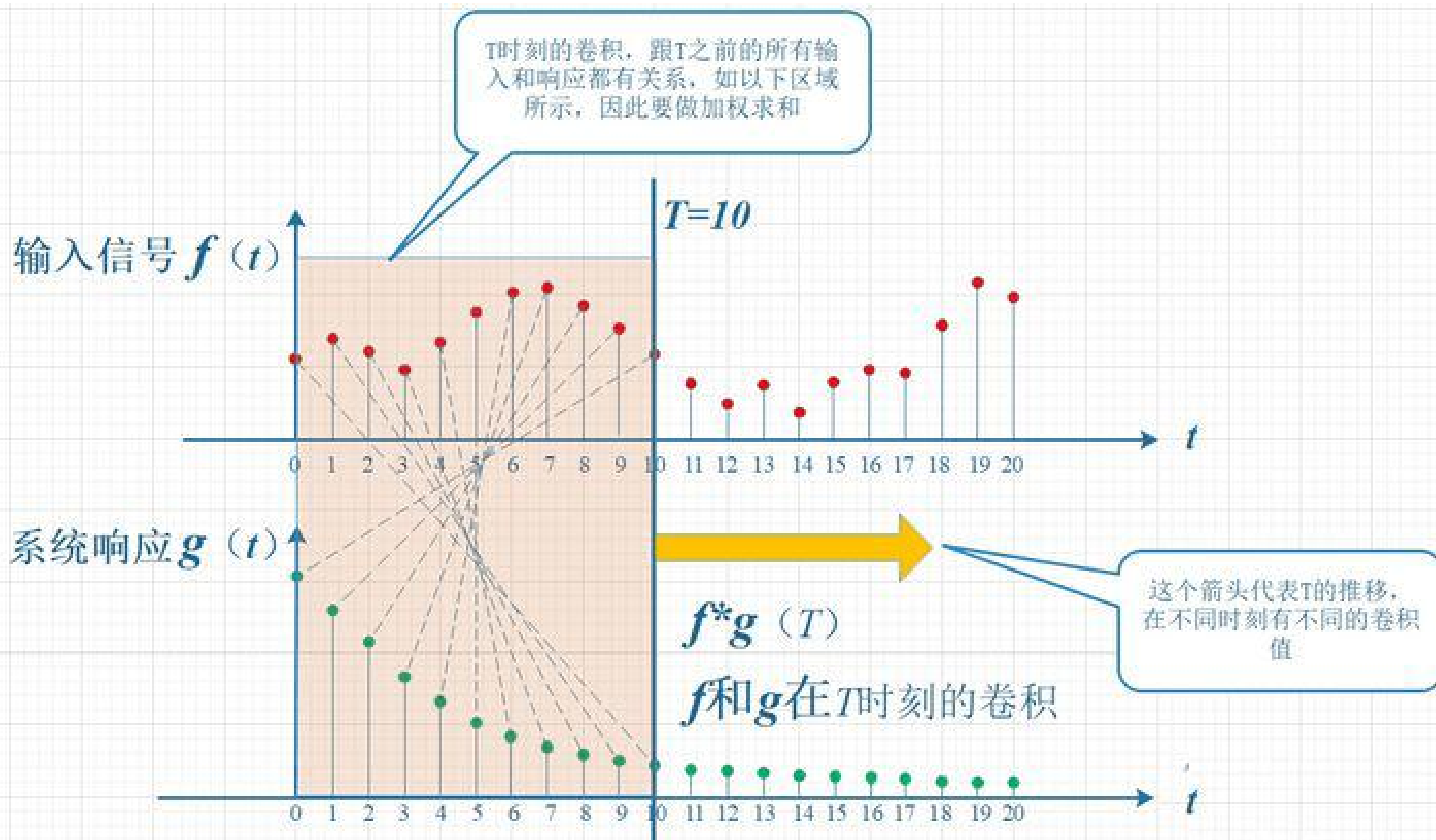
8.1 卷积运算

18



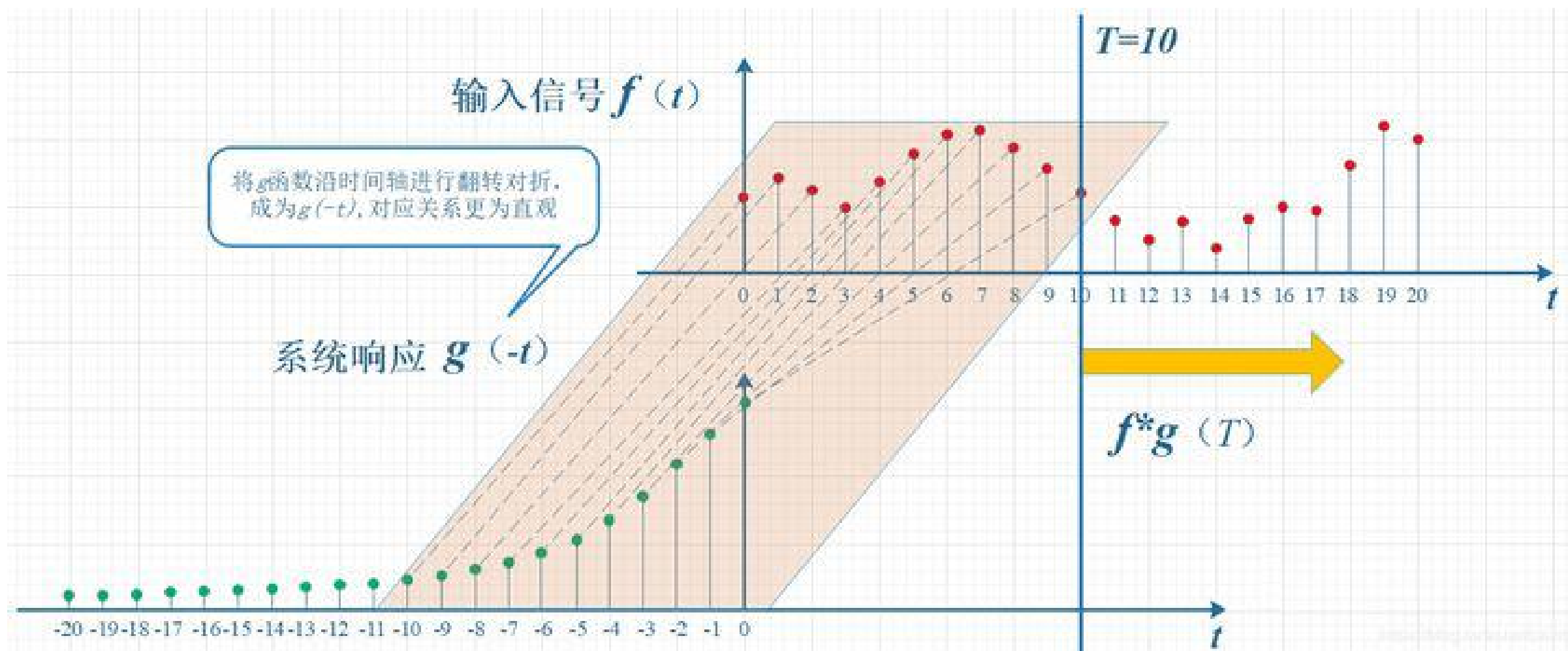
8.1 卷积运算

19



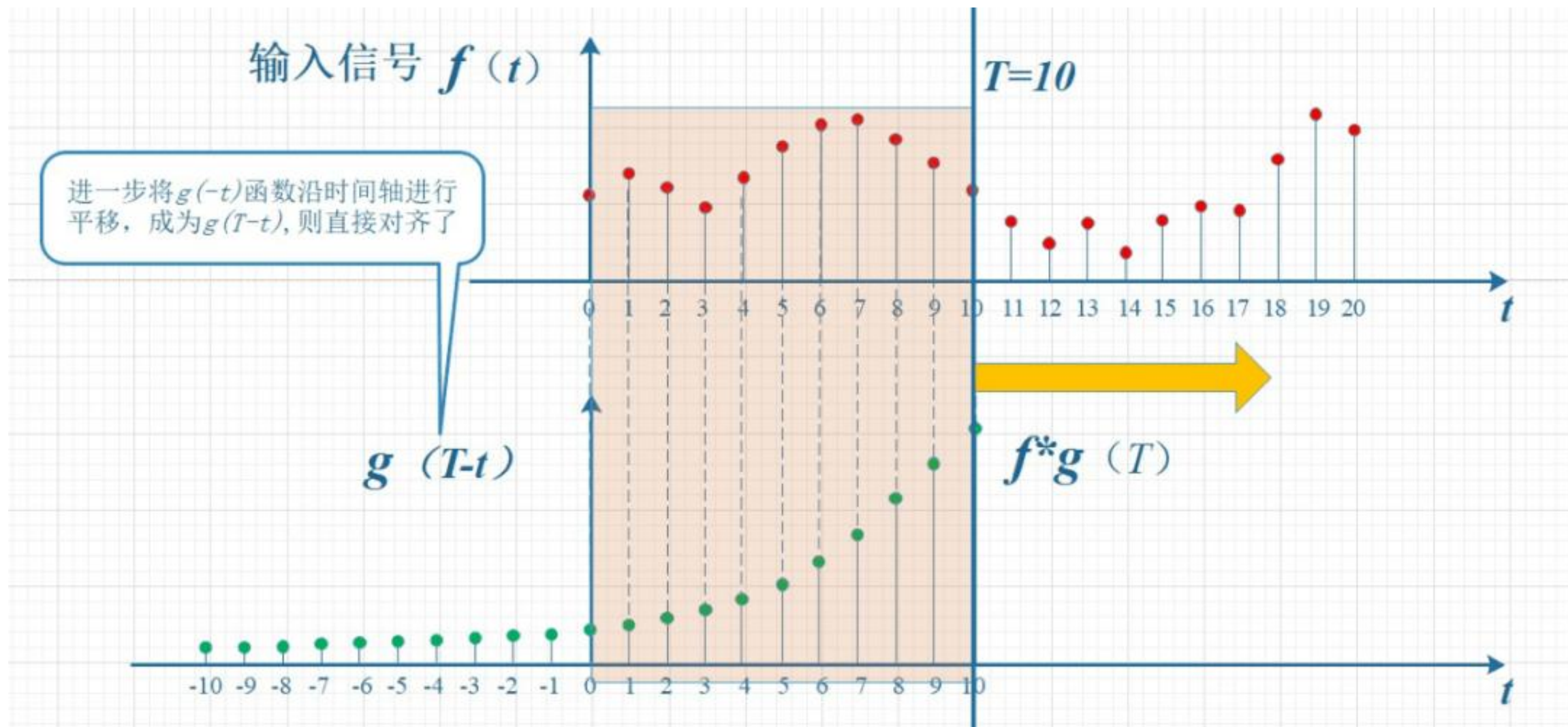
8.1 卷积运算

20



8.1 卷积运算

21



8.1 卷积运算

22

假设我们正在用激光传感器追踪一艘宇宙飞船的位置。我们的激光传感器给出一个单独的输出 $x(t)$ ，表示宇宙飞船在时刻 t 的位置。 x 和 t 都是实值的，这意味着我们可以在任意时刻从传感器中读出飞船的位置。

8.1 卷积运算

23

现在假设我们的传感器受到一定程度的噪声干扰。为了得到飞船位置的低噪声估计，我们对得到的测量结果进行平均。显然，时间上越近的测量结果越相关，所以我们采用一种加权平均的方法，对于最近的测量结果赋予更高的权重。我们可以采用一个加权函数 $w(a)$ 来实现，其中 a 表示测量结果距当前时刻的时间间隔。如果我们对任意时刻都采用这种加权平均的操作，就得到了一个新的对于飞船位置的平滑估计函数 s ：

$$s(t) = \int x(a)w(t-a)da.$$

8.1 卷积运算

24

$$s(t) = \int x(a)w(t-a)da.$$

在卷积网络的术语中，卷积的第一个参数（在这个例子中，函数 x ）通常叫做输入（input），第二个参数（函数 w ）叫做核函数（kernel function）。输出有时被称作特征映射（feature map）。

8.1 卷积运算

25

在本例中，激光传感器在每个瞬间反馈测量结果的想法是不切实际的。一般地，当我们用计算机处理数据时，时间会被离散化，传感器会定期地反馈数据。所以在我们的例子中，假设传感器每秒反馈一次测量结果是比较现实的。这样，时刻 t 只能取整数值。如果我们假设 x 和 w 都定义在整数时刻 t 上，就可以定义离散形式的卷积：

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a).$$

8.1 卷积运算

26

最后，我们经常一次在多个维度上进行卷积运算。例如，如果把一张二维的图像 I 作为输入，我们也许也想要使用一个二维的核 K ：

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n) K(i - m, j - n).$$

卷积是可交换的 (commutative)，我们可以等价地写作：

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n).$$

8.1 卷积运算

27

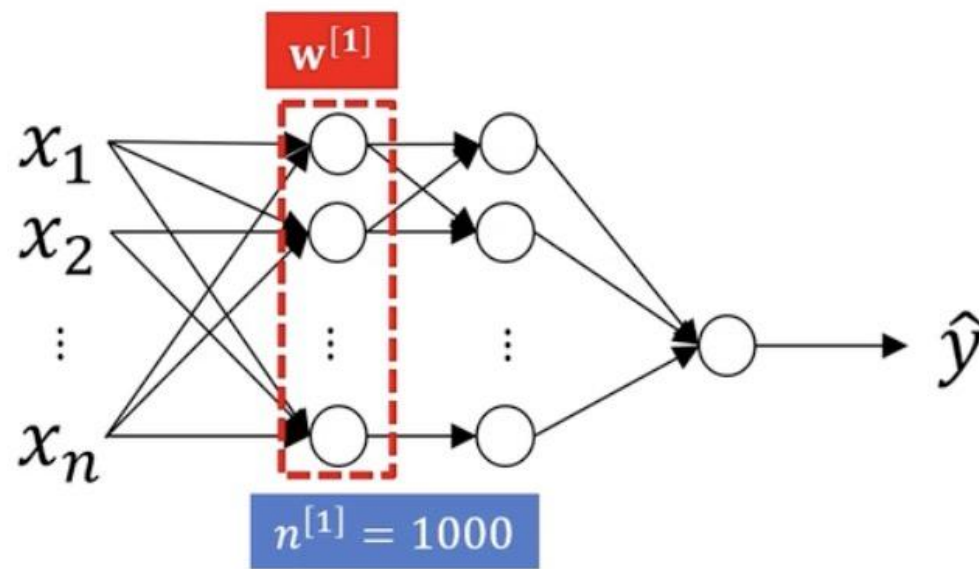


→ Cat?

64 X 64 X 3 = 12288



1000 X 1000 X 3 = 3,000,000



8.1 卷积运算

28



64 X 64 X 3 = 12288



1000 X 1000 X 3 = 3,000,000

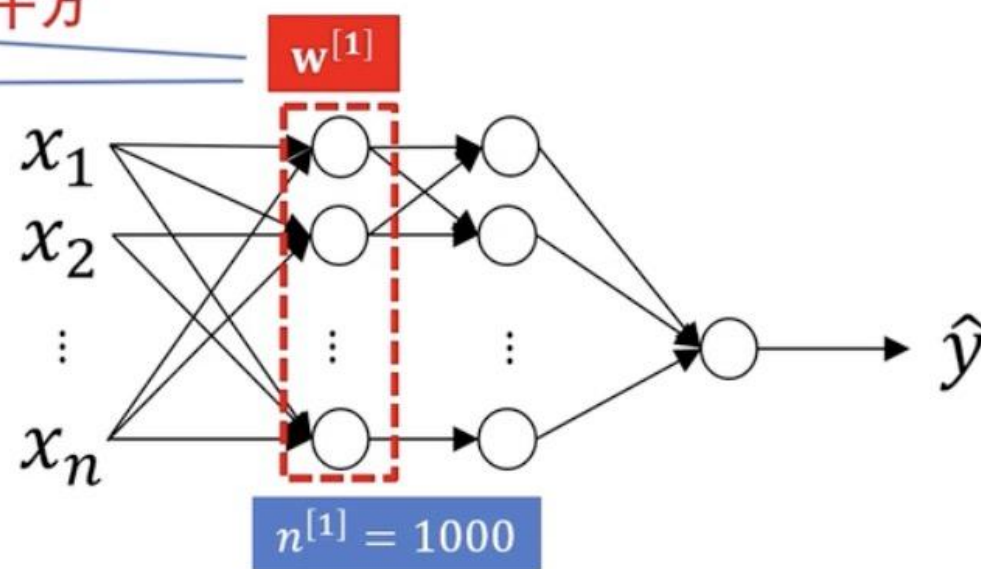
→ Cat? (0/1)

(1000, 12288)

≈ 1.2千万

(1000, 3 M)

= 30 亿



8.1 卷积运算

29

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6X6 X1 灰度图像

*

1	0	-1
1	0	-1
1	0	-1

3X3

过滤器

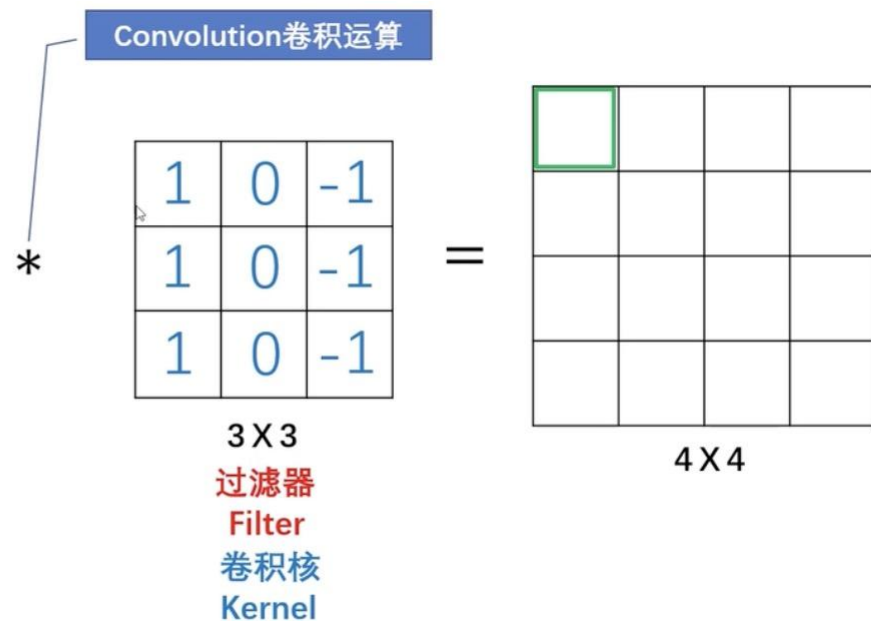
Filter

8.1 卷积运算

30

3 ¹	0 ⁰	1 ⁻¹	2	7	4
1 ¹	5 ⁰	8 ⁻¹	9	3	1
2 ¹	7 ⁰	2 ⁻¹	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6X6 X1 灰度图像



8.1 卷积运算

31

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

3 ¹	0 ⁰	1 ⁻¹	2	7	4
1 ¹	5 ⁰	8 ⁻¹	9	3	1
2 ¹	7 ⁰	2 ⁻¹	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6X6 X1 灰度图像

Convolution卷积运算

*

1	0	-1
1	0	-1
1	0	-1

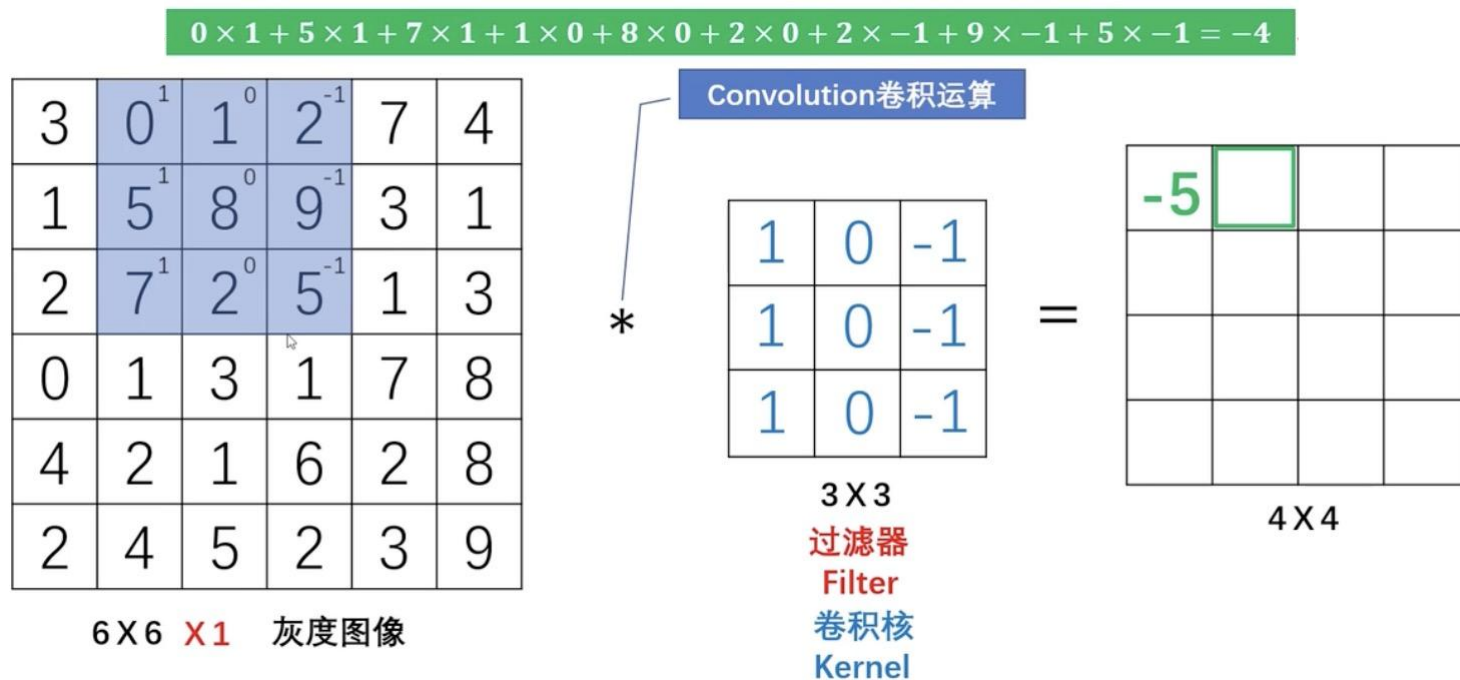
3X3
过滤器
Filter
卷积核
Kernel

=

4X4

8.1 卷积运算

32



8.1 卷积运算

33

3	0	1	2	7	4
1	5	8	9 ¹	3 ⁰	1 ⁻¹
2	7	2	5 ¹	1 ⁰	3 ⁻¹
0	1	3	1 ¹	7 ⁰	8 ⁻¹
4	2	1	6	2	8
2	4	5	2	3	9

6 X 6 X 1 灰度图像

Convolution卷积运算

*

1	0	-1
1	0	-1
1	0	-1

3 X 3
过滤器
Filter
卷积核
Kernel

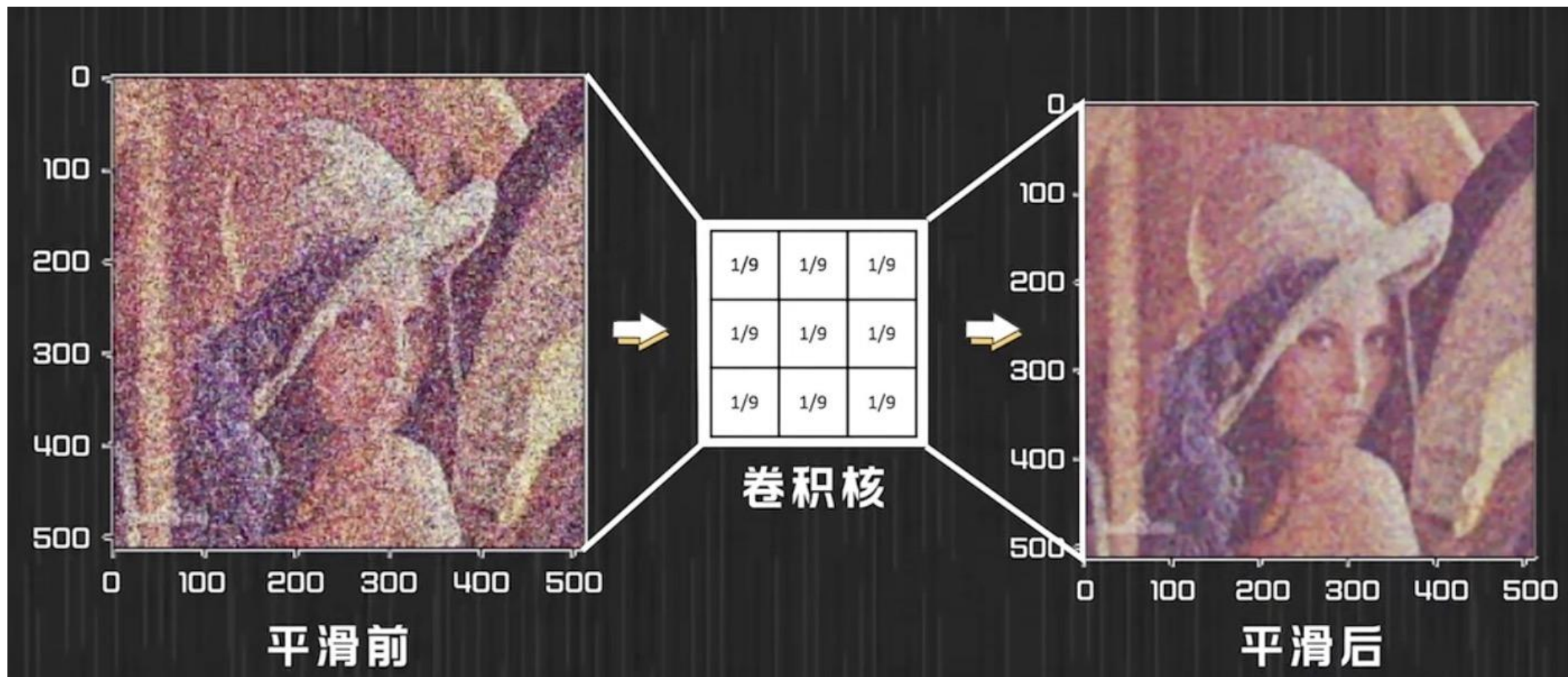
=

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4 X 4

8.1 卷积运算

34



8.1 卷积运算

35



vertical edges



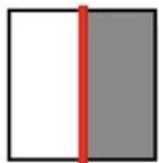
horizontal edges

8.1 卷积运算

36

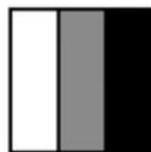
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

6 X 6 X 1 灰度图像



*

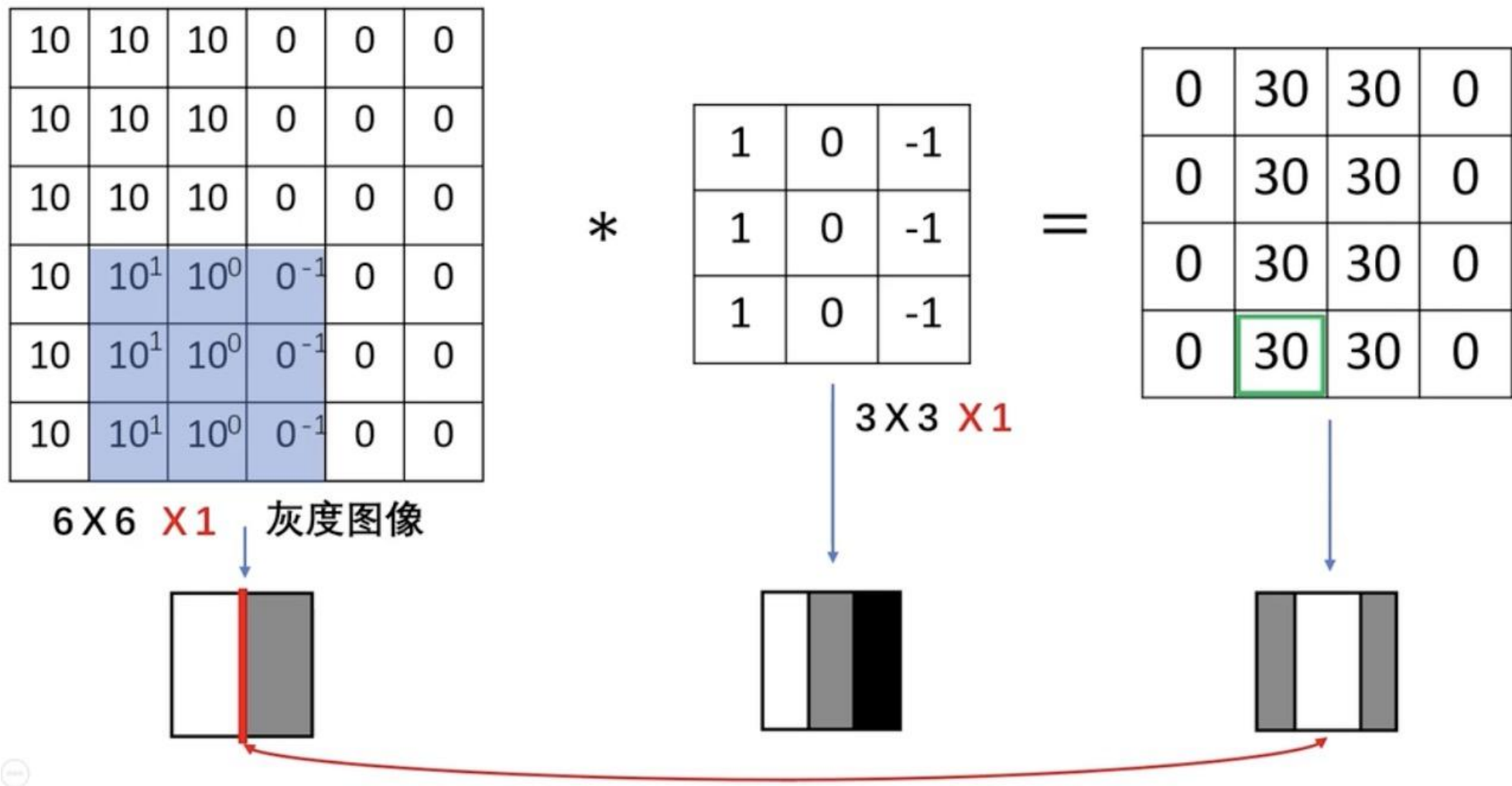
1	0	-1
1	0	-1
1	0	-1



2

8.1 卷积运算

37



8.1 卷积运算

38

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



纵向边缘检测，由亮到暗

0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10



*

1	0	-1
1	0	-1
1	0	-1



=

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0



纵向边缘检测，由暗到亮

8.1 卷积运算

39

1	0	-1
1	0	-1
1	0	-1

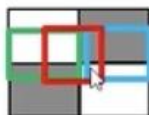
Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

10	10	10	0	0	0
10^1	10^1	10^1	0^1	0^1	0^1
10^0	10^0	10^0	0^0	0^0	0^0
0^{-1}	0^{-1}	0^{-1}	10^{-1}	10^{-1}	10^{-1}
0	0	0	10	10	10
0	0	0	10	10	10

*



1	1	1
0	0	0
-1	-1	-1

=

0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

8.1 卷积运算

40

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

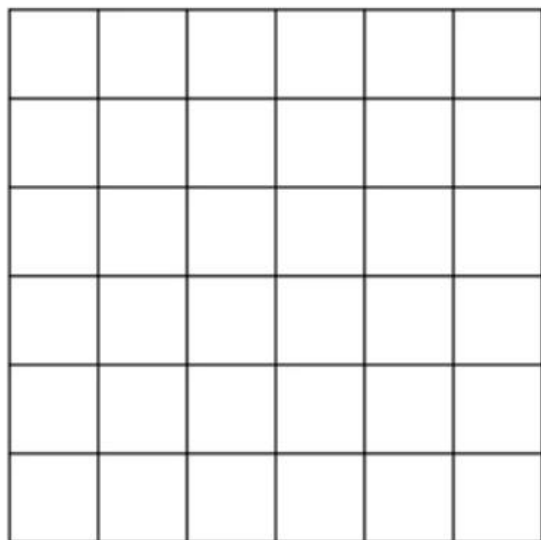
w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

=

45^0
 75^0

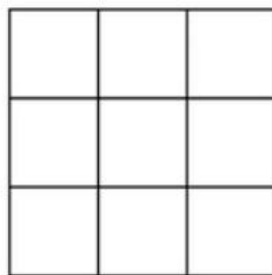
8.2 边缘扩充

41



6×6
 $n \times n$

*

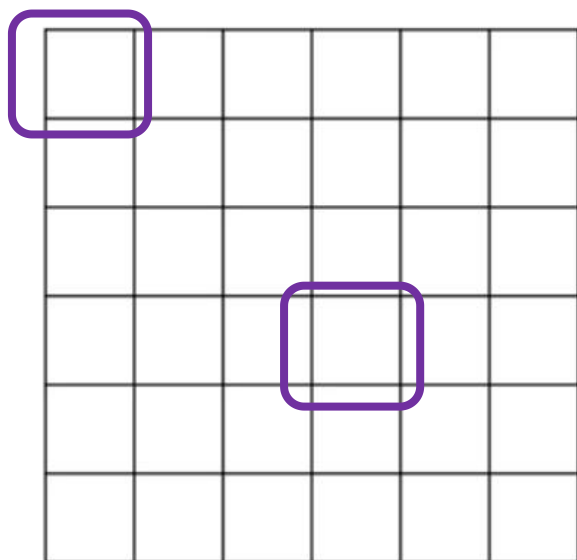


3×3
 $f \times f$

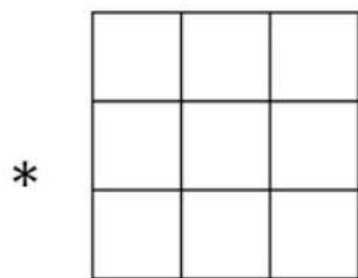
=

8.2 边缘扩充

42



6X6
 $n \times n$



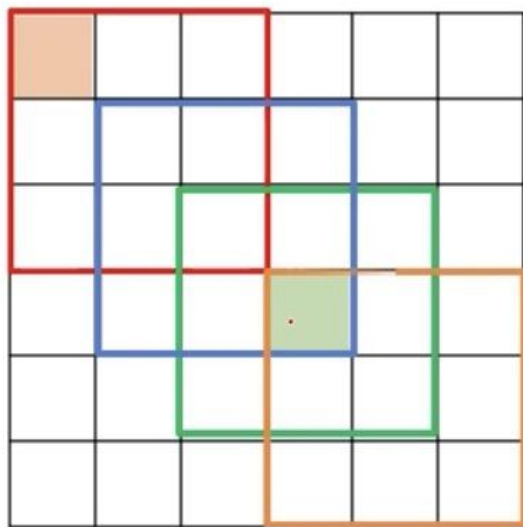
3X3
 $f \times f$

$(n - f + 1) \times (n - f + 1)$

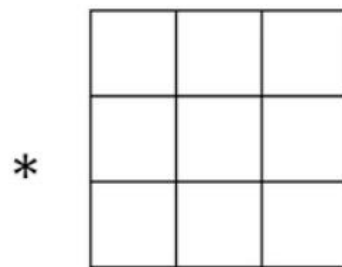
 4X4

8.2 边缘扩充

43



6×6
 $n \times n$



*

=

3×3
 $f \times f$

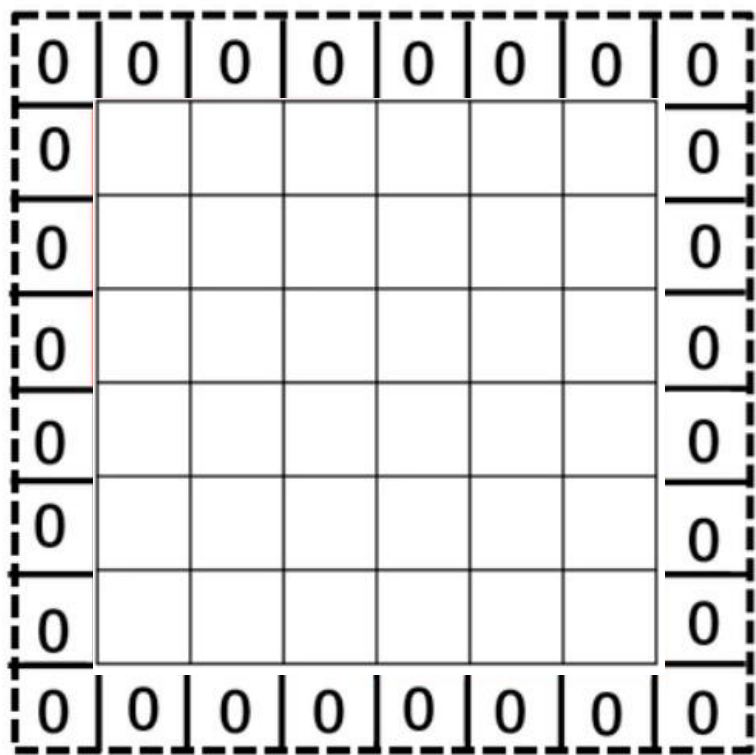
$(n - f + 1) \times (n - f + 1)$

4×4



8.2 边缘扩充

44



$$p = \text{padding} = 1$$

$$\begin{matrix} * & \begin{matrix} \square & \square & \square \\ \square & \square & \square \\ \square & \square & \square \end{matrix} & = \end{matrix}$$

3×3
 $f \times f$

$$(n + 2p - f + 1) \times (n + 2p - f + 1)$$

8.2 边缘扩充

45

Valid : $n \times n$ * $f \times f$ $\longrightarrow$?

6×6 * 3×3 $\longrightarrow$?

Same : $n \times n$ * $f \times f$ $\longrightarrow$?

8.2 边缘扩充

46

$$\text{Valid : } n \times n * f \times f \longrightarrow (n - f + 1) \times (n - f + 1)$$
$$6 \times 6 * 3 \times 3 \longrightarrow 4 \times 4$$

$$\text{Same : } n \times n * f \times f \longrightarrow n \times n \quad (n + 2p - f + 1) \times (n + 2p - f + 1)$$

$$n + 2p - f + 1 = n \longrightarrow p = ?$$

$$f = 3 \times 3, \quad p = ?$$

$$f = 5 \times 5, \quad p = ?$$

8.2 边缘扩充

47

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

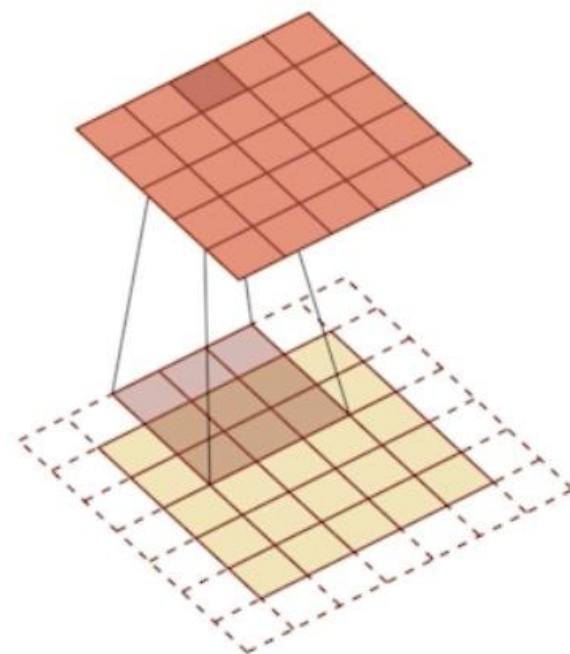
Padding=0

$*$

0	1
2	3

$=$

Output



8.2 边缘扩充

48

Input Kernel Output

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

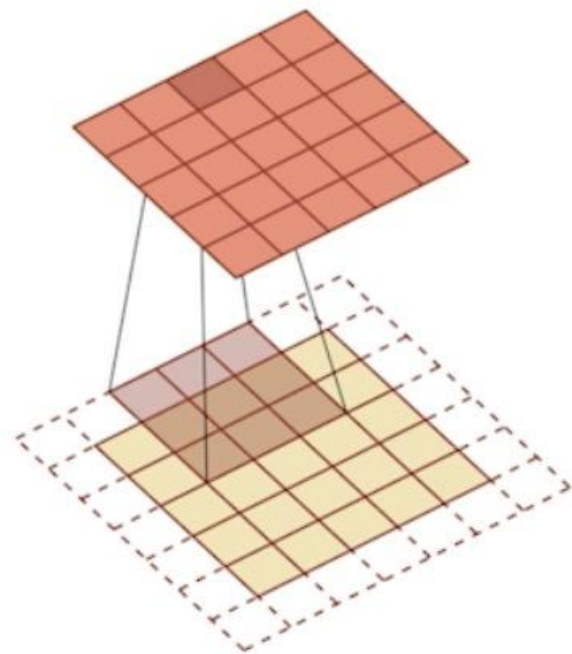
Padding=0

$*$

0	1
2	3

$=$

0	3	8	4
9	19	25	10
21	37	43	16
6	7	8	0



8.3 卷积步长

49

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

3×3

8.3 卷积步长

50

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

2^3	3^4	7^4	4	6	2	9
6^1	6^0	9^2	8	7	4	3
3^{-1}	4^0	8^3	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

=

91		

3×3

8.3 卷积步长

51

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

2	3	7 ³	4 ⁴	6 ⁴	2	9
6	6	9 ¹	8 ⁰	7 ²	4	3
3	4	8 ⁻¹	3 ⁰	8 ³	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3
Stride=2

=

91		

3×3

8.3 卷积步长

52

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

2	3	7	4	6 ³	2 ⁴	9 ⁴
6	6	9	8	7 ¹	4 ⁰	3 ²
3	4	8	3	8 ⁻¹	9 ⁰	7 ³
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3
Stride=2

=

91	100	83

3×3

8.3 卷积步长

53

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3 ³	4 ⁴	8 ⁴	3	8	9	7
7 ¹	8 ⁰	3 ²	6	6	3	4
4 ⁻¹	2 ⁰	1 ³	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7×7

*

3	4	4
1	0	2
-1	0	3

3×3

Stride=**2**

=

91	100	83
69		

3×3

8.3 卷积步长

54

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

$$n \times n \quad * \quad f \times f \quad \longrightarrow \quad (n + 2p - f + 1) \times (n + 2p - f + 1)$$

$$\begin{array}{l} \text{stride} = s \\ \text{padding} = p \end{array} \longrightarrow \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

8.3 卷积步长

55

2	3	7	4	6	2	9	
6	6	9	8	7	4	3	
3	4	8	3	8	9	7	
7	8	3	6	6	3	4	
4	2	1	8	3	4	6	
3	2	4	1	9	8	3	
0	1	3	9	2	1	4	

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

8.3 卷积步长

56

$n \times n$ image $f \times f$ filter

padding p stride s

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

8.3 卷积步长

57

Input (4,4) Kernel(3,3) Padding=0 Stride=1

Input (5,5) Kernel(3,3) Padding=1 Stride=1

Input (6,6) Kernel(3,3) Padding=1 Stride=2

8.3 卷积步长

58

Input (4,4) Kernel(3,3) Padding=0 Stride=1
Output = $(4 - 3)/1 + 1 = 2$

Input (5,5) Kernel(3,3) Padding=1 Stride=1
Output = $(5 + 2*1 - 3)/1 + 1 = 5$

Input (6,6) Kernel(3,3) Padding=1 Stride=2
Output = $(6 + 2*1 - 3)/2 + 1 = 2.5 + 1 = 3.5$

8.3 卷积步长

59

- 输入图片的尺寸统一为 $227 \times 227 \times 3$ (高度 $\times$ 宽度 $\times$ 颜色通道数) ,
- 本层一共具有96个卷积核,
- 每个卷积核的尺寸都是 $11 \times 11 \times 3$ 。
- 已知 $\text{stride} = 4$, $\text{padding} = 0$,
- 则输出的高度/宽度为

8.4 三维卷积

60

三维卷积的基本特征概括如下：

- (1) 输入图像拥有RGB三个通道，**过滤器的通道数必须与输入图像的通道数相同**。
- (2) 过滤器的通道与输入图像的通道**一一对应**，过滤器在对应图层上做二维卷积。
- (3) 3个RGB通道图层**各自与过滤器做二维卷积**，然后**求和**后算作一次三维卷积，**输出的目标特征图只有一个图层**。

8.4 三维卷积

61

- 用二维矩阵描述数字图像：灰度图像中，最低值(0)对应黑，最高值(255)对应白，黑白之间的亮度值是灰度阶(gray-level)



156	145	142	140	142	167	180	172	172	158	172	155	135	129	135	140	138	138	141
151	152	161	140	137	169	181	177	175	176	139	87	88	110	122	130	137	143	148
149	173	164	131	136	172	183	187	180	122	58	53	66	69	76	82	97	119	142
167	185	155	129	153	176	188	177	117	86	105	110	98	84	65	39	39	51	77
183	181	135	140	170	193	169	101	88	115	123	110	127	128	102	79	51	40	33
190	166	136	160	194	156	80	76	85	92	100	65	81	128	89	81	73	71	60
188	164	157	188	140	55	52	58	50	40	45	24	29	55	28	28	64	80	84
180	171	187	130	37	35	30	23	25	20	15	17	23	25	17	11	13	47	98
177	190	132	39	30	22	17	18	21	17	17	16	49	114	81	23	11	7	46
194	144	49	40	27	13	24	38	28	28	12	38	61	175	177	114	30	35	44
162	69	62	41	23	13	33	72	47	54	40	50	51	177	197	189	99	51	72
84	79	89	74	63	23	23	79	77	45	43	42	123	200	194	190	134	61	77
81	85	96	105	97	62	40	64	94	90	76	128	187	194	196	165	114	96	79
93	92	98	112	110	103	79	71	76	93	109	128	142	153	151	155	136	117	99
98	103	104	110	110	111	105	107	85	77	67	97	98	89	103	132	144	142	110
103	111	111	115	117	109	112	115	103	100	92	103	106	102	120	126	121	118	114
107	118	122	120	124	125	123	115	112	115	112	115	122	129	136	138	132	138	117
105	117	123	126	135	136	135	138	133	133	133	140	146	146	142	142	139	138	108
104	114	123	126	137	140	138	151	151	144	142	146	150	144	144	147	139	127	109

8.4 三维卷积

62

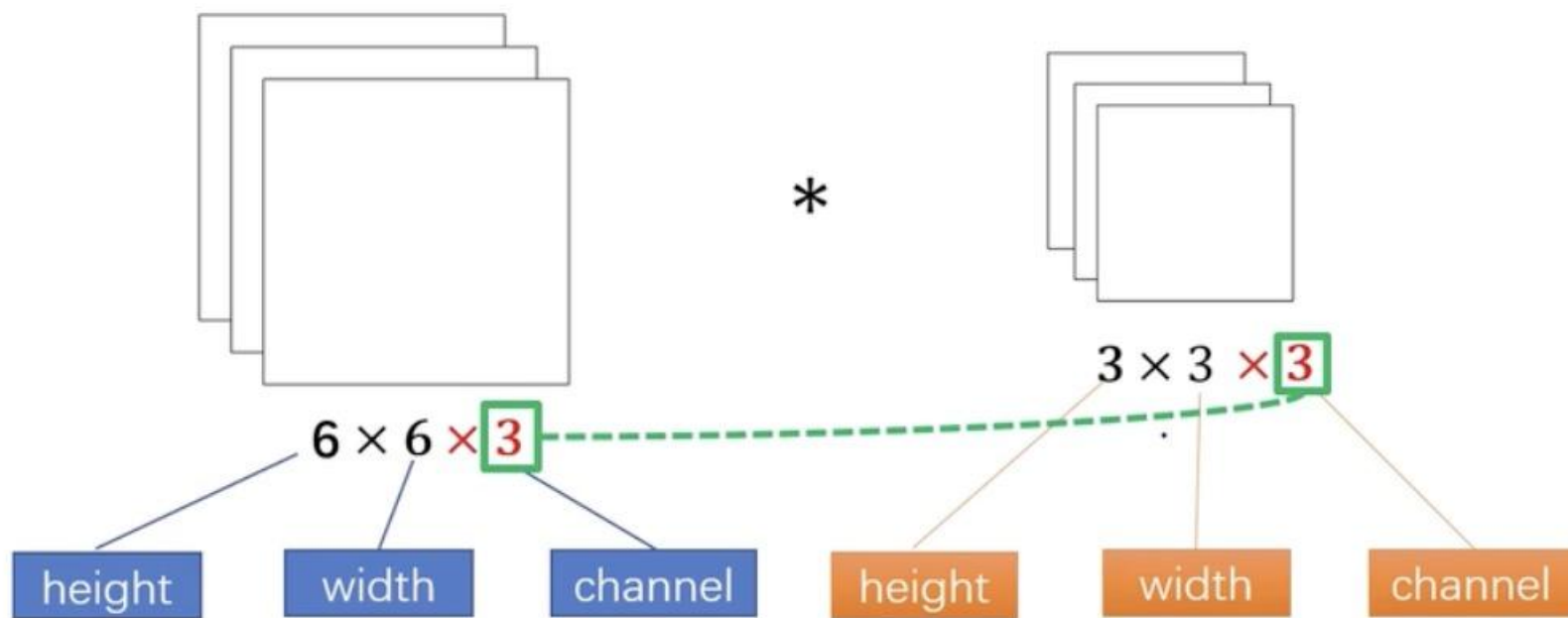
- 用二维矩阵描述数字图像：灰度图像中，最低值对应黑，最高值对应白，黑白之间的亮度值是灰度阶(gray-level)
- 用三维矩阵（矢量函数）描述彩色图像



214	211	206	206	204	205	205	217	216	221										
212	206	204	207	203	204	208	218	210	220										
211	199	204	208	204	156	147	138	133	130	135	146	167	173	183					
211	192	201	203	207	156	144	136	133	129	134	140	168	176	183					
207	201	213	203	202	158	139	135	134	133	142	137	129	127	127	133	140	160	166	174
200	198	207	203	217	158	132	134	132	136	141	133	127	130	126	132	145	161	169	174
201	204	207	211	206	148	133	138	129	141	142	128	128	133	131	137	155	166	173	173
203	204	206	211	216	136	128	134	133	157	142	122	126	130	134	141	161	169	175	170
205	201	206	208	218	131	130	137	147	152	132	122	132	126	140	149	172	175	172	178
207	206	215	215	219	129	130	140	153	166	124	120	128	131	156	155	174	178	167	180
					133	131	146	157	175	123	127	135	147	150	166	170	166	173	172
					139	140	159	168	183	126	131	141	152	165	171	173	175	175	176
										134	133	146	154	168	167	166	168	172	160
										138	141	158	162	171	175	174	173	159	115

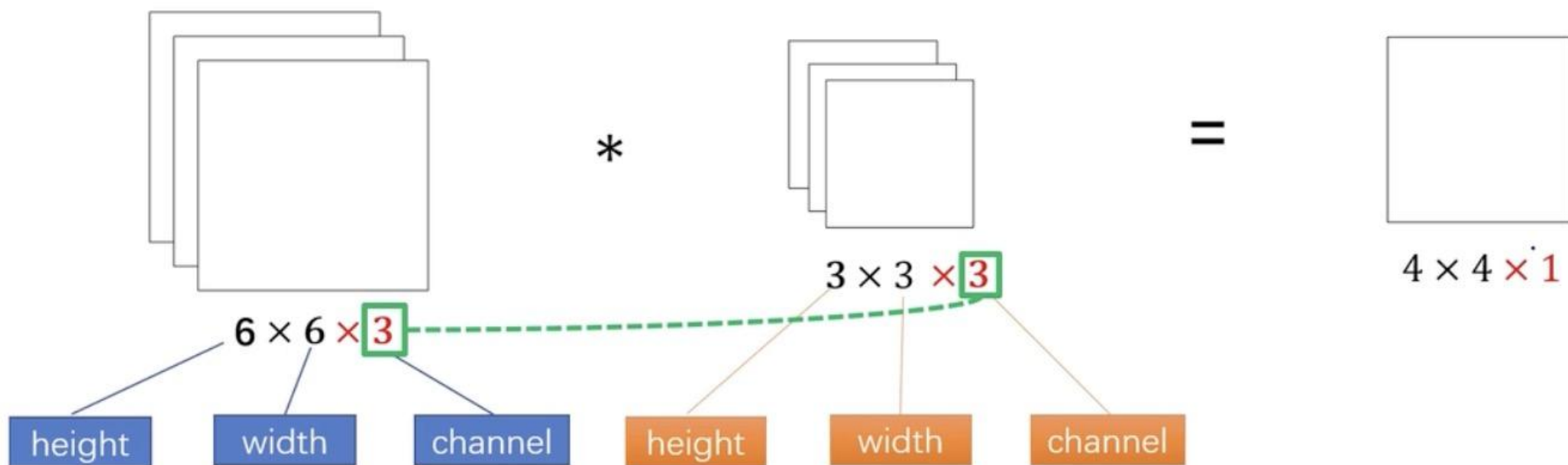
8.4 三维卷积

63



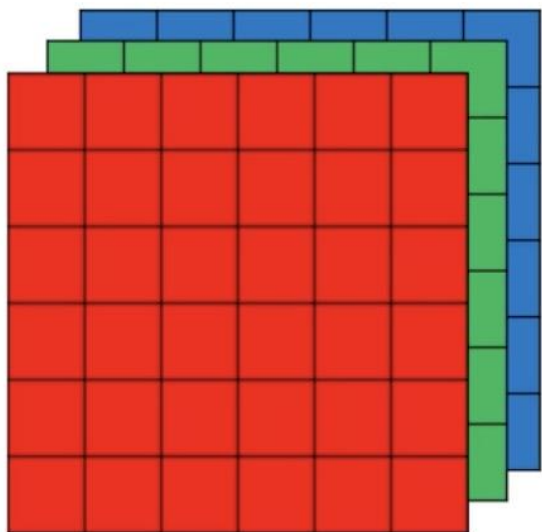
8.4 三维卷积

64

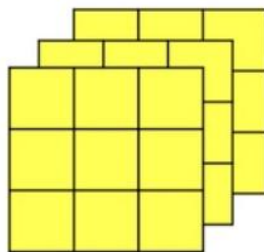


8.4 三维卷积

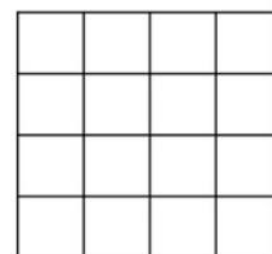
65



*



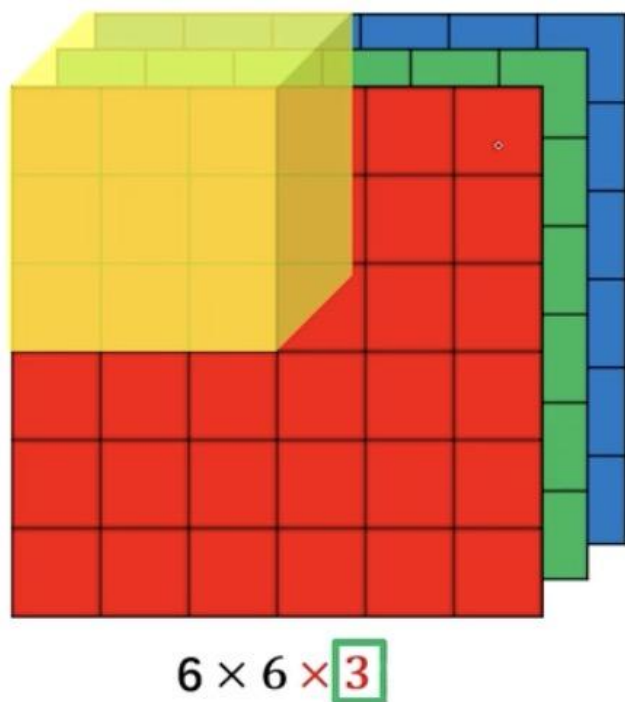
=



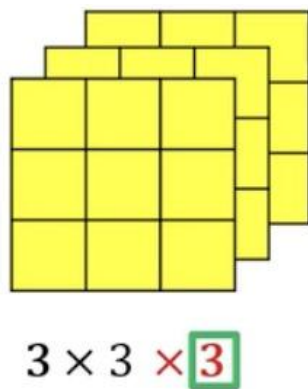
4 x 4

8.4 三维卷积

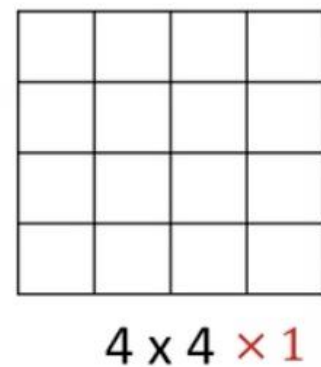
66



*

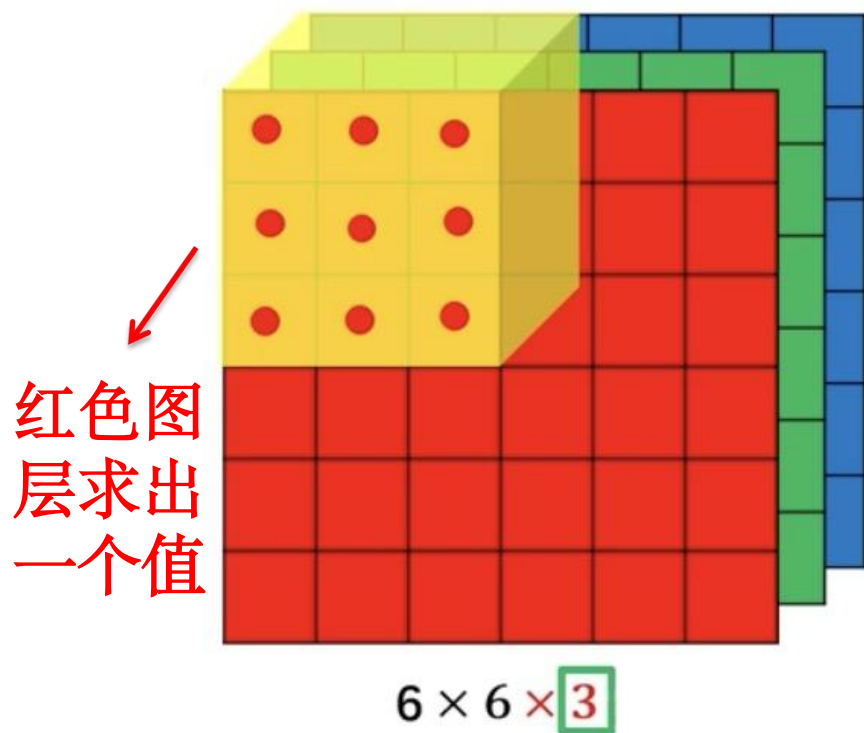


=

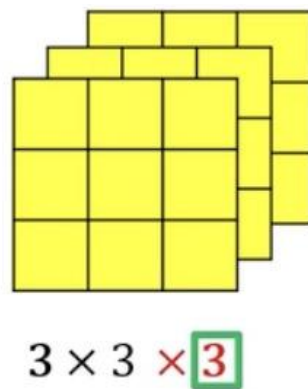


8.4 三维卷积

67

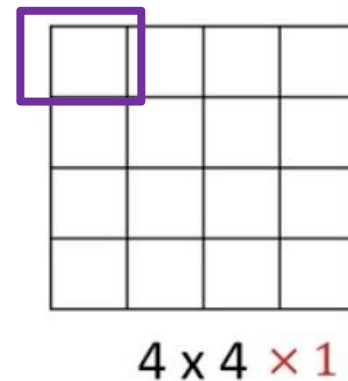


*



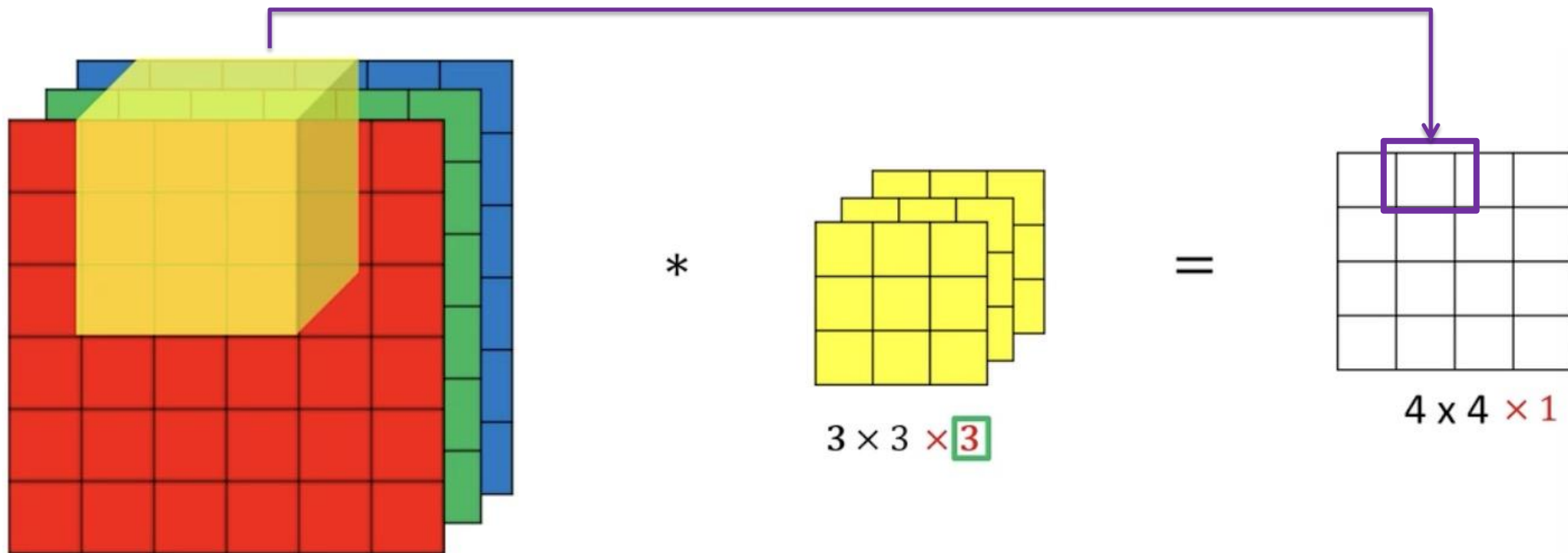
=

三个通道求和



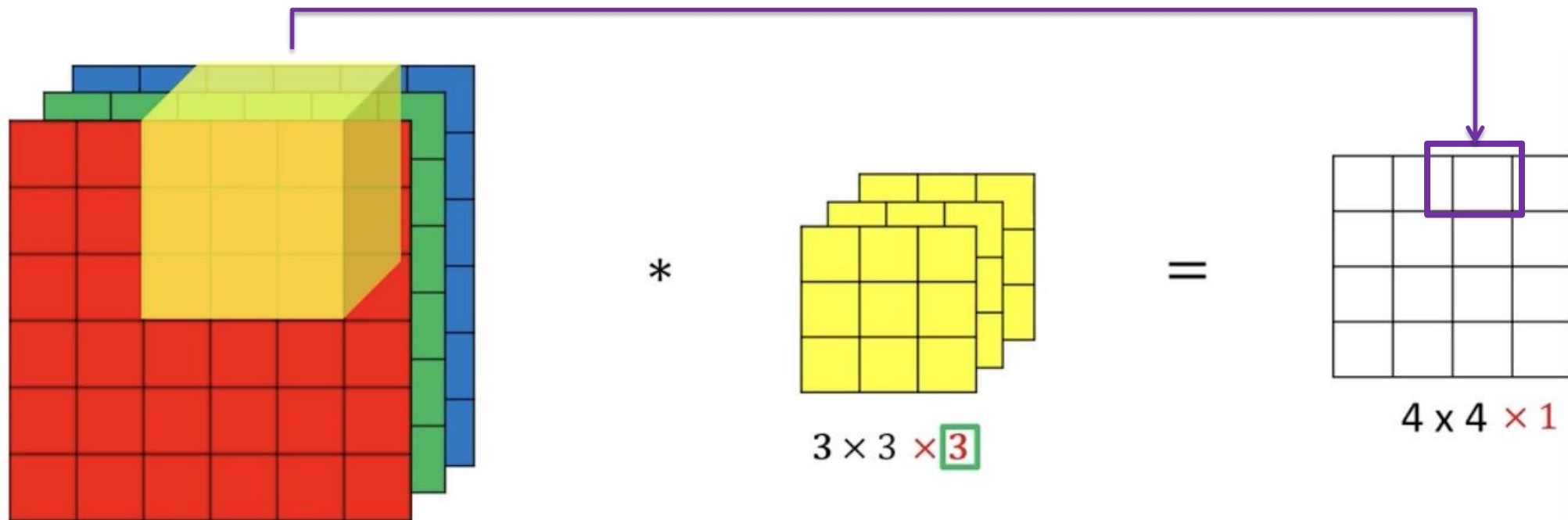
8.4 三维卷积

68



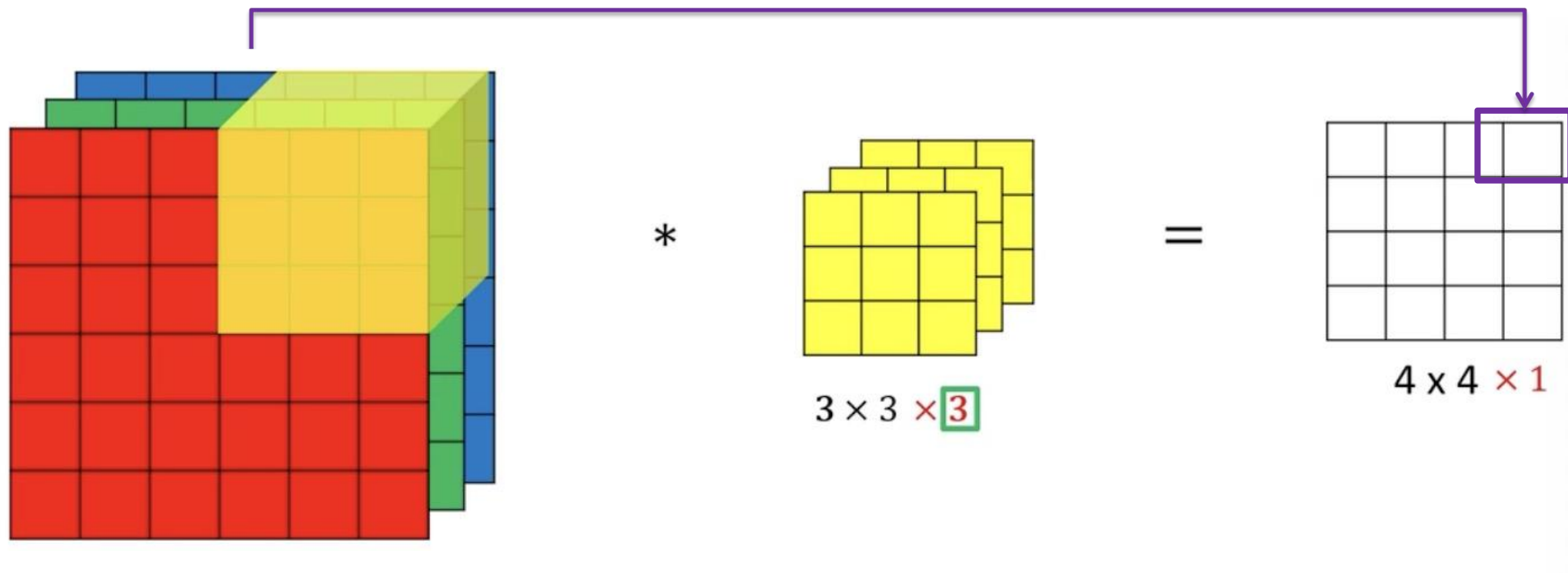
8.4 三维卷积

69



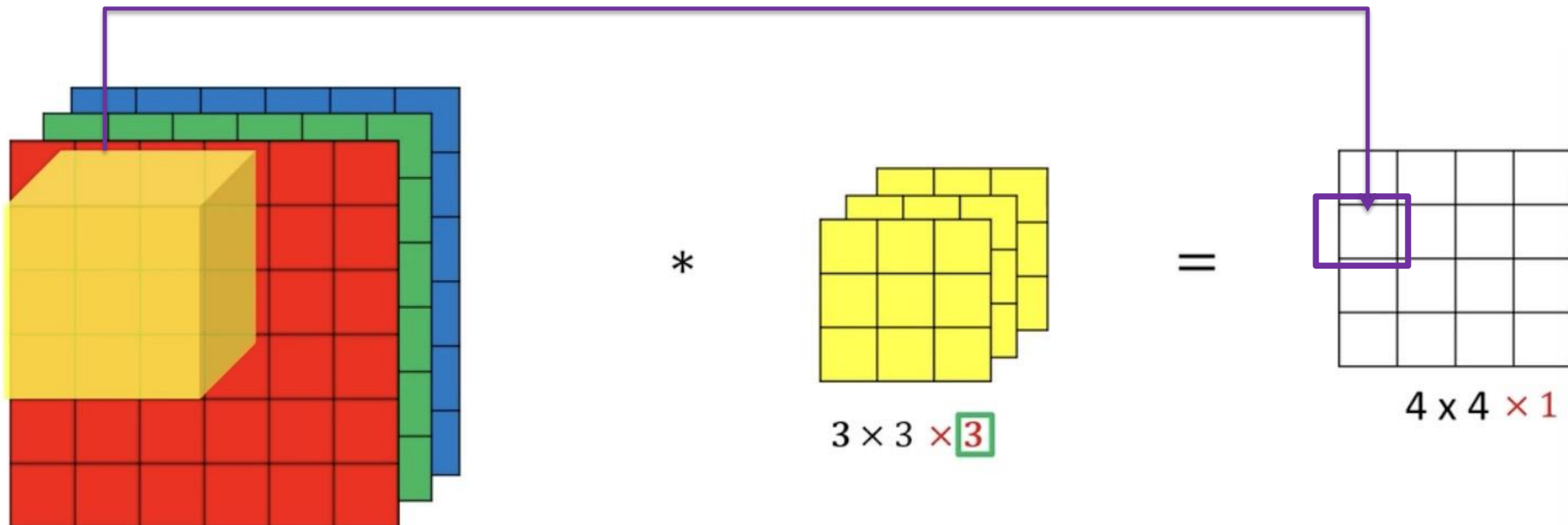
8.4 三维卷积

70



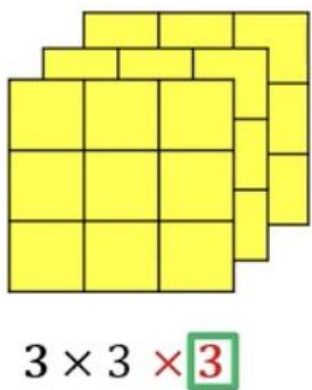
8.4 三维卷积

71



8.4 三维卷积

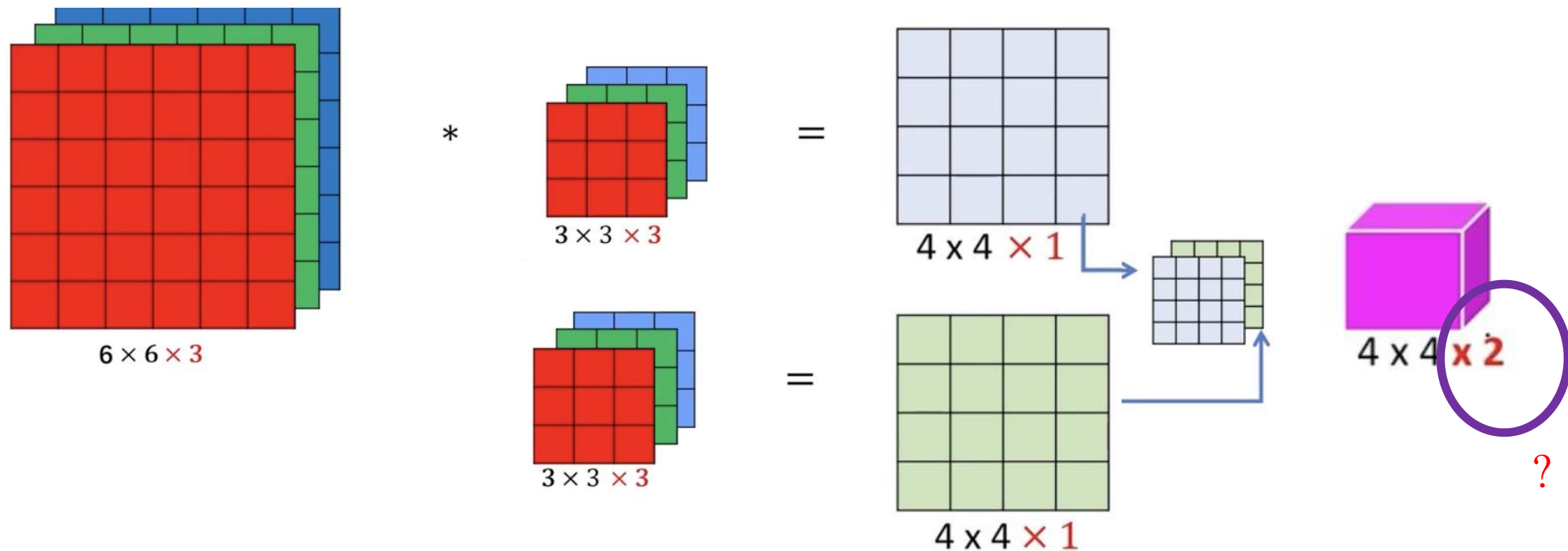
72



1	0	-1	0	0	0	0	0	0
1	0	-1	0	0	0	0	0	0
1	0	-1	0	0	0	0	0	0

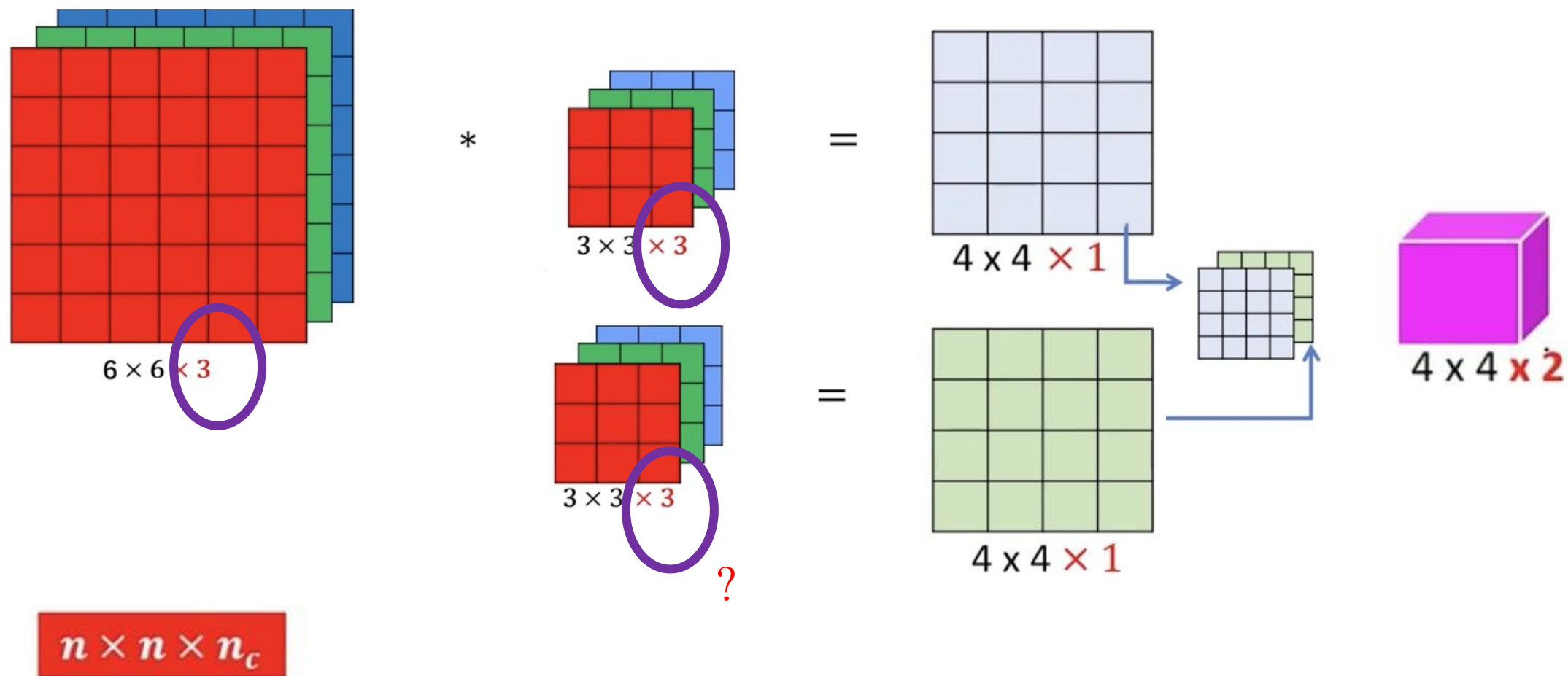
8.4 三维卷积

73



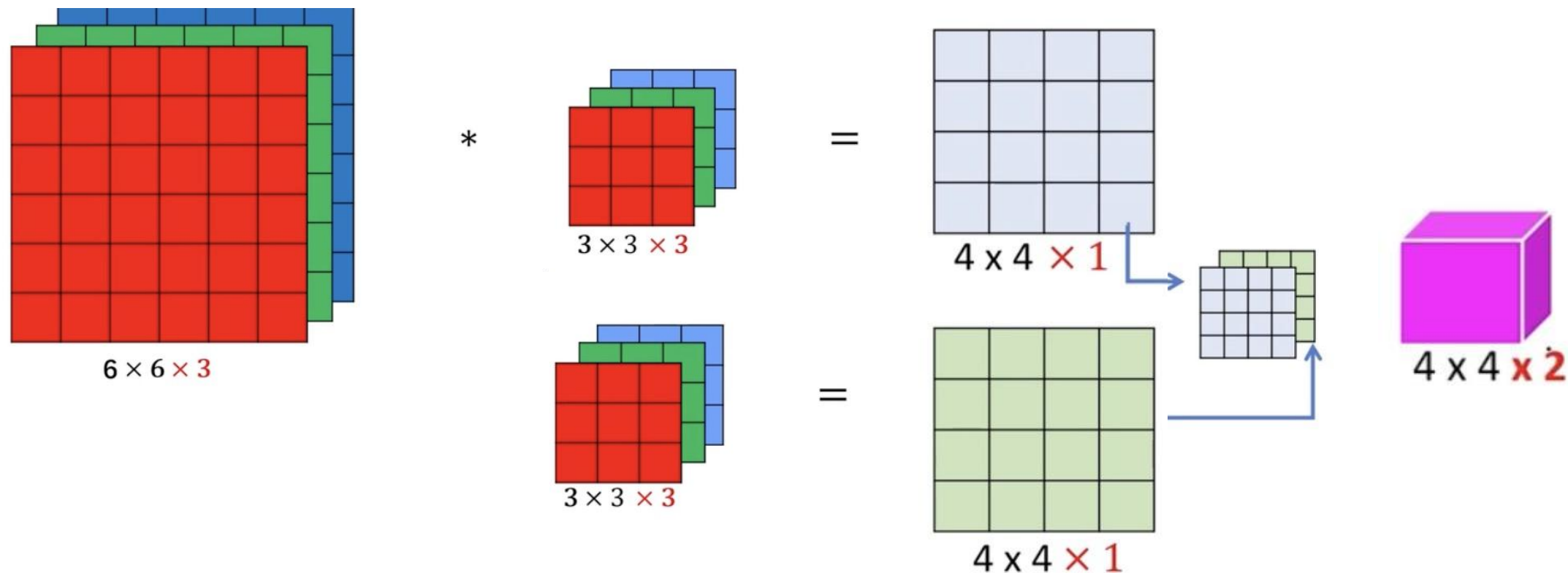
8.4 三维卷积

74



8.4 三维卷积

75



$$n \times n \times n_c$$

$$f \times f \times n_c \times n_f \quad (n - f + 1) \times (n - f + 1) \times n_f$$

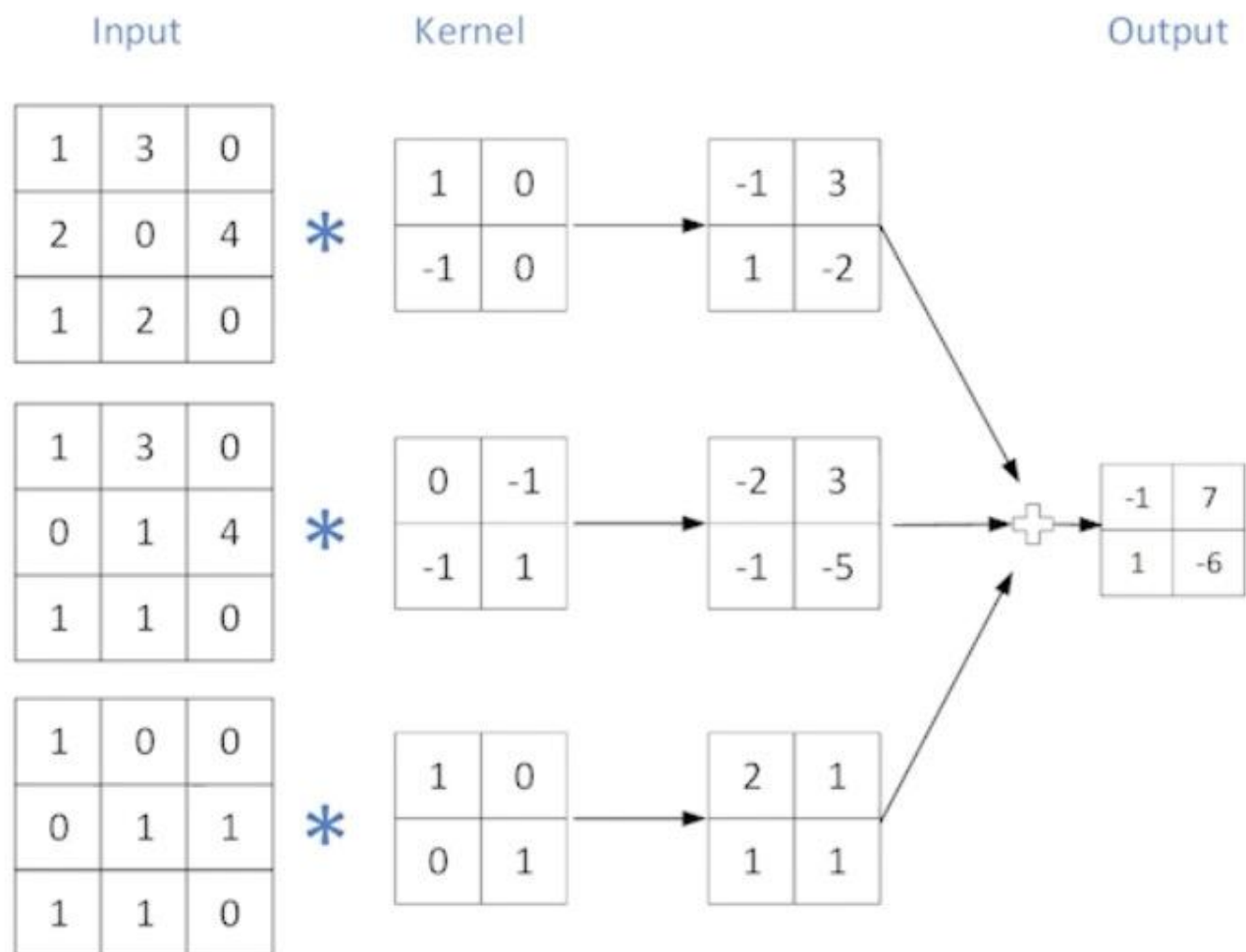
8.4 三维卷积

76

Input		Kernel		Output													
<table><tr><td>1</td><td>3</td><td>0</td></tr><tr><td>2</td><td>0</td><td>4</td></tr><tr><td>1</td><td>2</td><td>0</td></tr></table>	1	3	0	2	0	4	1	2	0	*	<table><tr><td>1</td><td>0</td></tr><tr><td>-1</td><td>0</td></tr></table>	1	0	-1	0	-	
1	3	0															
2	0	4															
1	2	0															
1	0																
-1	0																
<table><tr><td>1</td><td>3</td><td>0</td></tr><tr><td>0</td><td>1</td><td>4</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	1	3	0	0	1	4	1	1	0	*	<table><tr><td>0</td><td>-1</td></tr><tr><td>-1</td><td>1</td></tr></table>	0	-1	-1	1	-	
1	3	0															
0	1	4															
1	1	0															
0	-1																
-1	1																
<table><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	1	0	0	0	1	1	1	1	0	*	<table><tr><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td></tr></table>	1	0	0	1	-	
1	0	0															
0	1	1															
1	1	0															
1	0																
0	1																

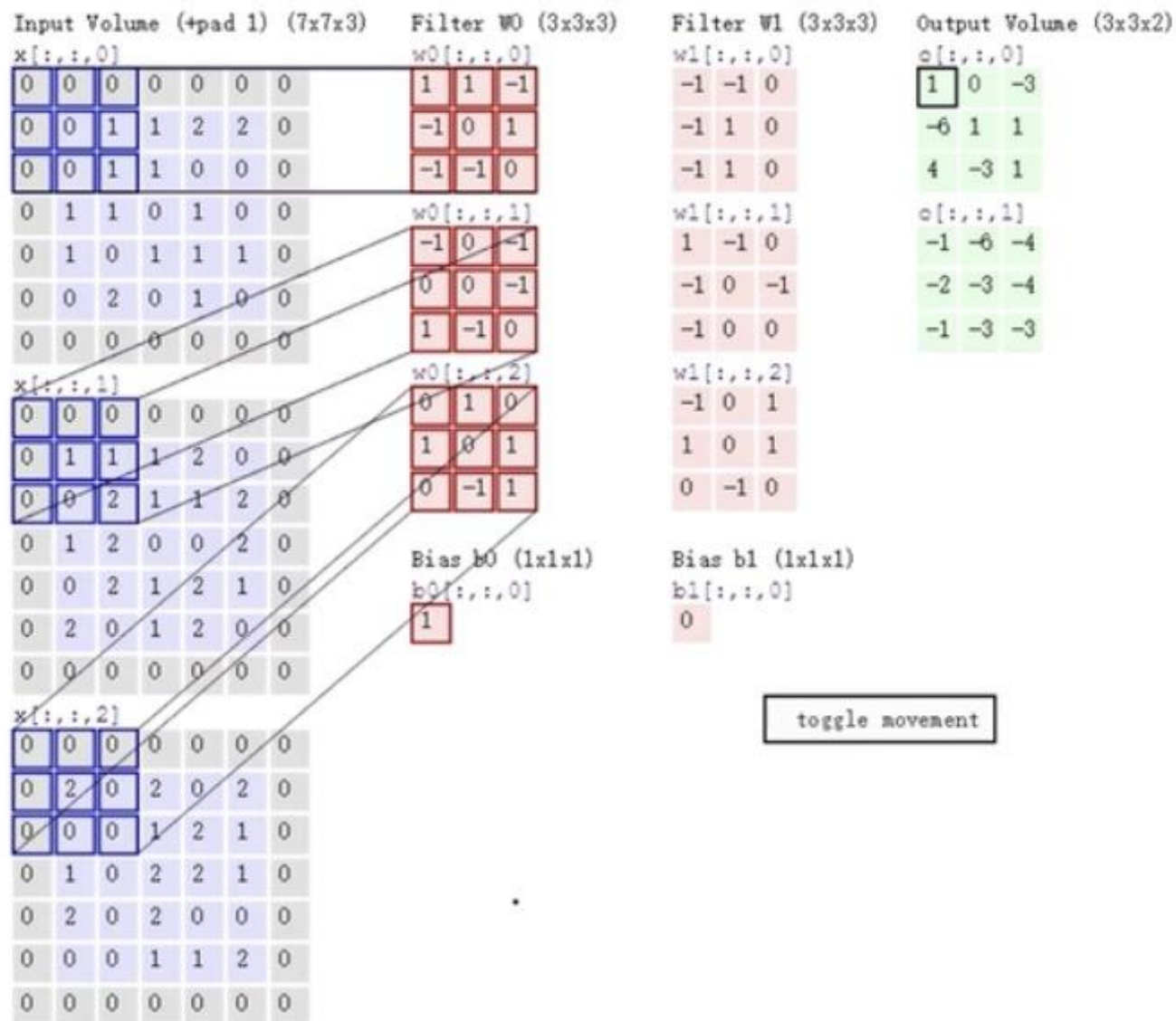
8.4 三维卷积

77



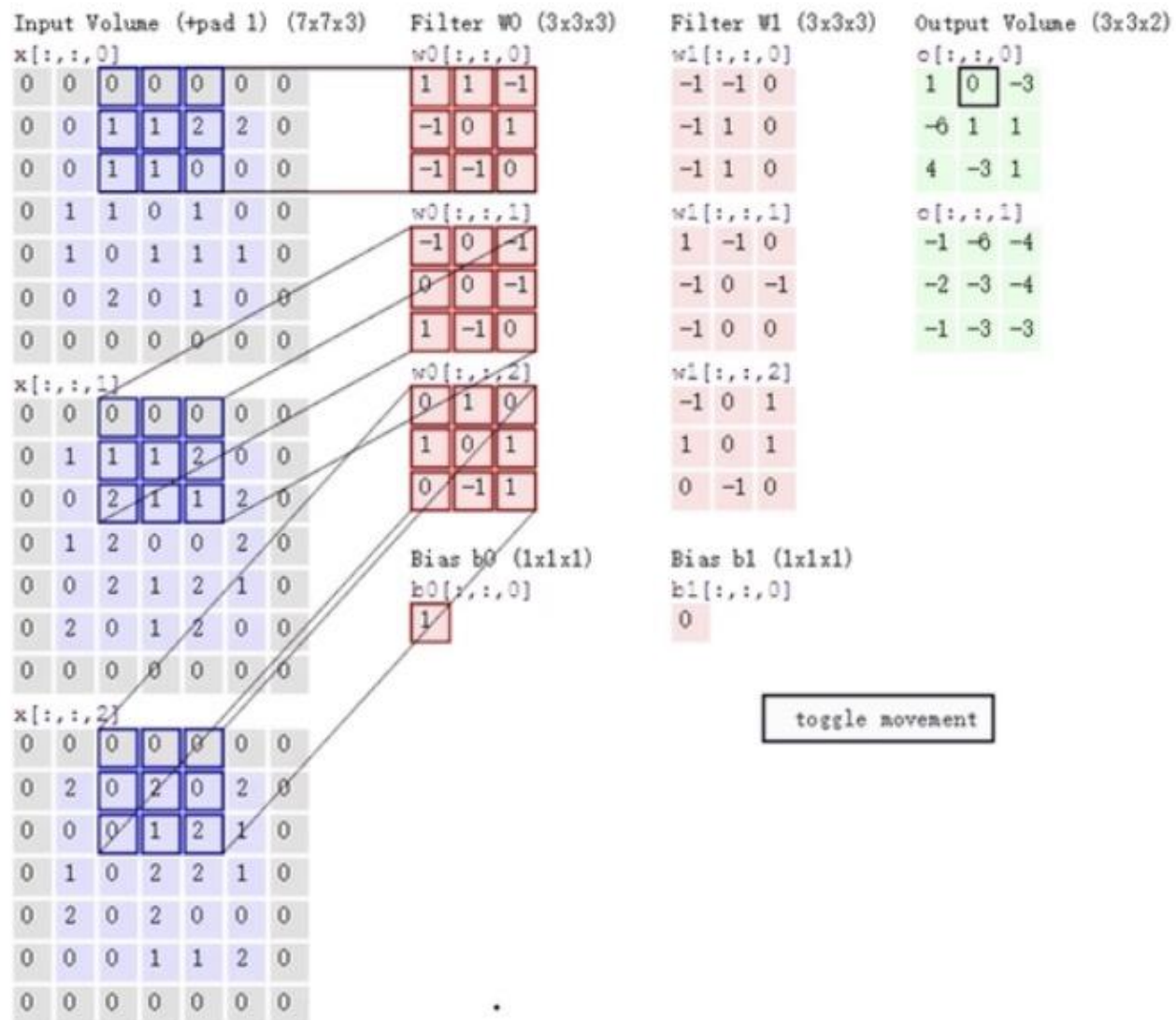
8.4 三维卷积

78



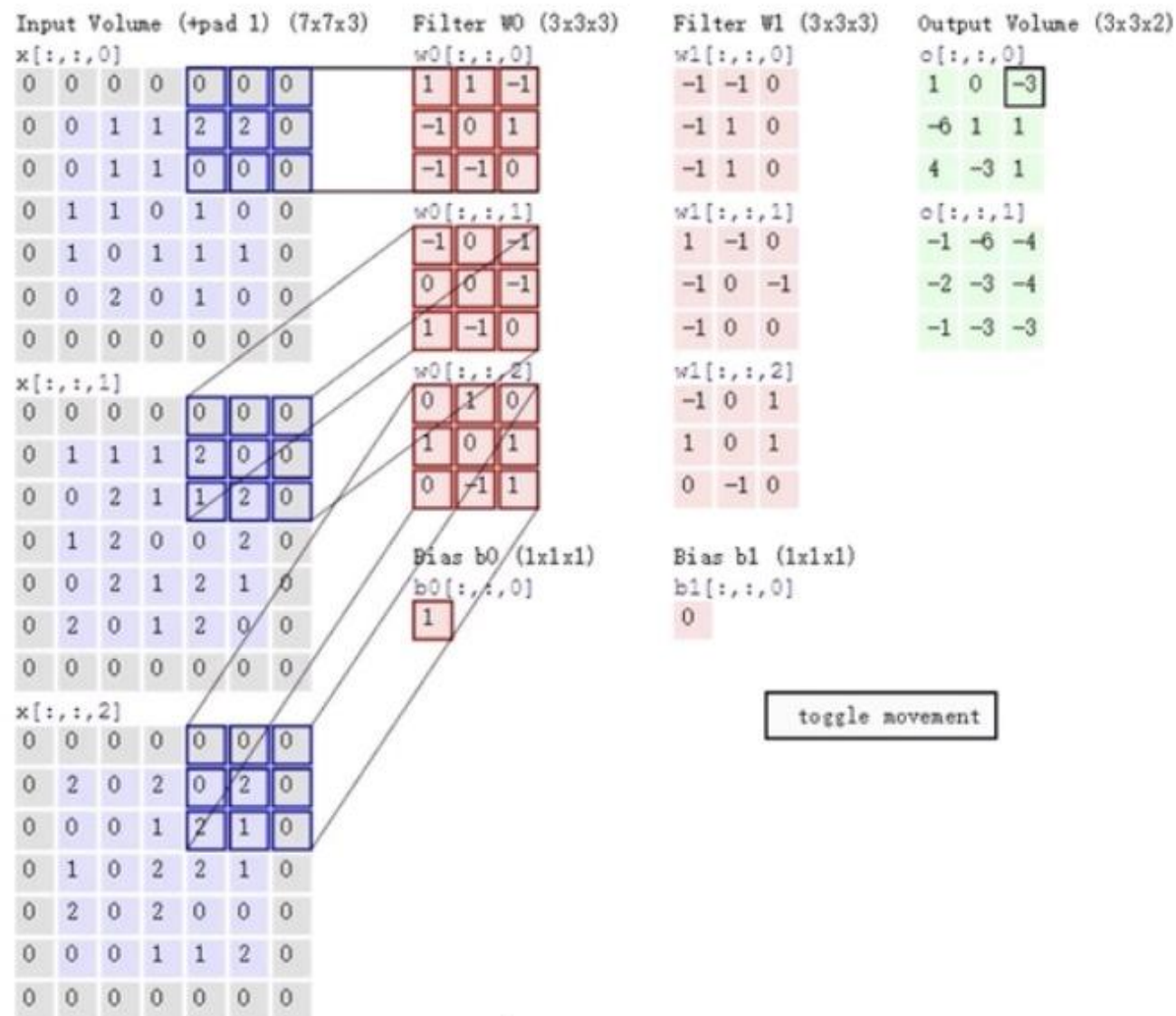
8.4 三维卷积

79



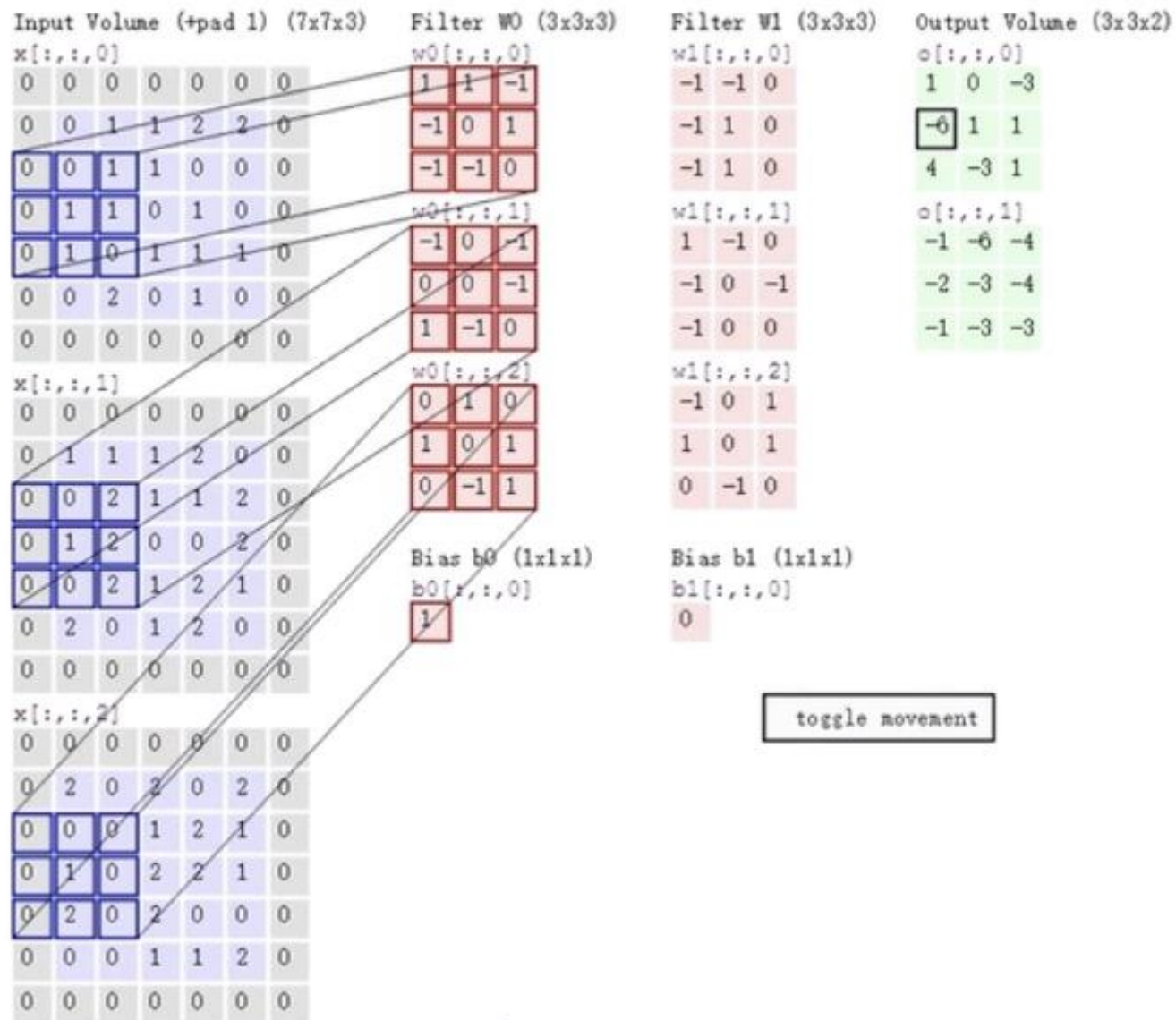
8.4 三维卷积

80



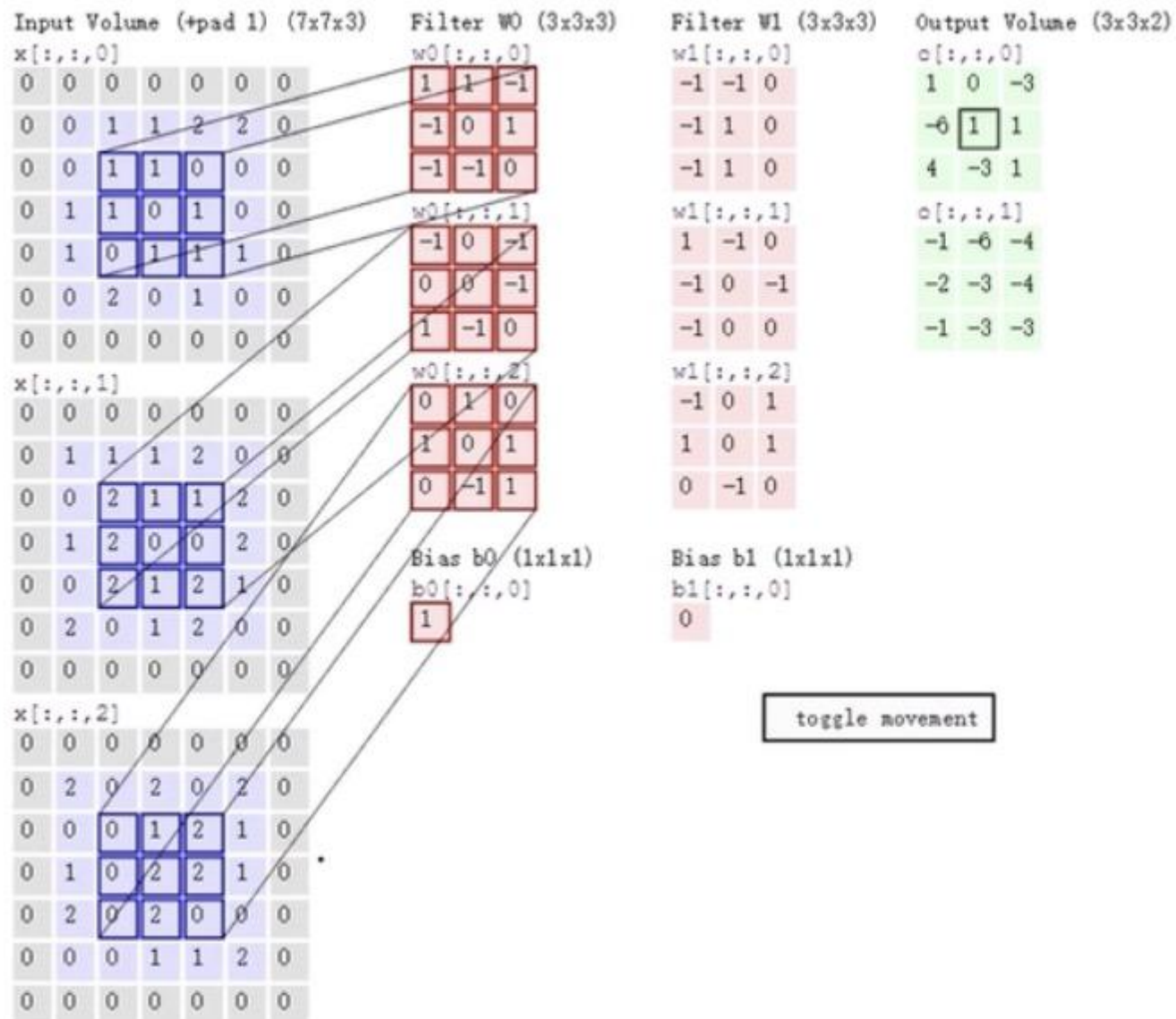
8.4 三维卷积

81



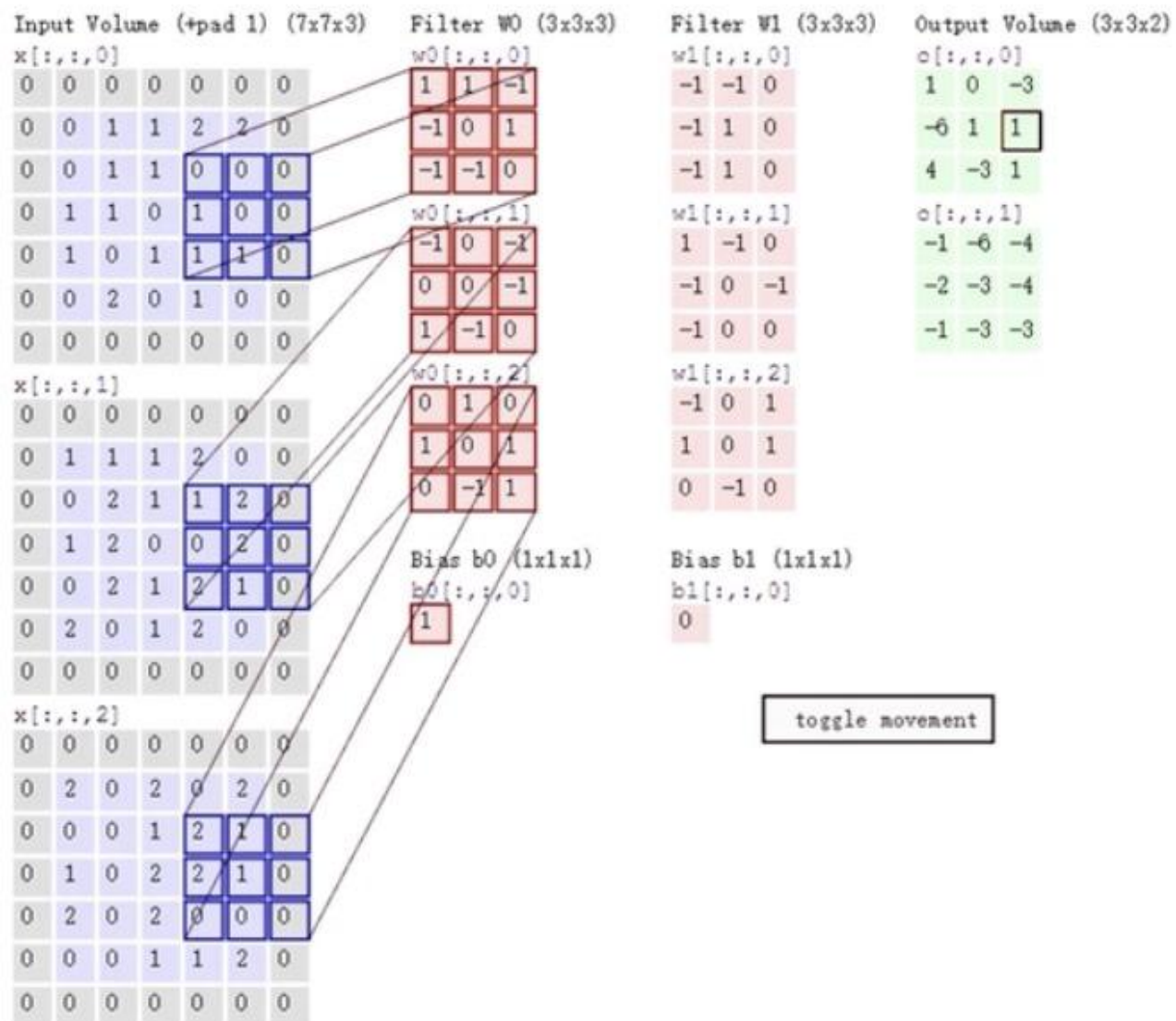
8.4 三维卷积

82



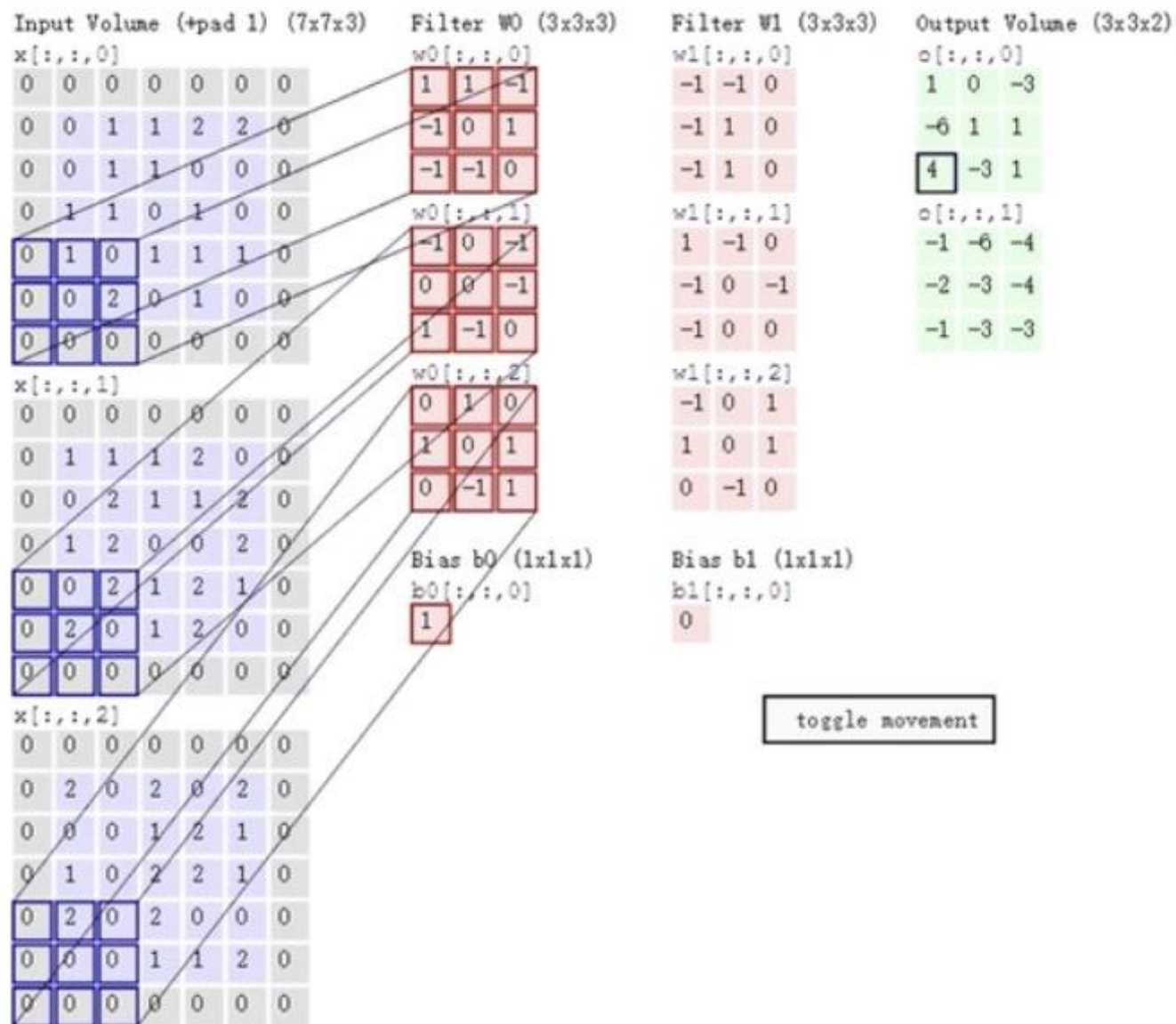
8.4 三维卷积

83



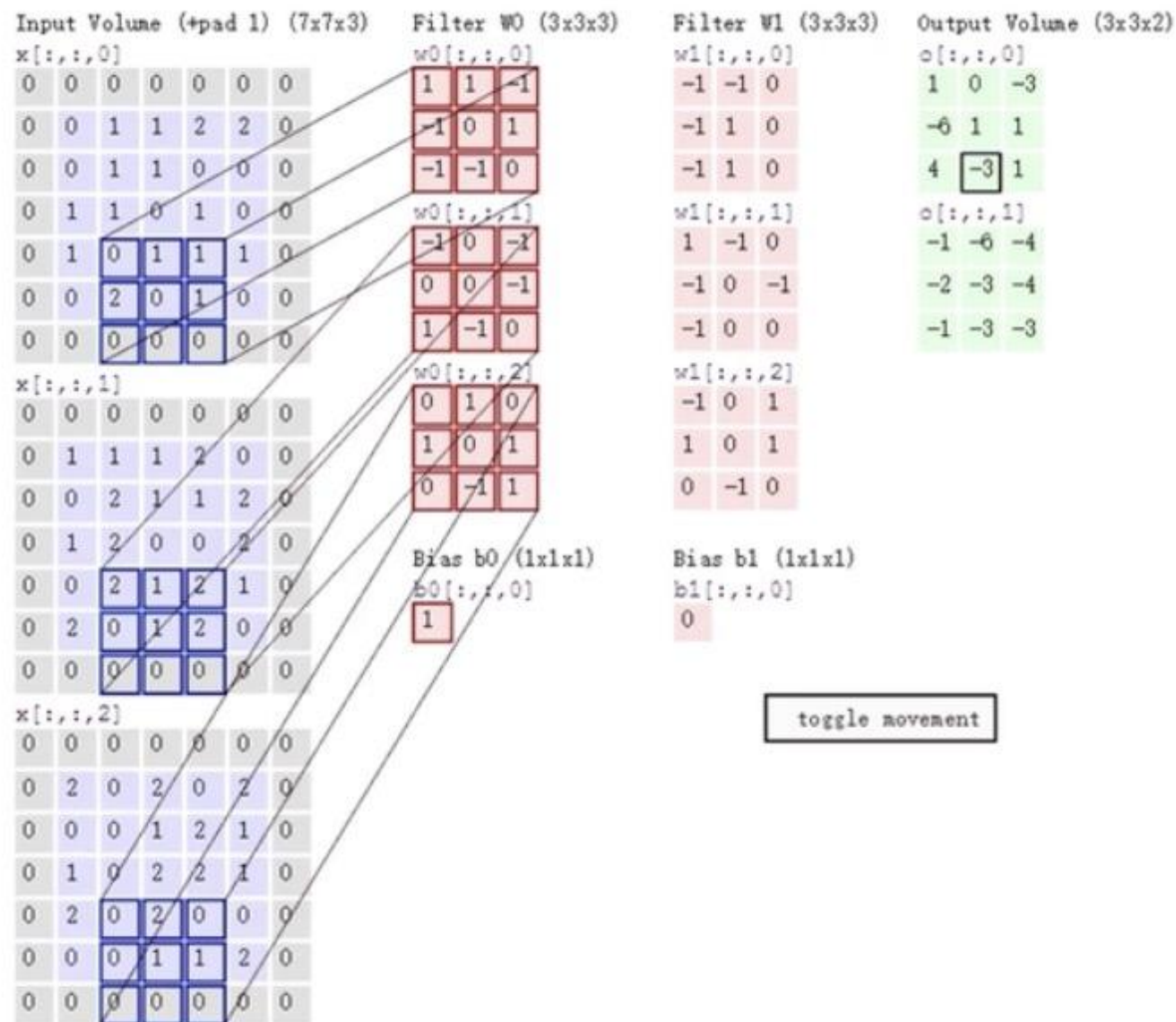
8.4 三维卷积

84



8.4 三维卷积

85



8.5 卷积层

86

卷积层，是将输入的图像，根据过滤器、边缘扩充、卷积步长等参数做卷积运算，输出目标图像的神经网络结构。

如何定义卷积层？ 假定第 l 层为卷积层：

过滤器的参数： $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$?

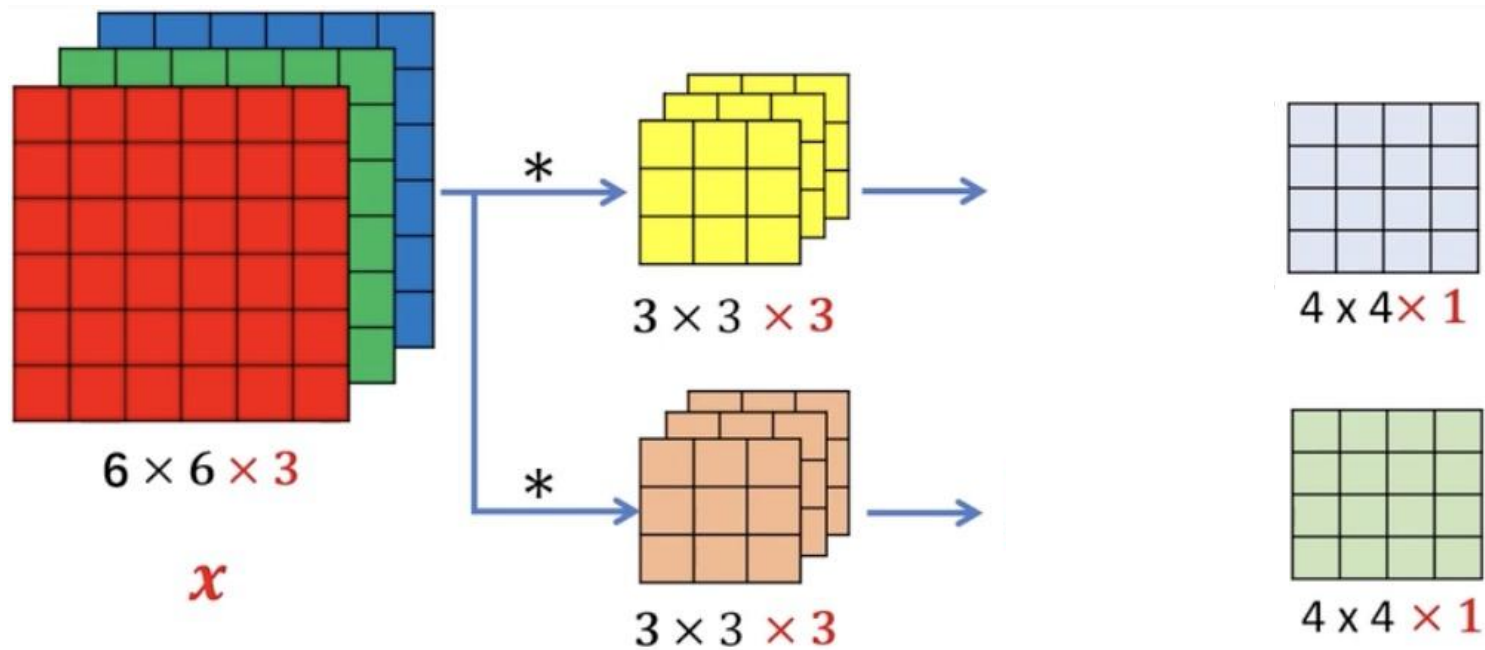
边缘扩充： $p^{[l]} = \text{padding size}$?

卷积步长： $s^{[l]} = \text{stride}$ ：第 l 层卷积步长？

如果是卷积网络的第1层，input size？

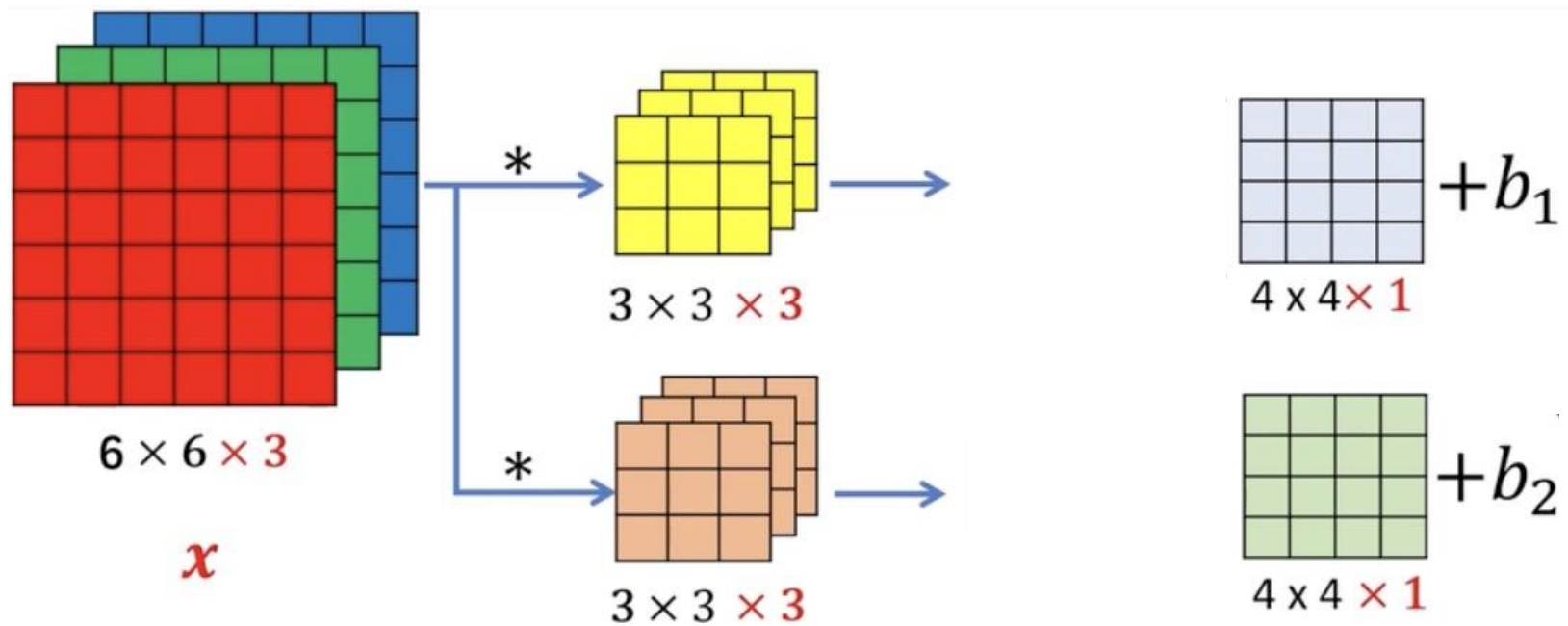
8.5 卷积层

87



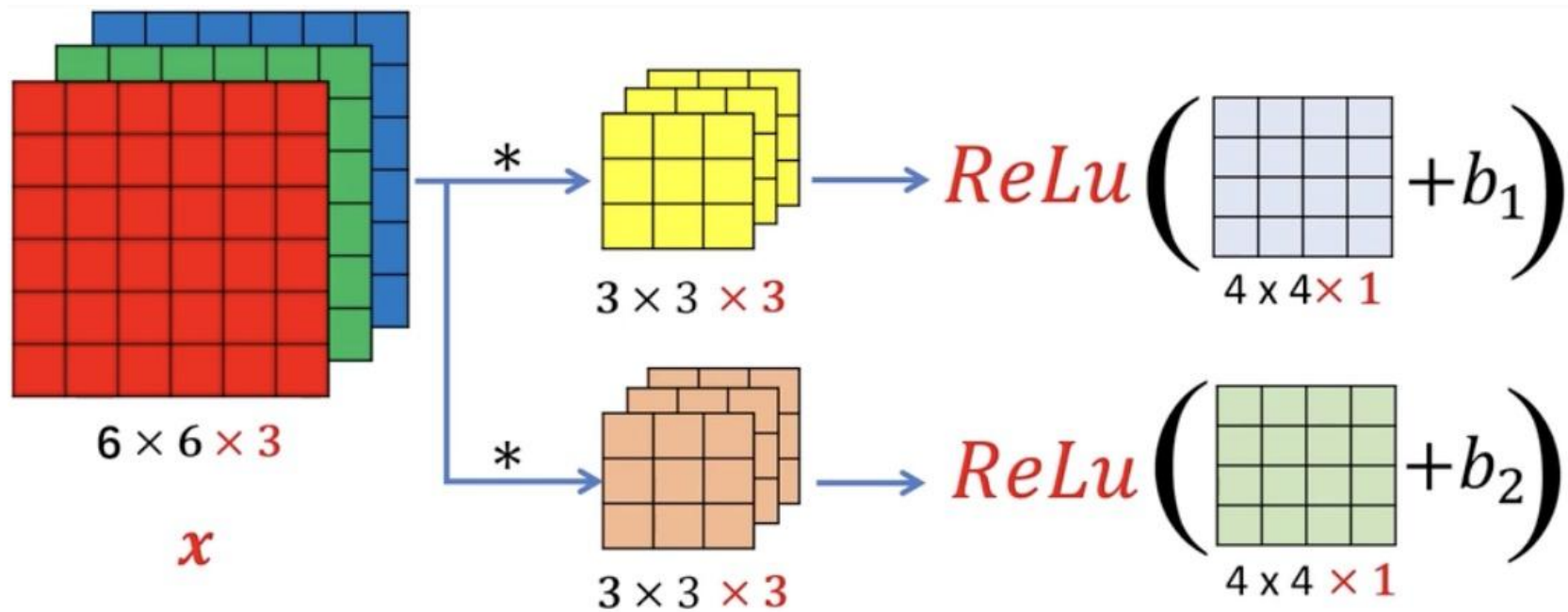
8.5 卷积层

88



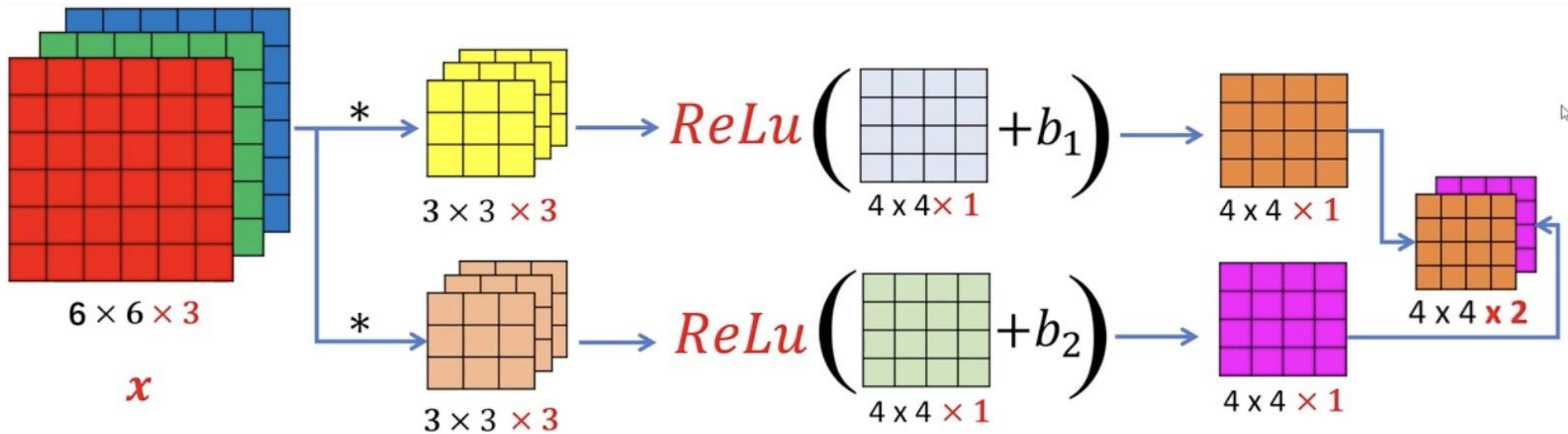
8.5 卷积层

89



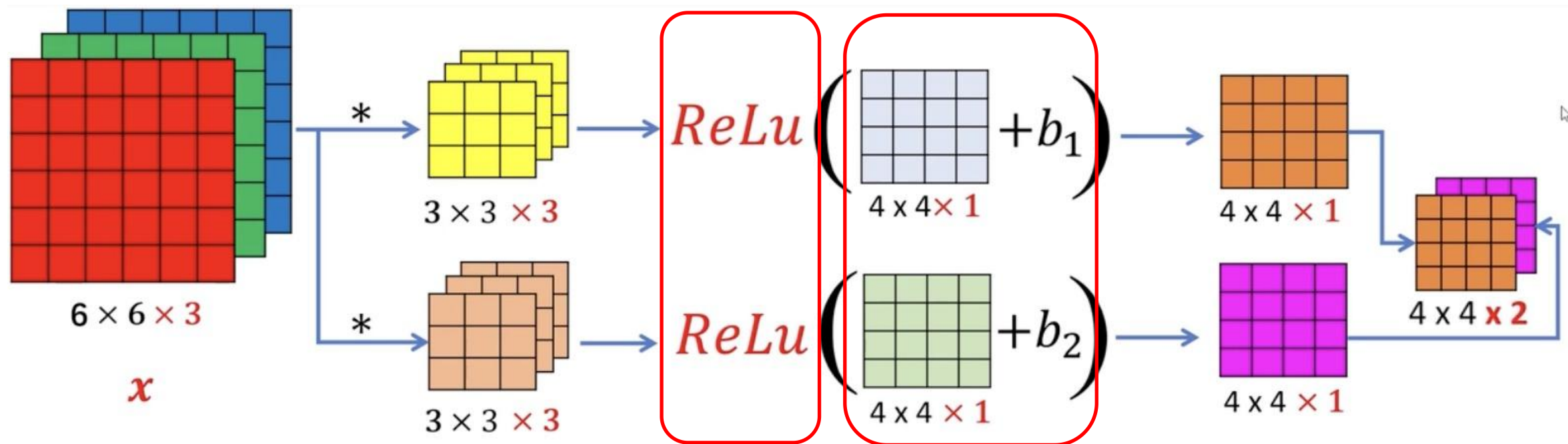
8.5 卷积层

90



8.5 卷积层

91



$$z^{[1]} = w^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

8.5 卷积层

92

假定卷积层采用10个 $3 \times 3 \times 3$ 的过滤器做卷积运算，那么这个卷积层将有多少个参数？

8.5 卷积层

93

假定卷积层采用10个 $3 \times 3 \times 3$ 的过滤器做卷积运算，那么这个卷积层将有多少个参数？



$3 \times 3 \times 3$

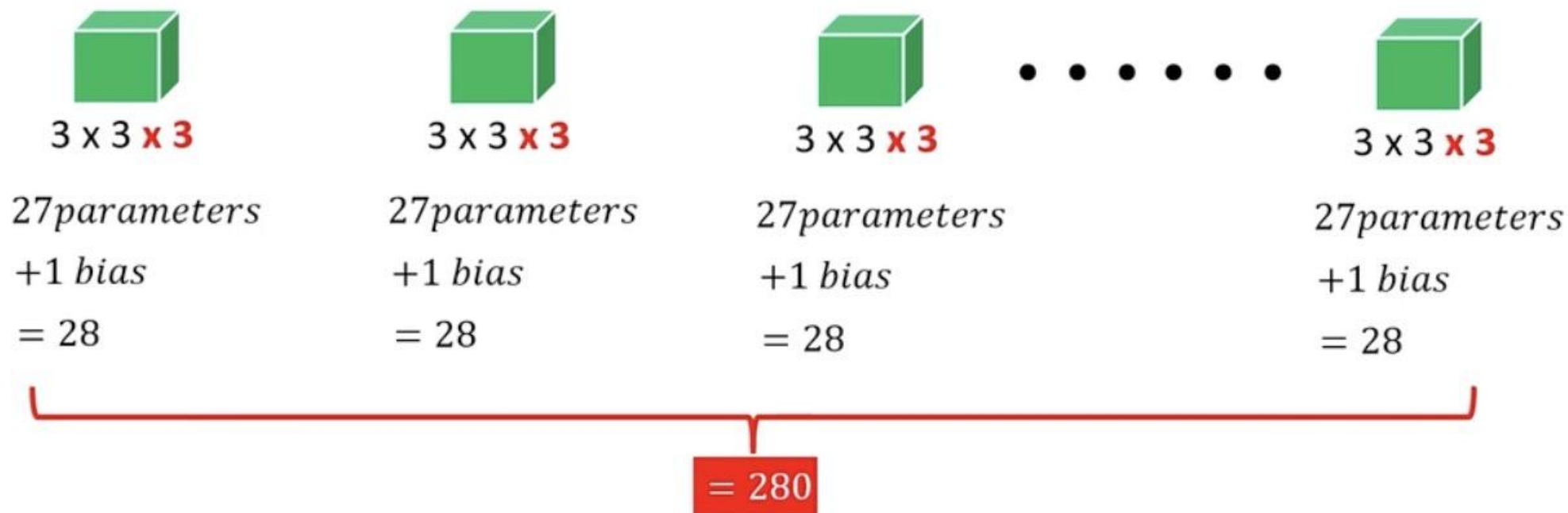
27 parameters

+1 bias

8.5 卷积层

94

假定卷积层采用10个 $3 \times 3 \times 3$ 的过滤器做卷积运算，那么这个卷积层将有多少个参数？



8.5 卷积层

95

- 【1】 $f^{[l]}$ = filter size : 第 l 层过滤器的高和宽为 $f \times f$
- 【2】 $p^{[l]}$ = padding size : 第 l 层padding大小为 p
- 【3】 $s^{[l]}$ = stride : 第 l 层卷积步长
- 【4】 第 $l-1$ 层的输出, 即本层的输入表示为: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
- 【5】 第 l 层的输出表示为: ? $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
- 【6】 $n_c^{[l]}$ = number of filters: 第 l 层过滤器数量
- 【7】 每个过滤器的维数为: ? $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$
- 【8】 $a^{[l]}$ 的维数为: ? $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
- 【9】 批量或小批量梯度下降 $A^{[l]}$ 的维数为: ? (batch_size=m) $m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
- 【10】 权重 $w^{[l]}$ 的维数为: ? $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$
- 【11】 偏差 $b^{[l]}$ 的维数为: ? $1 \times 1 \times 1 \times n_c^{[l]}$

8.5 卷积层

96

- 【1】 $f^{[l]}$ = filter size : 第 l 层过滤器的高和宽为 $f \times f$
- 【2】 $p^{[l]}$ = padding size : 第 l 层padding大小为 p
- 【3】 $s^{[l]}$ = stride : 第 l 层卷积步长
- 【4】 第 $l-1$ 层的输出, 即本层的输入表示为: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
- 【5】 第 l 层的输出表示为: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
- 【6】 $n_c^{[l]}$ = number of filters: 第 l 层过滤器数量
- 【7】 每个过滤器的维数为: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]}$
- 【8】 $a^{[l]}$ 的维数为: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
- 【9】 批量或小批量梯度下降 $A^{[l]}$ 的维数为: $m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$
- 【10】 权重 $w^{[l]}$ 的维数为: $f^{[l]} \times f^{[l]} \times n_c^{[l-1]} \times n_c^{[l]}$
- 【11】 偏差 $b^{[l]}$ 的维数为: $1 \times 1 \times 1 \times n_c^{[l]}$

计算第 l 层卷积神经网络输出图像的高度与宽度

【12】 输出图像的高度与宽度计算:

$$n_H^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$n_W^{[l]} = \left\lfloor \frac{n_W^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

8.5 卷积层

97

例1:

```
X = torch.rand(8, 8)
```

```
nn.Conv2d(in_channels=1, out_channels=1, kernel_size=(5, 3), padding=(2, 1))
```

```
torch.Size([8, 8])
```

8.5 卷积层

98

例2:

```
X = torch.rand(8, 8)
```

```
nn.Conv2d(1, 1, kernel_size=3, padding=1, stride=2)
```

```
torch.Size([4, 4])
```

8.5 卷积层

99

例3:

```
X = torch.rand(8, 8)
nn.Conv2d(1, 1, kernel_size=(3, 5), padding=(0, 1), stride=(3, 4))
```

8.5 卷积层

100

0	1	2
3	4	5
6	7	8

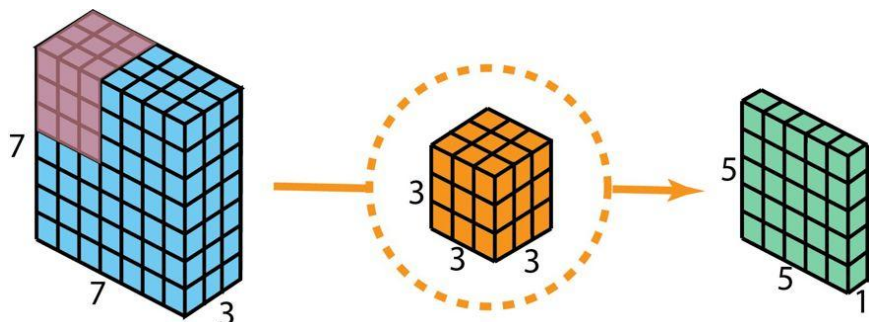
0	1
2	3

padding=1

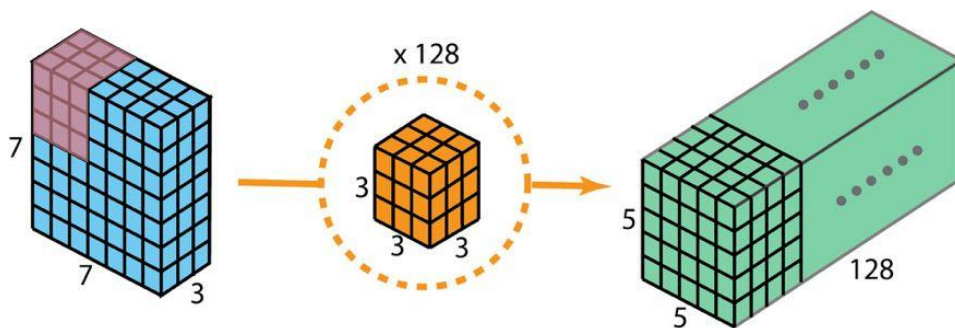
8.5 卷积层

101

为了便于比较，这里仍先给出常规卷积过程，如图所示。



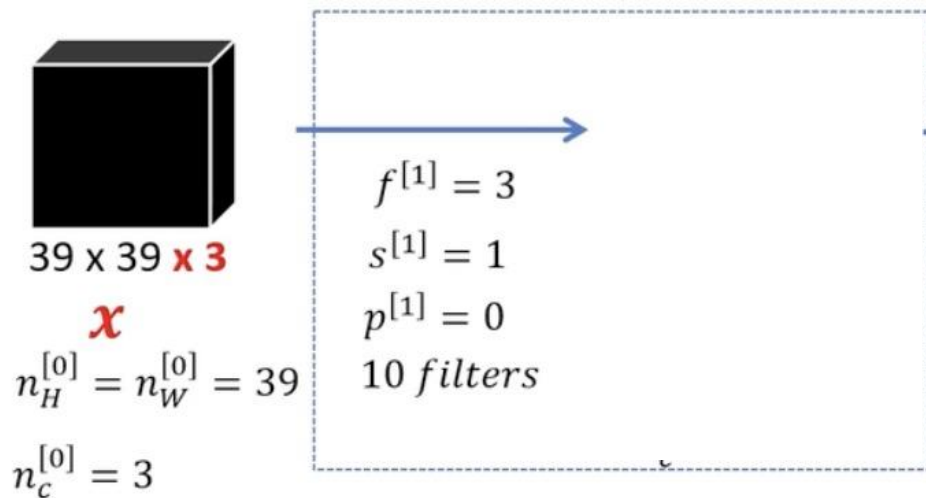
(a) 1层的输出标准2D卷积，使用1个过滤器



(b) 有128层的输出的标准2D卷积，要使用128个过滤器

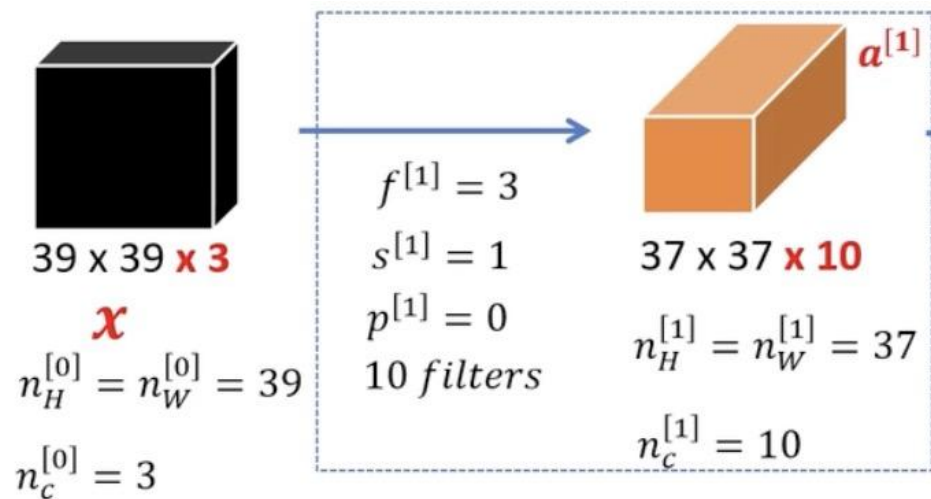
8.6 简单卷积网络

102



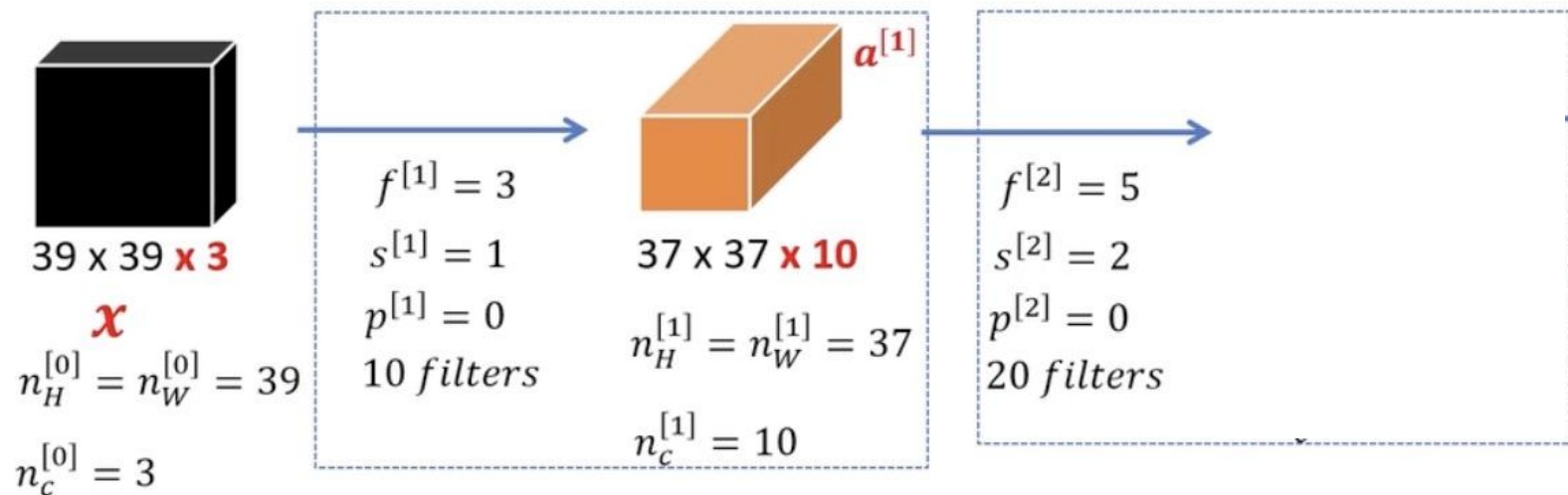
8.6 简单卷积网络

103



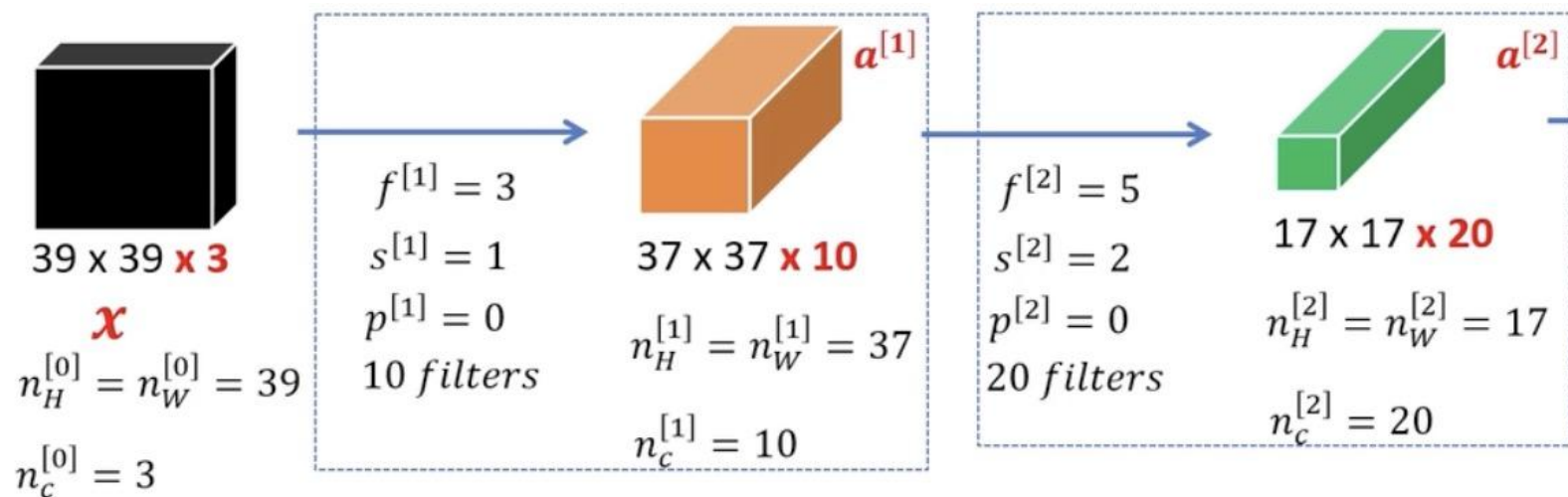
8.6 简单卷积网络

104



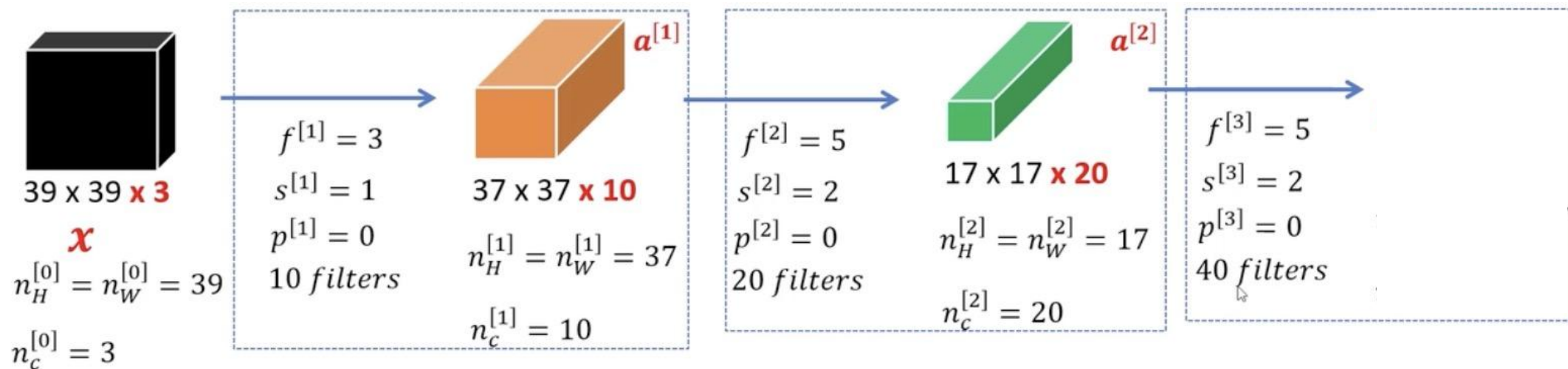
8.6 简单卷积网络

105



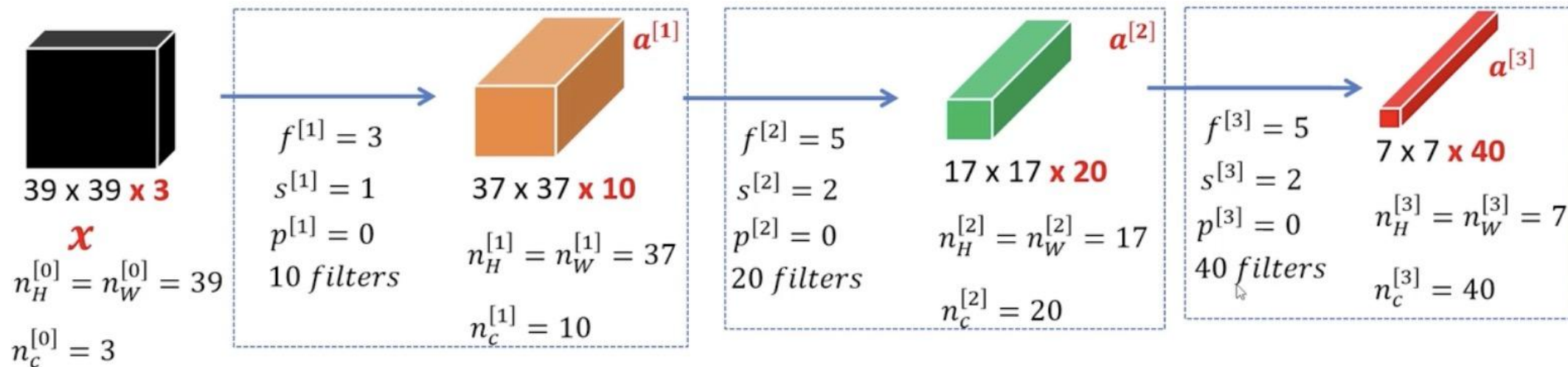
8.6 简单卷积网络

106



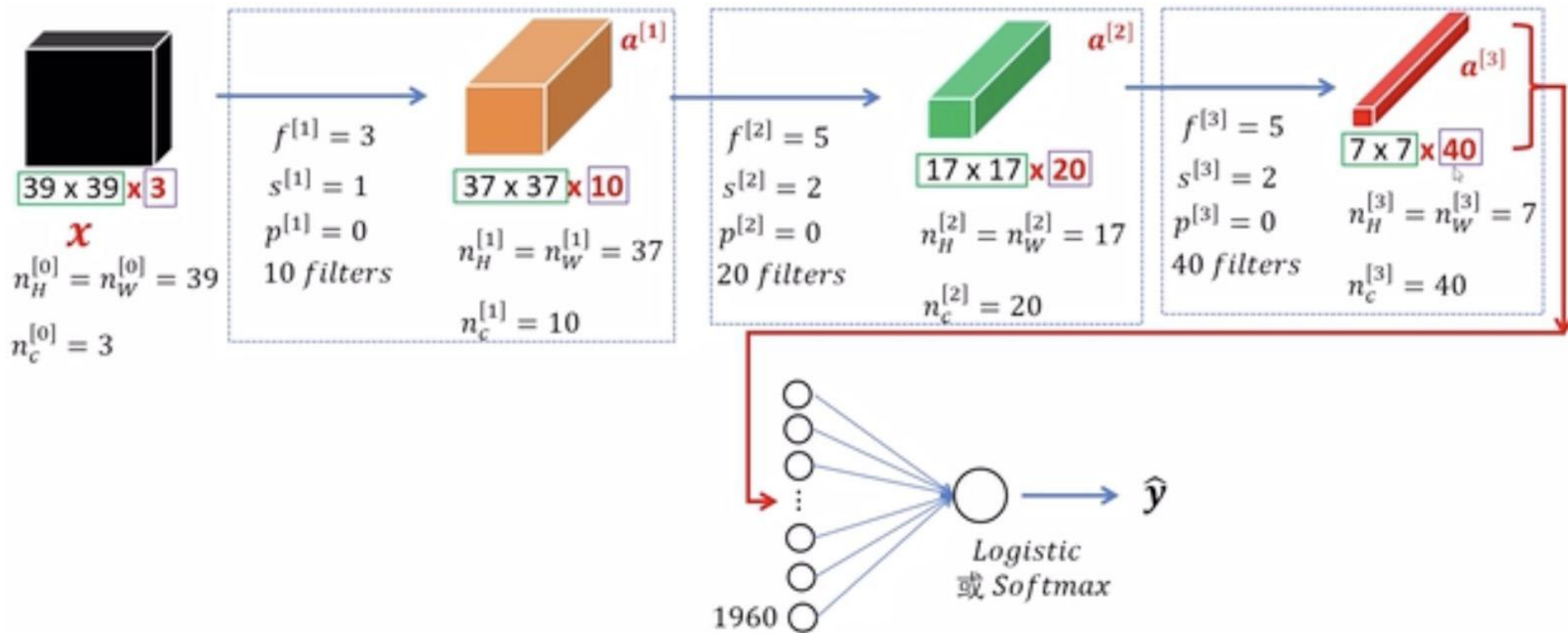
8.6 简单卷积网络

107



8.6 简单卷积网络

108



8.7 池化层

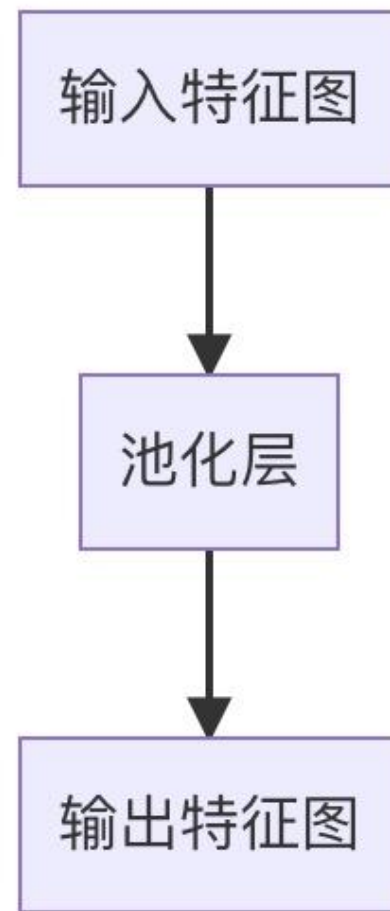
109



8.7 池化层

110

- 池化层的结构与卷积层类似，它也由多个滤波器组成，每个滤波器对输入数据进行卷积操作，得到一个输出特征图。
- 不同的是，池化层的卷积操作通常不使用权重参数，而是使用一种固定的池化函数，例如**最大池化**、**平均池化**等。
- 池化层主要用于减少数据量和计算复杂度，同时保留重要的特征信息。它可以提高模型的训练速度和泛化能力，避免过拟合等问题。

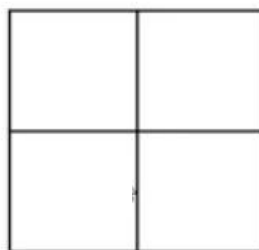


8.7 池化层

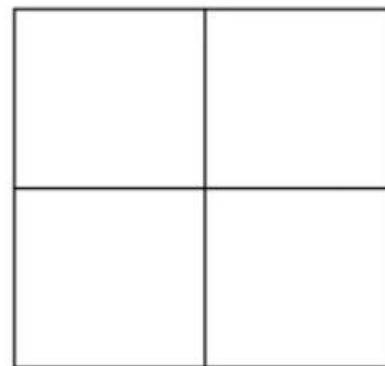
111

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

4 x 4 x 1



过滤器参数: $f = 2$
 $S = 2$



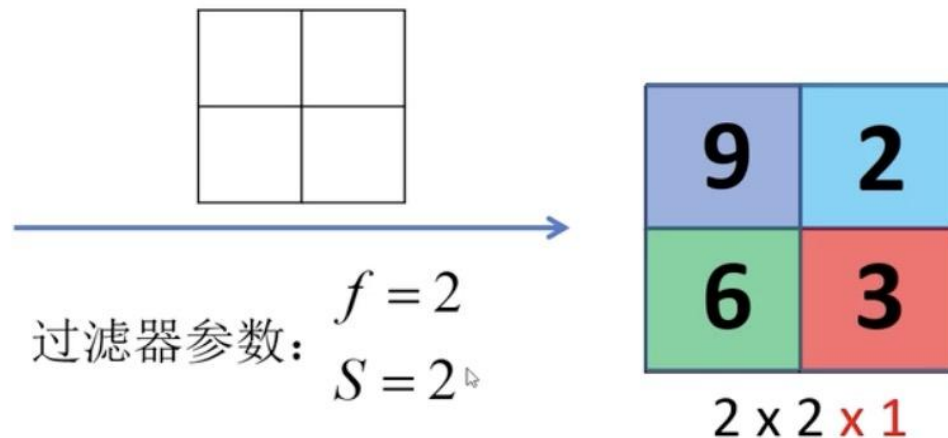
2 x 2 x 1

8.7 池化层

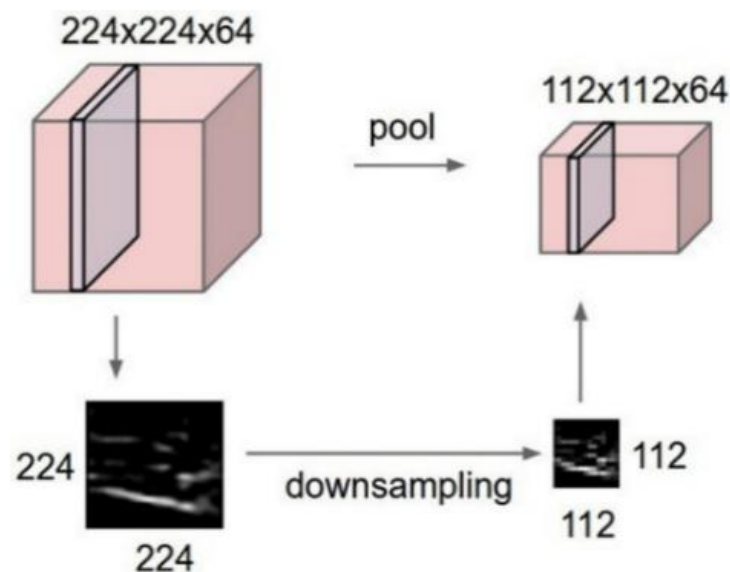
112

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

4 x 4 x 1



- (1) 过滤器参数是固定的，不需要在梯度下降过程中学习
- (2) 实践证明最大池化具备高效率的降维效果



8.7 池化层

113

1	3	2	1	3
2	9	1	1	5
1	3	2	3	2
8	3	5	1	0
5	6	1	2	9

5 x 5

Parameters:

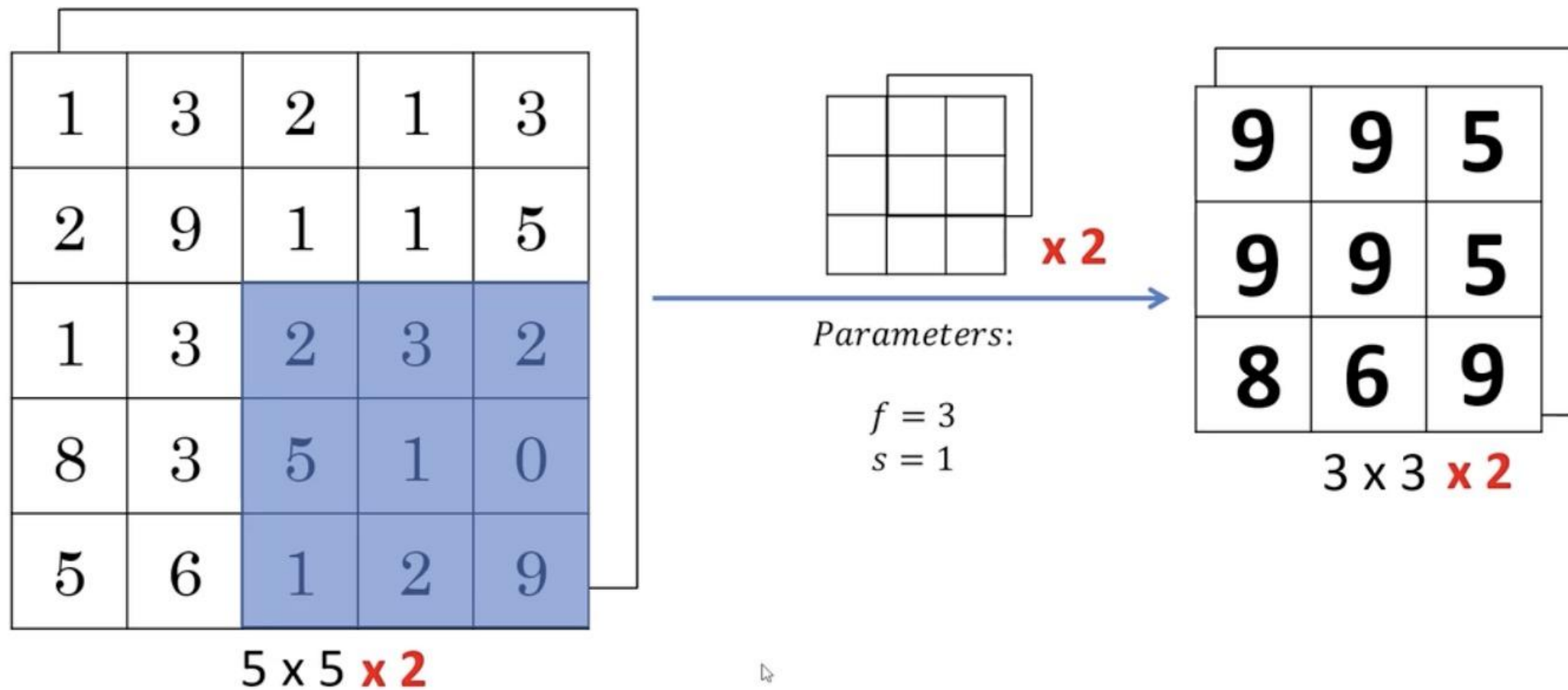
$$f = 3$$

$$s = 1$$

3 x 3

8.7 池化层

114

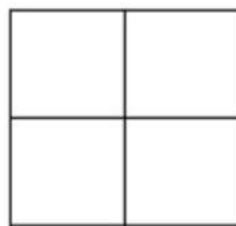


8.7 池化层

115

1	3	2	1
2	9	1	1
1	3	2	3
5	6	1	2

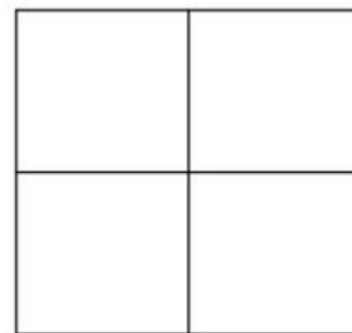
4 x 4



Parameters:

$$f = 2$$

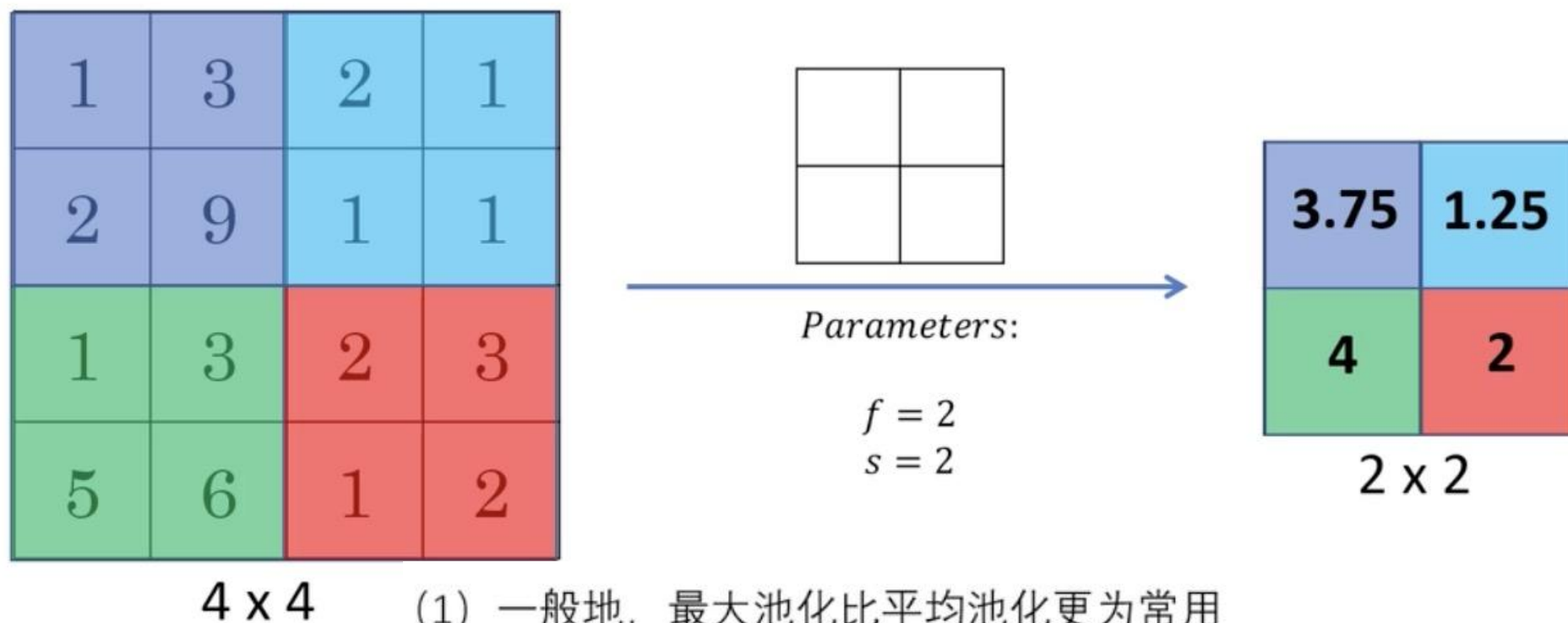
$$s = 2$$



2 x 2

8.7 池化层

116



(1) 一般地，最大池化比平均池化更为常用

(2) 也有例外，当**网络层数很深**时，用平均池化分解网络的表示层可能效果更好

例如： **7×7** $\times 1000 \rightarrow$ **1×1** $\times 1000$

8.7 池化层

117

f : filter的高和宽

s : stride

Max or Average Pooling

$$f = 2, \quad s = 2$$

目标图像高度和宽度各减一半

$$n_H \times n_W \times n_C$$

$$\left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor$$

8.8 经典卷积神经网络

118

卷积：

- ✓ 局部特征提取
- ✓ 训练中进行参数学习
- ✓ 每个卷积核提取特定模式的特征

池化（下采样）：

- ✓ 降低数据维度，避免过拟合
- ✓ 增强局部感受野
- ✓ 提高平移不变性

全连接：

- ✓ 特征提取到分类的桥梁

8.8 经典卷积神经网络

119

➤ 经典深度神经网络模型

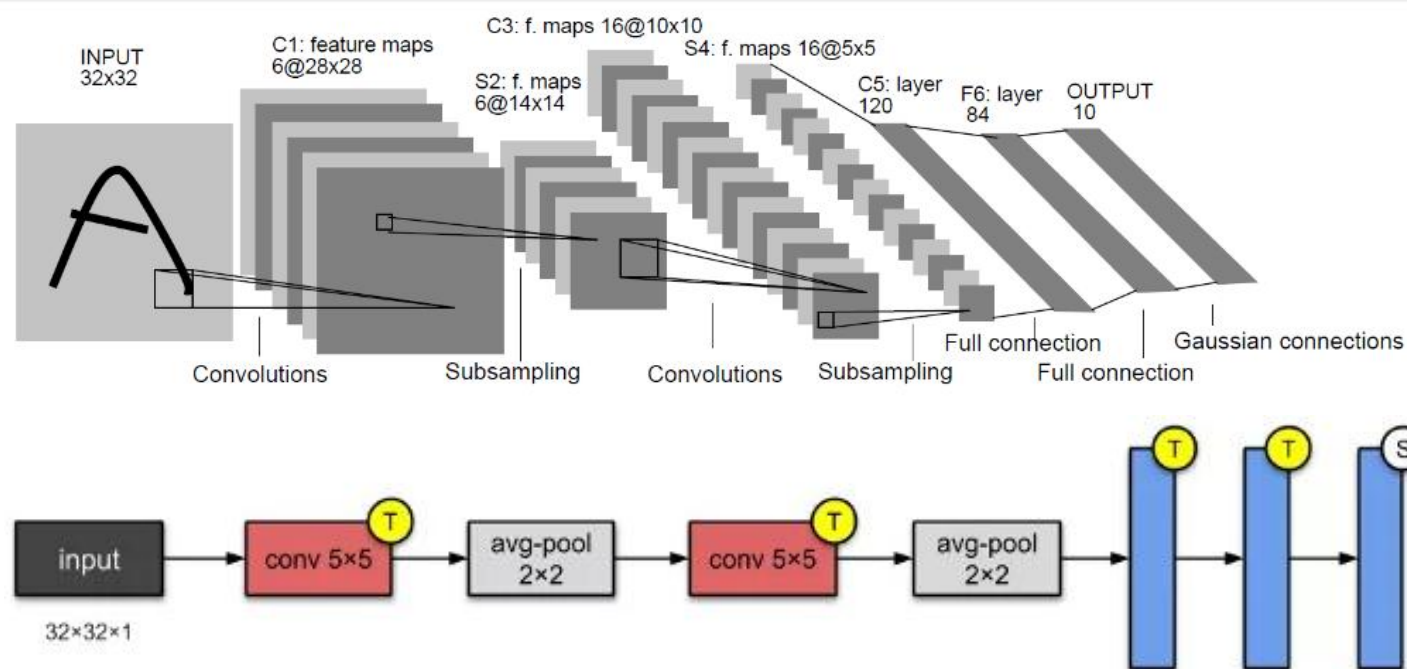
- LeNet
- AlexNet
- VGGNet
- ResNet

8.8 经典卷积神经网络

120

□ 8.8.1 LeNet-5模型

LeNet-5模型是1998年Yann LeCun教授提出的，是第一个成功应用于数字识别问题的卷积神经网络，在MNIST数据集上，LeNet模型识别的正确率高达99.2%。LeNet-5结构，如图所示。



8.8 经典卷积神经网络

层名称	输入大小	核大小	核个数	padding	stride	输出大小
Conv1	(3,32,32)	5*5	6	0	1	(6,28,28)
Pool1	(6,28,28)	2*2	None	0	2	(6,14,14)
Conv2		5*5	16	0	1	
Pool2		2*2	None	0	2	
FC1		120	None	None	None	
FC2		84	None	None	None	
FC3		10	None	None	None	

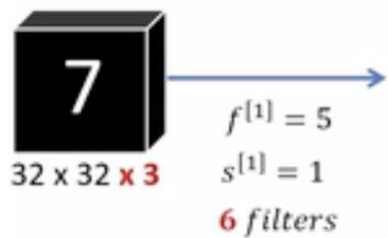
8.8 经典卷积神经网络

122

层名称	输入大小	核大小	核个数	padding	stride	输出大小
Conv1	(3,32,32)	5*5	6	0	1	(6,28,28)
Pool1	(6,28,28)	2*2	None	0	2	(6,14,14)
Conv2	(6,14,14)	5*5	16	0	1	(16,10,10)
Pool2	(16,10,10)	2*2	None	0	2	(16,5,5)
FC1	16*5*5	120	None	None	None	120
FC2	120	84	None	None	None	84
FC3	84	10	None	None	None	10

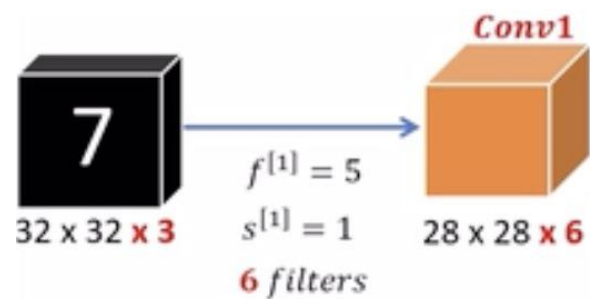
8.8 经典卷积神经网络

123



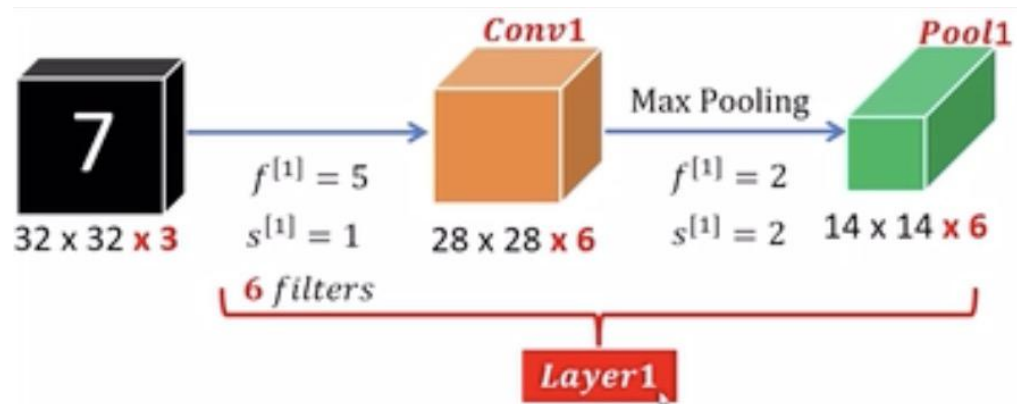
8.8 经典卷积神经网络

124



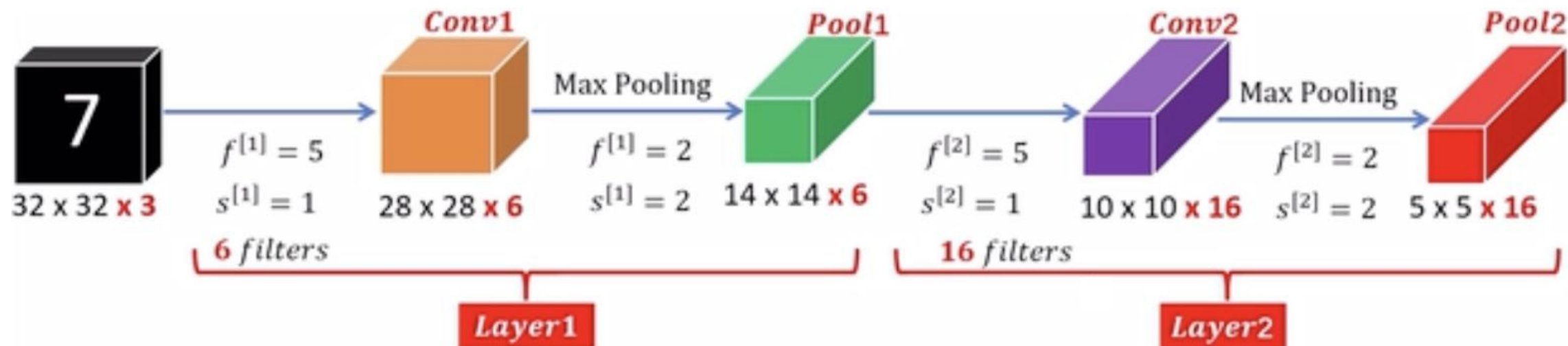
8.8 经典卷积神经网络

125



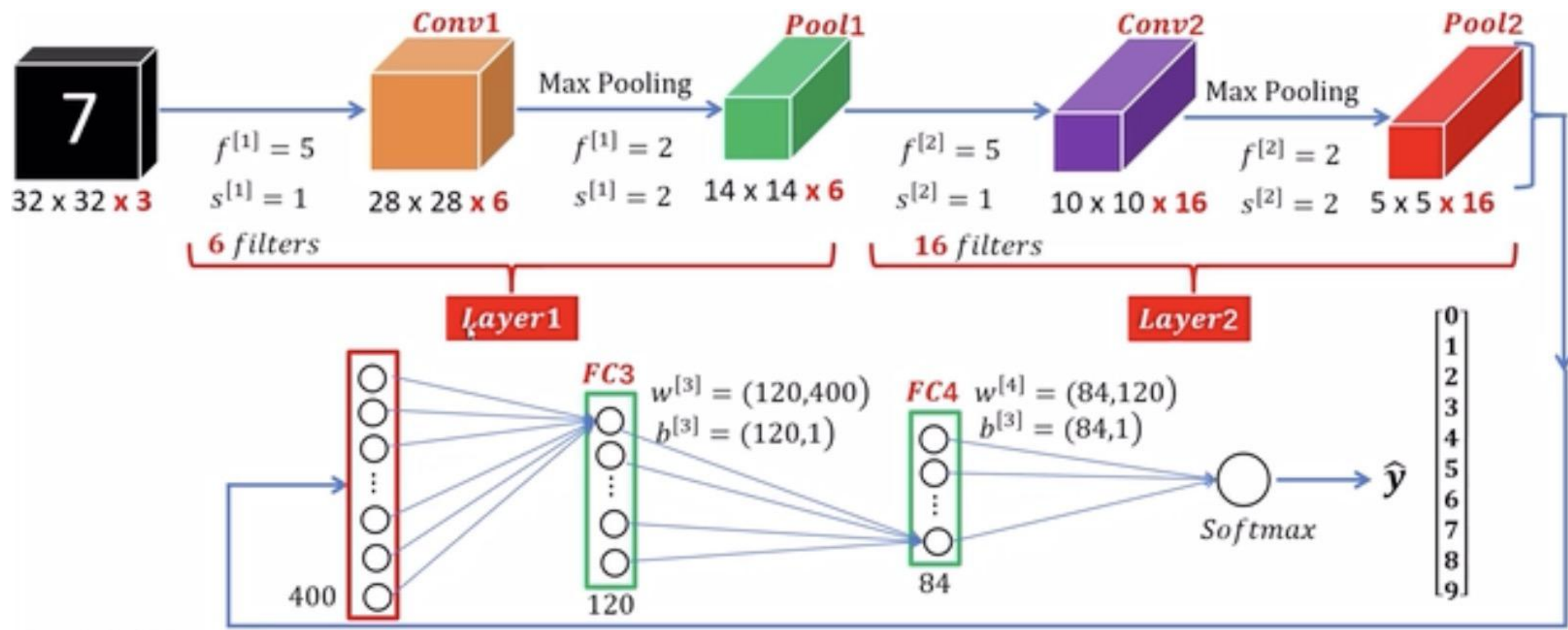
8.8 经典卷积神经网络

126



8.8 经典卷积神经网络

127



8.8 经典卷积神经网络

128

网络各层	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072	
CONV1 (f=5, s=1, filters=6)	(28,28,6)	4,704	
POOL1(f=2, s=2)	(14,14,6)	1,176	
CONV2 (f=5, s=1, filters=16)	(10,10,16)	1,600	
POOL2(f=2, s=2)	(5,5,16)	400	
FC3	(120,1)	120	
FC4	(84,1)	84	
Softmax	(10,1)	10	
合计			

8.8 经典卷积神经网络

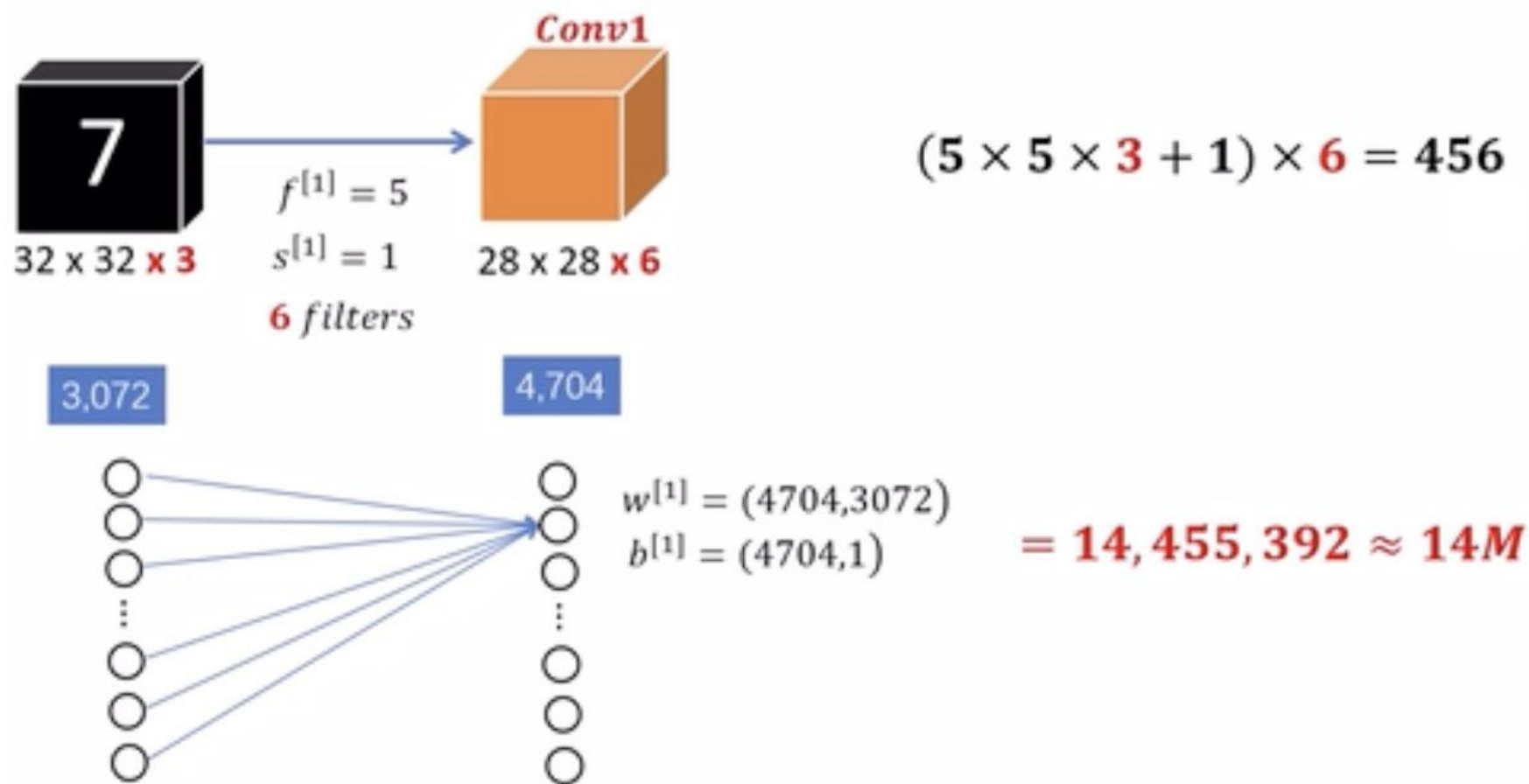
129

网络各层	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072	0
CONV1 (f=5, s=1, filters=6)	(28,28,6)	4,704	456
POOL1(f=2, s=2)	(14,14,6)	1,176	0
CONV2 (f=5, s=1, filters=16)	(10,10,16)	1,600	2,416
POOL2(f=2, s=2)	(5,5,16)	400	0
FC3	(120,1)	120	48,120
FC4	(84,1)	84	10,164
Softmax	(10,1)	10	850
合计			61,795

8.8 经典卷积神经网络

130

卷积神经网络和全连接网络的参数比较

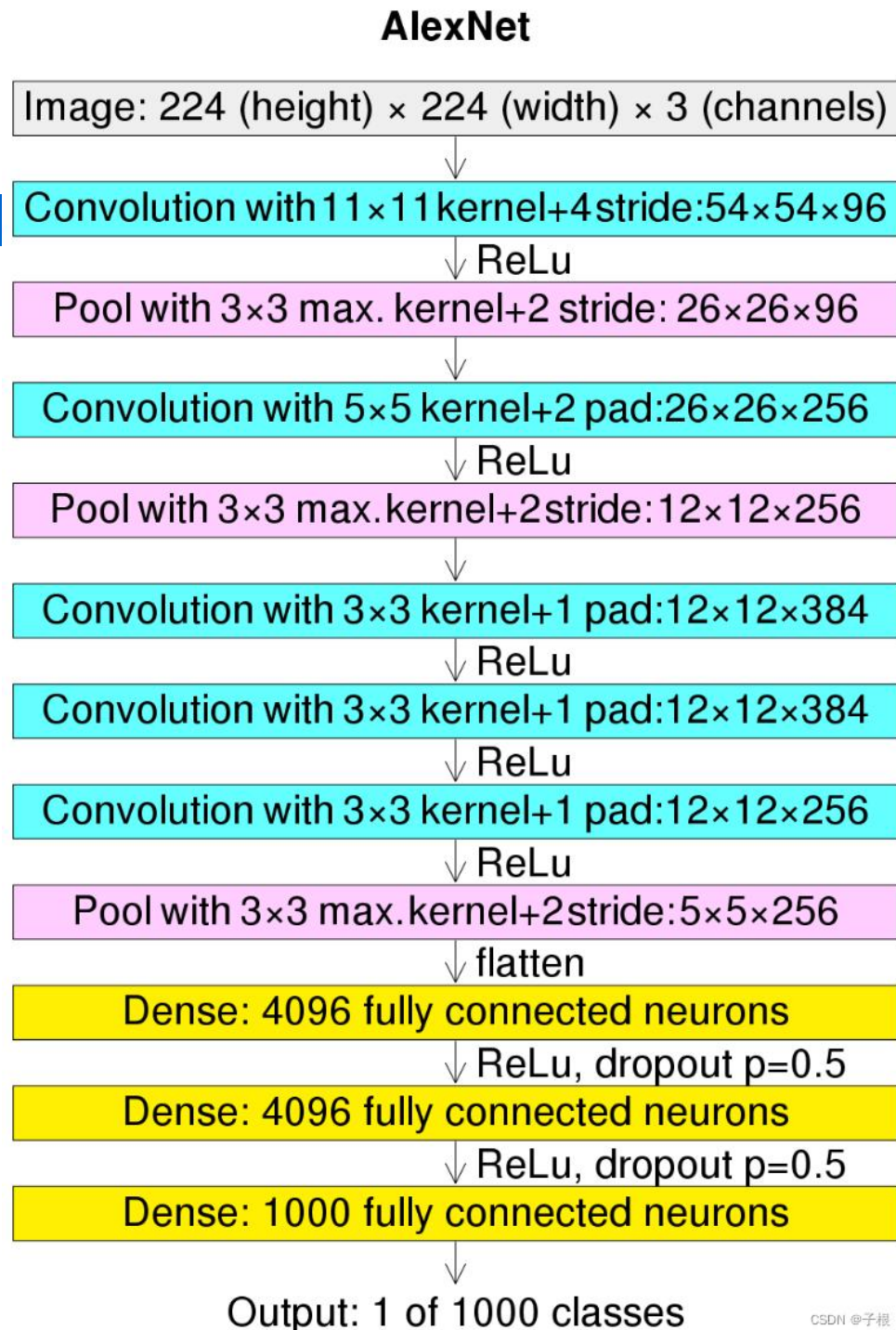


8.8 经典卷积神经网络

131

□ 8.8.2 AlexNet模型

- AlexNet网络在2012年提出，在ImageNet比赛当中以领先第二名10%精确度的成绩夺得冠军，相对于LeNet，层数更深，同时第一次引入了激活层ReLU，在全连接层引入了Dropout层防止过拟合。

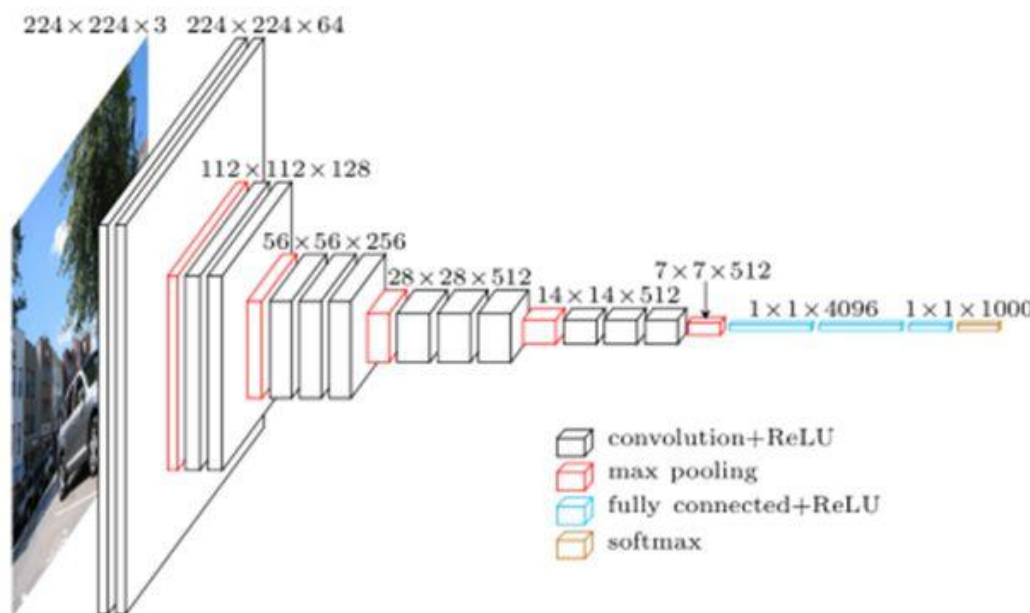


8.8 经典神经网络

132

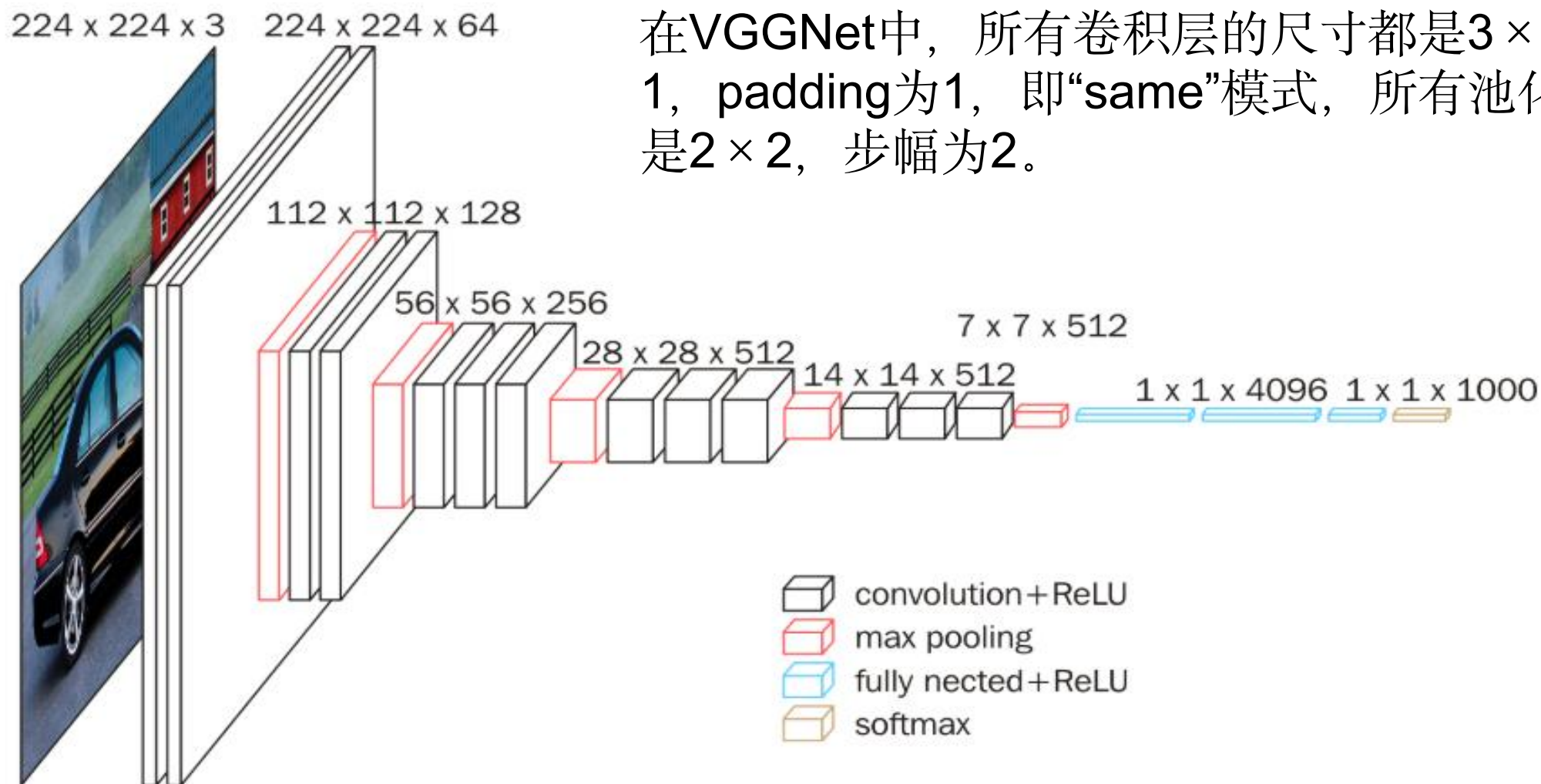
□ 8.8.3 VGGNet模型

VGGNet由牛津大学的视觉几何组（Visual Geometry Group）和Google DeepMind公司的研究员一起研发的深度卷积神经网络，其主要贡献是展示出网络的深度（depth），这是算法优良性能的关键部分。在此网络结构基础上，出现了ResNet（152-1000层）、GooleNet（22层）、VGGNet（19层）。到目前为止，VGGNet依然经常被用来提取图像特征。VGGNet16结构，如图所示。



8.8 经典神经网络

133



8.8 经典神经网络

134

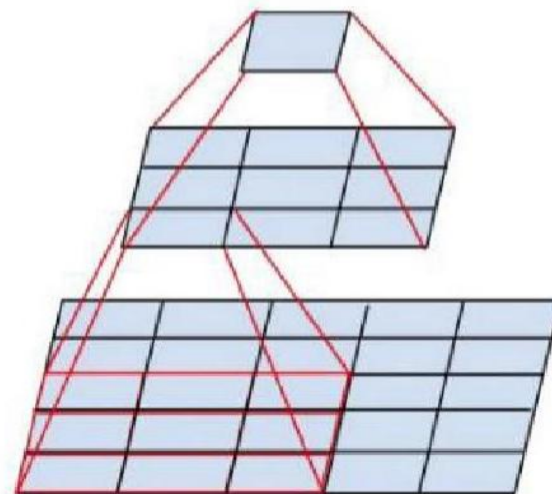
VGGNet

结构特点：

- 对卷积核和池化大小进行了统一。网络中进行 3×3 的卷积操作和 2×2 的最大池化操作。
- 采用卷积层堆叠的策略，将多个连续的卷积层构成卷积层组。

优点：

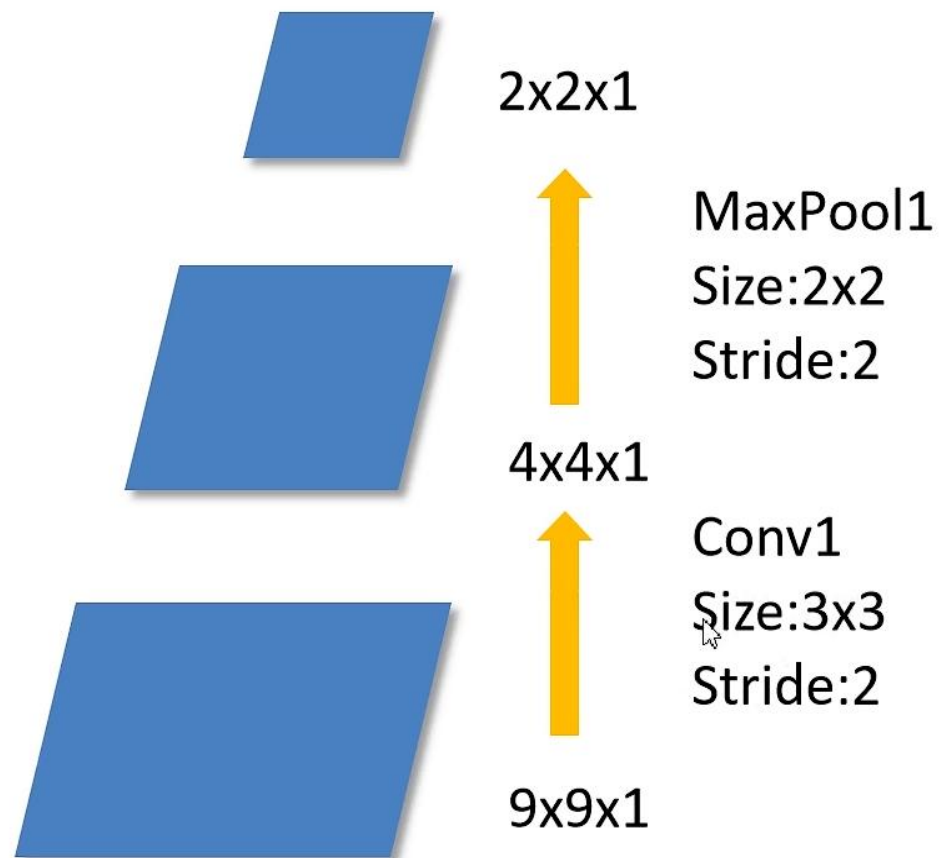
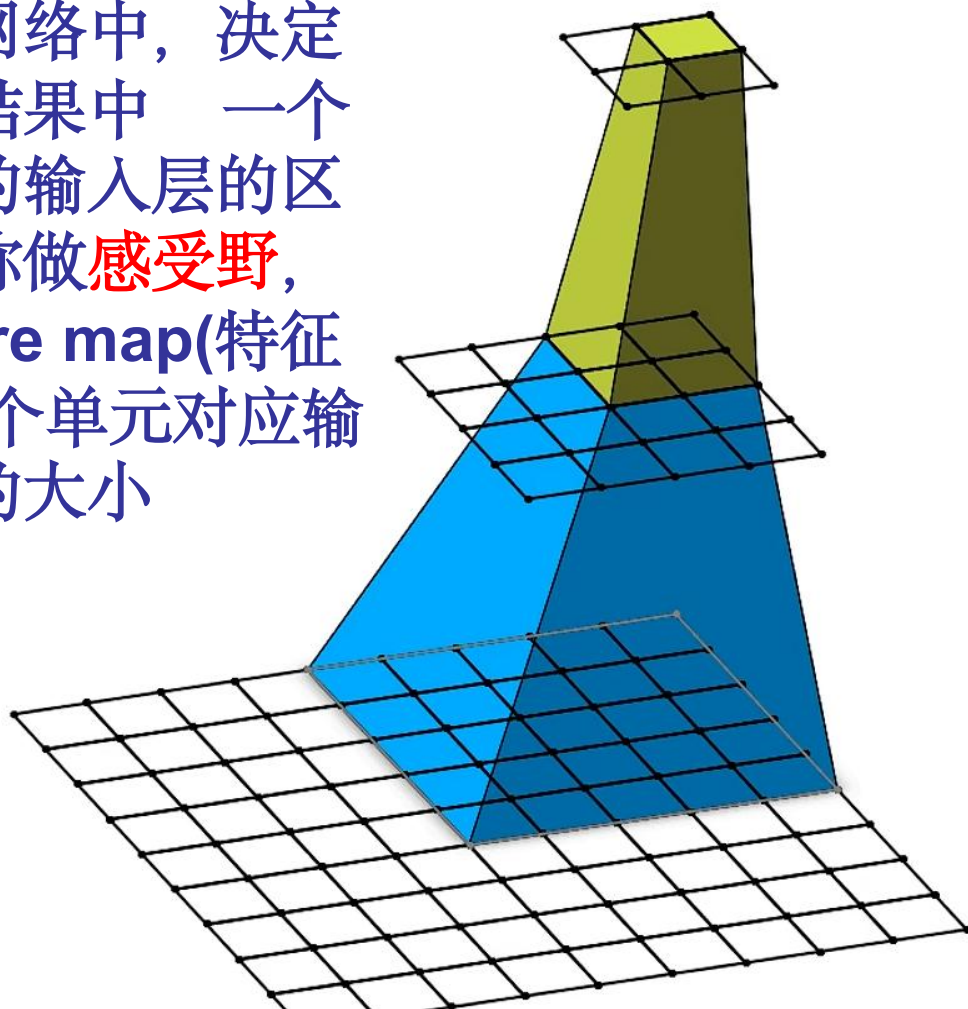
- 和单个卷积层相比，卷积组可以提高感受野范围，增强网络的学习能力和特征表达能力；
- 和具有较大核的卷积层相比，采用多个具有小卷积核的卷积层串联的方式能够减少网络参数；
- 另外，在每层卷积之后进行ReLU非线性操作可以进一步提升网络的特征学习能力。



8.8 经典神经网络

135

在卷积神经网络中，决定某一层输出结果中一个元素所对应的输入层的区域大小，被称做**感受野**，即输出**feature map**(特征矩阵)上的一个单元对应输入层上区域的大小

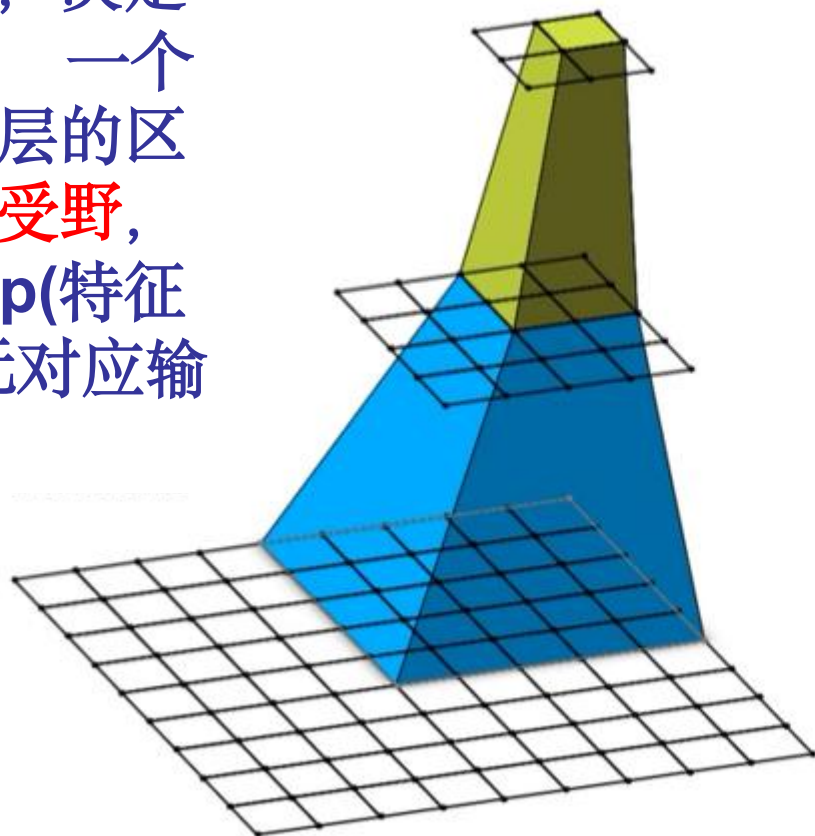


$$out_{size} = \lfloor (in_{size} - F_{size} + 2P) / S + 1 \rfloor$$

8.8 经典神经网络

136

在卷积神经网络中，决定某一层输出结果中一个元素所对应的输入层的区域大小，被称做**感受野**，即输出**feature map**(特征矩阵)上的一个单元对应输入层上区域的大小



感受野计算公式：

$$F(i) = (F(i+1) - 1) \times Stride + Ksize$$

$F(i)$ 为第 i 层感受野，
 $Stride$ 为第 i 层的步距，
 $Ksize$ 为卷积核或池化核尺寸

Feature map: $F = 1$

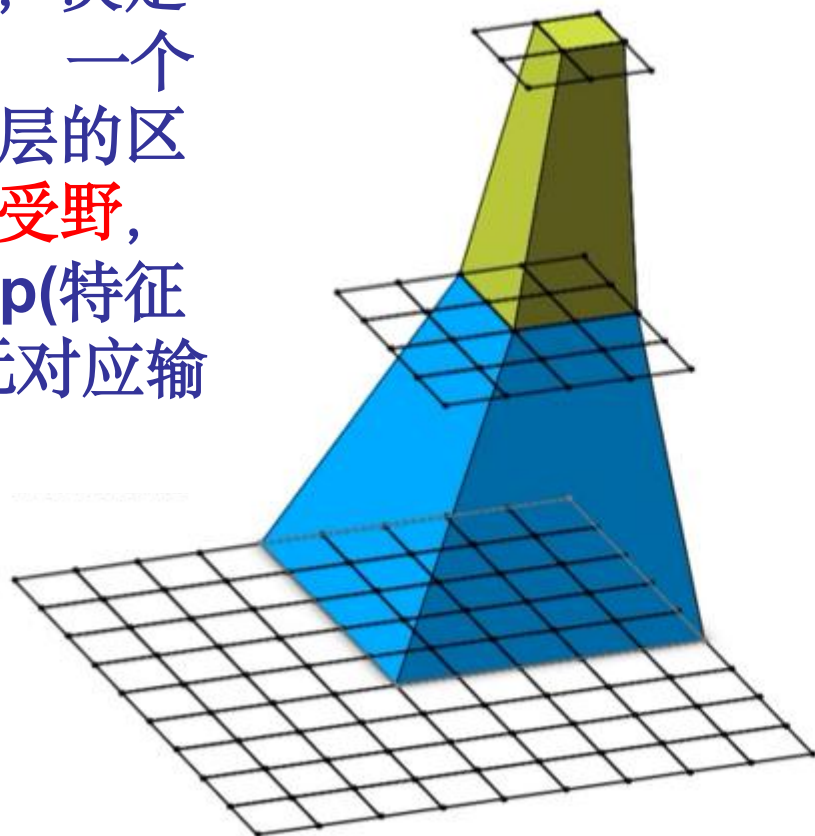
Pool1: **感受野大小？**

Conv1:

8.8 经典神经网络

137

在卷积神经网络中，决定某一层输出结果中一个元素所对应的输入层的区域大小，被称做**感受野**，即输出**feature map**(特征矩阵)上的一个单元对应输入层上区域的大小



感受野计算公式：

$$F(i) = (F(i+1) - 1) \times \text{Stride} + \text{Ksize}$$

$F(i)$ 为第 i 层感受野，
 Stride 为第 i 层的步距，
 Ksize 为卷积核或池化核尺寸

Feature map: $F = 1$

Pool1: $F = (1 - 1) \times 2 + 2 = 2$

Conv1: $F = (2 - 1) \times 2 + 3 = 5$

8.8 经典神经网络

138

- 堆叠2个 3×3 的卷积核 (stride=1) 可以替代一个? $\times$? 的卷积核?
- 堆叠3个呢?

*替代指感受野相同

8.8 经典神经网络

139

- 使用多个小卷积核的可以节省训练参数的个数、增加非线性变换。
- 计算一下使用 7×7 卷积核所需要的参数与堆叠三个 3×3 卷积核所需要的参数(假设输入和输出的通道数都是 C):

7×7 卷积核: $7 \times 7 \times C \times C = 49C^2$ (一个对应输入的通道数, 一个是输出的通道数, 卷积核个数)

3 个 3×3 卷积核: $3 \times 3 \times C \times C + 3 \times 3 \times C \times C + 3 \times 3 \times C \times C = 27C^2$

8.8 经典神经网络

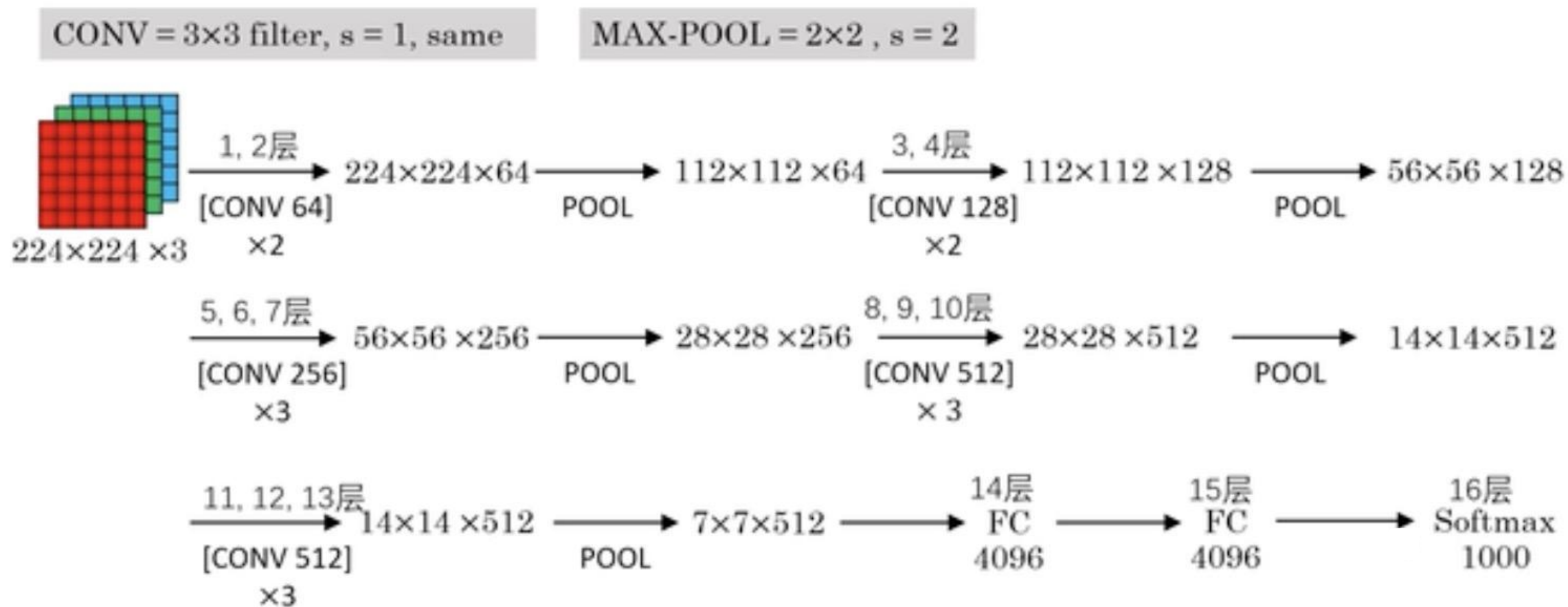
140

感受野的情况:

- 值越大: 它所能接触到的原始图像的范围越大, 意味着它可能蕴含更为全局、语义层次更高的特征
- 值越小: 它所包含的特征越趋向局部和细节
- 感受野的值可以大致判断每一层的抽象层次

8.8 经典神经网络

141



卷积块 (block) : 多个卷积层构成, 使网络有更大感受野的同时能降低网络参数

8.8 经典神经网络

142

在VGGNet中:

- 选择采用 3×3 的卷积核是因为 3×3 是最小的能够捕捉像素8邻域信息的尺寸。
- 2个 3×3 卷积堆叠等于1个 5×5 卷积，3个 3×3 堆叠等于1个 7×7 卷积，感受野大小不变，而采用更多层、更小的卷积核可以引入更多非线性（更多的隐藏层，从而带来更多非线性函数），提高决策函数判决力，并且带来更少参数。
- 每个VGG网络都有3个FC层，5个池化层，1个softmax层。
- 用得最多的是VGG16（13层conv + 3层FC）和VGG19（16层conv + 3层FC），注意算层数时不算maxpool层和softmax层，只算conv层和fc层。

8.8 经典神经网络

143

使用3x3小卷积核的好处有：

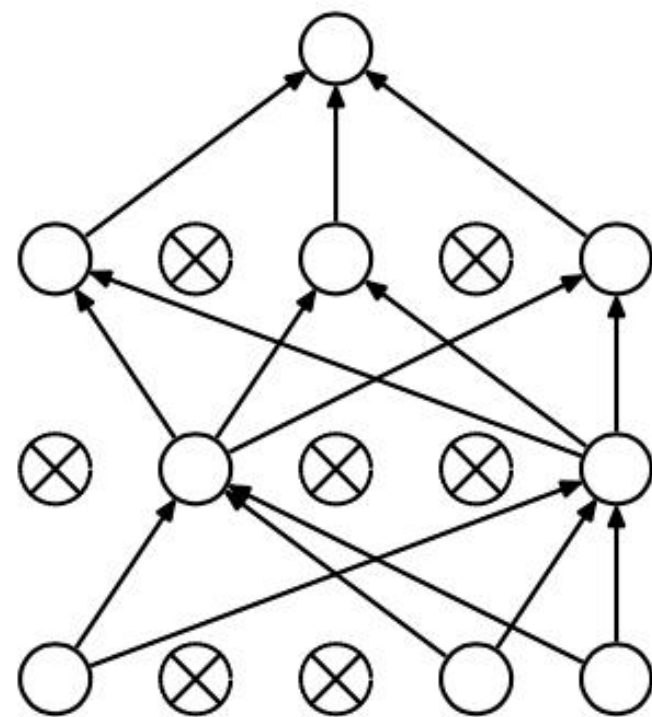
- 大幅度减少模型参数数量。
- 小卷积核选取小的 **stride**可以防止较大的**stride**导致细节信息的丢失。
- 多层卷积层（每个卷积层后都有非线性激活函数），增加非线性，提升模型性能。

8.8 经典神经网络

144

在VGGNet中:

- 第1、2层全连接层由4096个神经元组成。处理流程是: **FC-->ReLU-->Dropout**。
- 在FC层中间采用dropout层, 防止过拟合:
 - (1)、随机关闭网络中的一些隐藏神经元;
 - (2)、将输入通过修改后的网络进行前向传播, 然后将误差通过修改后的网络进行反向传播;
 - (3)、对于另外一批的训练样本, 重复上述操作(1)、(2)。



8.8 经典神经网络

145

深度学习，神经网络越深越好吗？

8.8 经典神经网络

146

深度学习，神经网络越深越好吗？

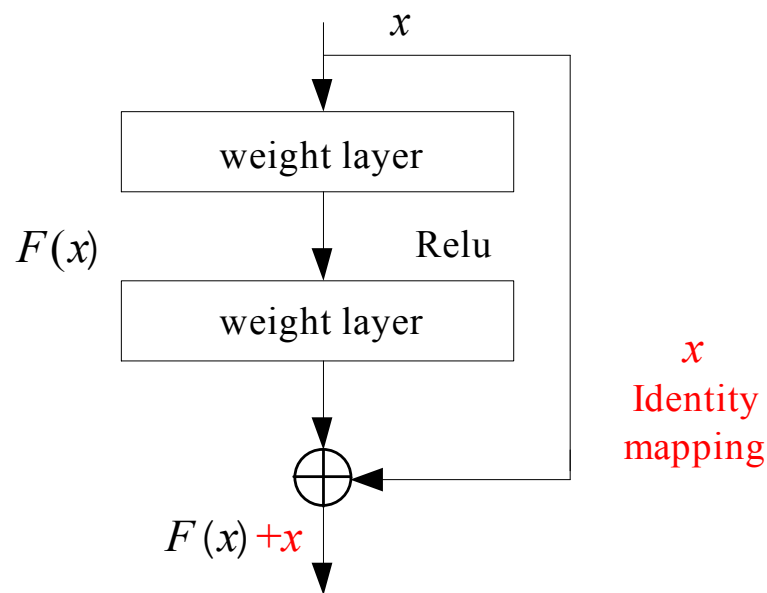
- 梯度消失、梯度爆炸
- 过拟合
- 退化
- 在反向传播过程中需要对激活函数进行求导，如果导数大于1，那么随着网络层数的增加梯度更新将会朝着指数爆炸的方式增加这就是**梯度爆炸**。
- 同样如果导数小于1，那么随着网络层数的增加梯度更新信息会朝着指数衰减的方式减少，这就是**梯度消失**。
- 因此，梯度消失、爆炸，其**根本原因在于反向传播链式法则**，属于先天不足

8.8 经典卷积神经网络

147

□ 8.8.4 ResNets模型

神经网络实际上是将一个空间维度向量 x ，经过非线性变换 $H(x)$ 映射到另外一个空间维度中。但 $H(x)$ 非常难以优化，所以转而求 $H(x)$ 的残差 $F(x)=H(x)-x$ 。假设求解 $F(x)=H(x)-x$ 比求 $H(x)$ 简单，就可以通过 $F(x)+x$ 来达到最终输出 x 的目标，如图所示。



8.8 经典卷积神经网络

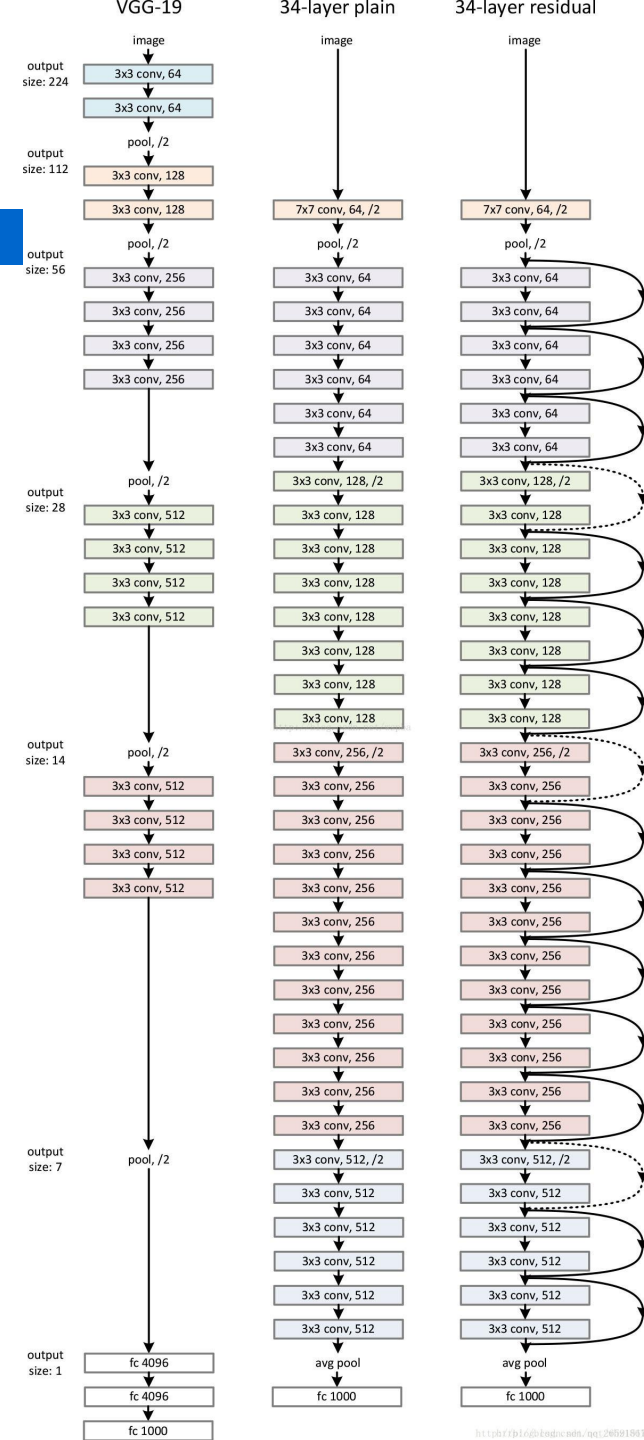
148

8.8.4 ResNet模型

右图就是一个残差网络和平整网络。**34-layer** 表示含可训练参数的层数为**34**层，池化层不含可训练参数。图左所示的残差网络和图中间所示的平整网络唯一的区别就是快捷连接。这两个网络都是当特征映射减半时，滤波器的个数翻倍，这样保证了每一层的计算复杂度一致。

ResNet因为使用恒等映射，在快捷连接上没有参数，所以图中平整网络和残差网络的计算复杂度都是一样的，都是**3.6 billion FLOPs**。

残差网络的引入使神经网络深度在尽可能加深的情况下，不会出现准确率下降等问题，广泛运用在计算机视觉任务中。

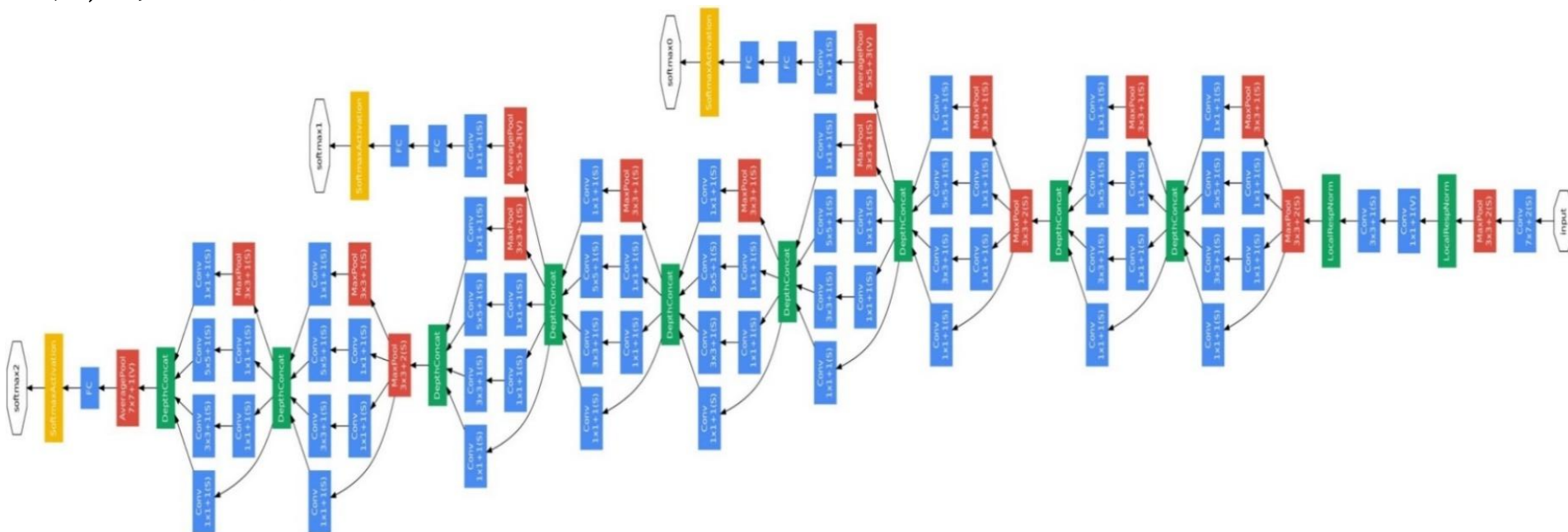


8.8 经典卷积神经网络

149

□ 8.8.5 GoogLeNet模型

在Inception网络中，最早的Inception v1版本就是非常著名的GoogLeNet，它由9个Inception v1模块和5个池化层以及其他一些卷积层和全连接层构成，总共为22层网络，如图所示。



8.8 经典卷积神经网络

150

□ 8.8.5 GoogLeNet模型

为了解决梯度消失问题，GoogLeNet在网络中间层引入两个辅助分类器来加强监督信息。在Inception网络中，比较有代表性的改进版本为Inception v3网络，它用多层的小卷积核来替换大的卷积核，减少了计算量和参数量，并保持感受野不变。具体包括：1) 使用两层 3×3 的卷积来代替Inception v1中的 5×5 的卷积；2) 使用连续的 $K\times 1$ 和 $1\times K$ 来代替 $K\times K$ 的卷积。此外，Inception v3网络同时也引入了标签平滑以及批归一化等优化方法进行训练。

Inception V2

2015年2月，Batch-normalized Inception (BN) 被引入作为Inception V2。

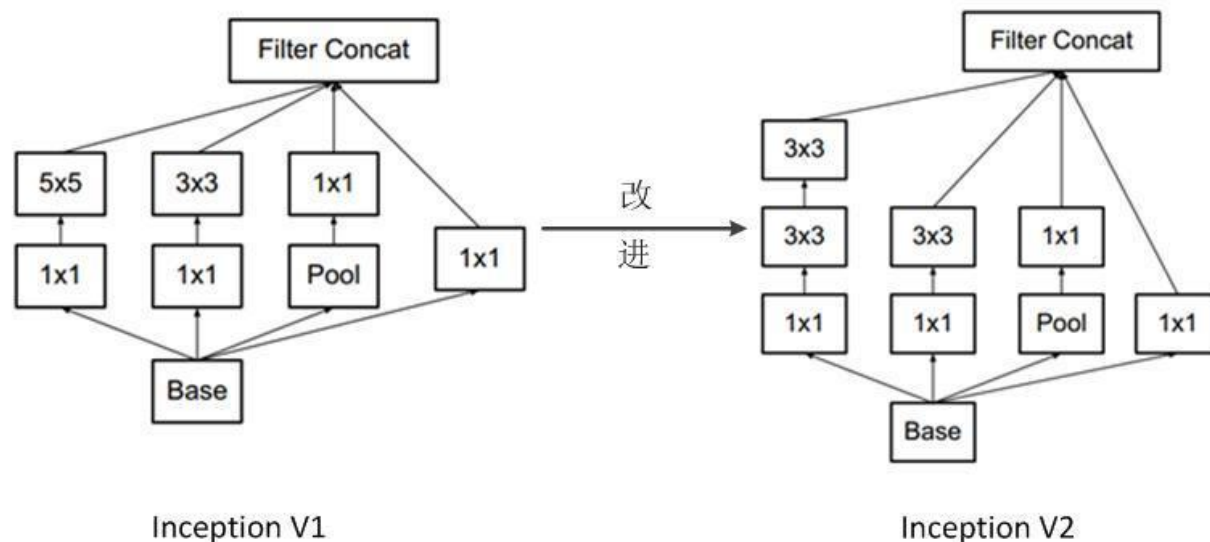
8.8 经典卷积神经网络

151

8.8.5 GoogLeNet模型

Batch-normalization在一层的输出上计算所有特征映射的均值和标准差，并使用这些值规范化其响应。这相当于数据增白（whitening），因此使得所有神经图（neural maps）在同样范围有响应，而且是零均值。在下一层不需要从输入数据中学习offset时，这有助于训练，还能重点关注如何最好的结合这些特征。

在Inception V2中，用两个 3×3 的卷积代替 5×5 的大卷积（用以降低参数量并减轻过拟合），如图所示。



练习题

152

假设输入是一张300x300彩色图像，第一个隐藏层使用了 100 个5*5卷积核做卷积操作，这个隐藏层有多少个参数（包括偏置参数）？

A.7601

B.7600

C. 7500

D.2501

练习题

153

有一个 $44 \times 44 \times 16$ 的输入，并使用大小为 5×5 的32个卷积核进行卷积，步长为1，无填充（nopadding），输出是多少？

A. $39 \times 39 \times 32$

B. $40 \times 40 \times 32$

C. $44 \times 44 \times 16$

D. $29 \times 29 \times 32$

练习题

154

假设某卷积层的输入和输出特征图大小分别为 $63*63*6$ 和 $31*31*12$ ，卷积核大小是 $5*5$ ，步长为2，那么Padding值为多少？

A.1

B.2

C.3

D.4

8.9 经典神经网络

155

ResNet18

```
import torchvision  
model = torchvision.models.resnet18(pretrained=False)  #不下载预训练权重  
print(model)
```

8.8 经典卷积神经网络

156

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision.datasets as datasets
import torchvision.transforms as transforms
```

8.8 经典卷积神经网络

157

```
class LeNet5(nn.Module):
    def __init__(self):
        super(LeNet5, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=6, kernel_size=5, stride=1)
        self.pool1 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5,
stride=1)
        self.pool2 = nn.AvgPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(in_features=16 * 5 * 5, out_features=120)
        self.fc2 = nn.Linear(in_features=120, out_features=84)
        self.fc3 = nn.Linear(in_features=84, out_features=10)
```

8.8 经典卷积神经网络

158

```
def forward(self, x):  
    x = self.pool1(torch.relu(self.conv1(x)))  
    x = self.pool2(torch.relu(self.conv2(x)))  
    x = x.view(-1, 16 * 4 * 4)  
    x = torch.relu(self.fc1(x))  
    x = torch.relu(self.fc2(x))  
    x = self.fc3(x)  
    return x
```


8.8 经典卷积神经网络

159

```
# 加载MNIST数据集
```

```
train_dataset = datasets.MNIST(root='./data', train=True, transform=transforms.ToTensor(),  
download=True)
```

```
test_dataset = datasets.MNIST(root='./data', train=False, transform=transforms.ToTensor())
```

```
# 定义数据加载器
```

```
train_loader = torch.utils.data.DataLoader(dataset=train_dataset, batch_size=64, shuffle=True)
```

```
test_loader = torch.utils.data.DataLoader(dataset=test_dataset, batch_size=64, shuffle=False)
```

8.8 经典卷积神经网络

160

```
# 定义模型、损失函数和优化器  
model = LeNet5()  
criterion = nn.CrossEntropyLoss()  
optimizer = optim.Adam(model.parameters())
```

8.8 经典卷积神经网络

161

训练模型

```
for epoch in range(10):
```

```
    for i, (images, labels) in enumerate(train_loader):
```

```
        optimizer.zero_grad()
```

```
        outputs = model(images)
```

```
        loss = criterion(outputs, labels)
```

```
        loss.backward()
```

```
        optimizer.step()
```

```
    if (i+1) % 100 == 0:
```

```
        print('Epoch [{} / {}], Step [{} / {}], Loss: {:.4f}'.format(epoch+1, 10, i+1,
len(train_loader), loss.item()))
```